ParBench: A Benchmark for Reliable Evaluation of LLM Parallel Code Translation

Anonymous Author(s)
Affiliation
Address
email

Abstract

Modern compute-intensive software is written against a rapidly changing ecosystem of accelerators, programming APIs, compiler stacks, and portability layers. As hardware and software platforms evolve, kernels in AI, scientific computing, simulation, graphics, and data-intensive workloads often need to migrate across CUDA, OpenMP, OpenCL, OpenMP target offload, and related parallel APIs. Large language models and autonomous coding agents are increasingly proposed as tools for this migration, but the field still lacks a reliable way to measure whether they can preserve the low-level parallel semantics that make such translations correct, including thread indexing, synchronization, memory management, host–device coordination, and API-specific execution structure.

We present PARBENCH, a kernel-centric benchmark framework designed to isolate and measure LLM-based parallel API translation under executable, reproducible conditions. Unlike general code benchmarks that are largely sequential, parallel-code benchmarks that emphasize generation from natural-language prompts, or repository-level studies that confound translation with build-system reconstruction, PARBENCH fixes the surrounding build, run, and verification infrastructure through declarative benchmark specifications and asks models to translate only the computational kernels. The benchmark draws on multiple open-source HPC suites to cover representative cross-API translation directions among CUDA, OpenMP, OpenCL, and OpenMP target offload. To probe whether success reflects robust translation rather than surface-form memorization, PARBENCH also includes AST-driven semantics-preserving source augmentation. We further evaluate PARBENCH on state-of-the-art open and proprietary LLMs, showing that it exposes persistent barriers to reliable parallel code translation, including direction asymmetry, multi-file coordination, incomplete API adaptation, and uneven robustness to source-level perturbations.

1 Introduction

Modern compute-intensive software is written against a rapidly changing ecosystem of accelerators, programming APIs, compiler stacks, and portability layers. Kernels in AI systems, scientific computing, simulation, graphics, and data-intensive workloads often need to move across CUDA, OpenMP, OpenCL, OpenMP target offload, and related parallel APIs as hardware platforms evolve or deployment requirements change. Large language models (LLMs) and autonomous coding agents are increasingly being explored for this migration task, but parallel API translation is fundamentally different from ordinary code translation. Correctness depends not only on preserving program intent, but also on maintaining thread-index arithmetic, synchronization semantics, memory-management patterns, host–device coordination, and API-specific execution structure—properties that go far beyond surface-level token replacement.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

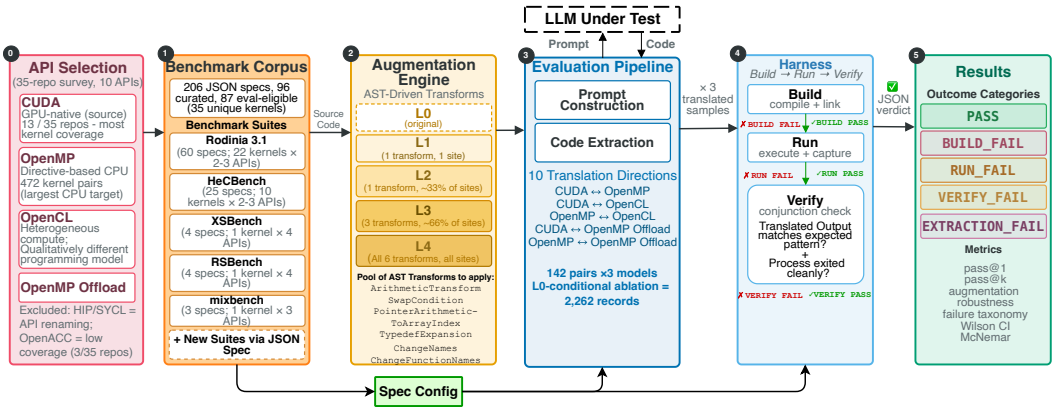

Figure 1: **PARBENCH isolates parallel API translation from repository-level confounds.** The benchmark pipeline starts by selecting representative parallel APIs and curating executable kernel specifications from multi-API HPC suites. Each specification fixes the build, run, and verification context, while the model is asked to translate only the computational kernel code. Semantics-preserving AST augmentations create surface-varied source variants for robustness testing, and translated outputs are evaluated through an end-to-end build–run–verify harness with conjunctive correctness checks. The final outcome taxonomy separates successful translations from extraction, build, runtime, and verification failures, enabling both aggregate scoring and diagnostic analysis of where parallel translation breaks.

Can current models actually perform this translation reliably? Existing evaluations leave this question unresolved because they do not isolate parallel API translation as the measured capability. Widely adopted code-generation benchmarks such as HumanEval and SWE-bench Chen et al. [2021], Jimenez et al. [2024] are predominantly sequential and therefore do not exercise parallel programming semantics. Parallel code-generation benchmarks such as ParEval Nichols et al. [2024a] primarily evaluate synthesis from natural-language descriptions, rather than translation of existing implementations across APIs. Repository-level translation work exposes important integration challenges, but it also entangles kernel translation with build-system reconstruction, file organization, dependency management, and execution setup Davis et al. [2025], Wang et al. [2026][1]. This leaves two core questions open: can models translate computational kernels across a broad set of parallel API directions when the surrounding infrastructure is fixed, and do successful translations reflect robust parallel-code translation rather than memorization of widely circulated benchmark code?

We present PARBENCH, a benchmark framework designed to answer these questions by isolating parallel kernel translation from repository reconstruction. When evaluating parallel code translation, build-system configuration, file scope, run arguments, and verification criteria vary across kernels and APIs; failures in any of these can confound assessment of translation quality itself. PARBENCH fixes the surrounding build, run, and verification infrastructure through declarative benchmark specifications, asks models to translate only computational kernel files, and evaluates each generated kernel with an end-to-end build-run-verify harness using conjunctive correctness checks. The corpus is survey-grounded: we screened 71 open-source HPC repositories spanning 29 parallel programming APIs and retained 35 that met inclusion criteria for kernel-level co-occurrence analysis, selecting five benchmark suites that provide equivalent multi-API kernels and automated correctness checks (Section 3.1; Appendix Figure 14). To test whether successful translations reflect robust translation rather than surface-form memorization, PARBENCH includes an AST-driven augmentation engine that generates semantically equivalent but syntactically novel source variants.

This paper makes three contributions:

- **Parallel code translation benchmark.** We introduce PARBENCH, a benchmark for LLM-based parallel code translation that provides executable kernel specifications spanning five HPC

[1] An extended comparison with code-generation benchmarks, repository-level translation studies, HPC-specific LLM evaluation, and robustness benchmarks in Appendix A.

benchmark suites, six standard translation directions among CUDA, OpenMP, and OpenCL, and four OpenMP-target case-study directions. By fixing the surrounding build infrastructure as context, PARBENCH enables direct build-run-verify evaluation of kernel translation without the build-system-generation confound identified by repository-level work.

- **AST-driven augmentation engine.** We develop an augmentation engine with six semantics-preserving source-level transformations applied at four intensity levels (L1–L4). The engine produces structurally varied but functionally equivalent source variants, allowing us to test whether models preserve parallel semantics beyond surface similarity to widely circulated benchmark code.
- **Empirical characterization.** Across three models and 2,262 valid translation records, we report a multi-suite, multi-direction evaluation of LLM-based parallel code translation under stochastic sampling with pass@k analysis. Pass@1 ranges from 23.9% (Qwen 3.5) to 62.7% (GPT-5.4 and GPT-5.3-codex) on 142 unique no-augmentation (L0) tasks, with the code-specialized GPT-5.3-codex statistically indistinguishable from the general-purpose GPT-5.4. Key findings include build-stage API adaptation as the main bottleneck, strong direction and file-boundary effects, broad stability under augmentation, and limited gains from repeated sampling.

2 PARBENCH Framework

To better evaluate LLM-based translation of parallel programming APIs, PARBENCH defines a controlled, kernel-centric setting that separates parallel kernel translation from repository-level challenges such as build-system reconstruction. The framework realizes this setting through four components: (1) declarative specifications, (2) an AST-driven augmentation engine, (3) an evaluation pipeline that orchestrates LLM translation and records diagnostic failure categories, and (4) a build-run-verify harness. Together, these components enable direct measurement of parallel API translation capability under end-to-end executability constraints while supporting controlled robustness analysis. Figure 1 shows the overall architecture.

2.1 Declarative Specifications and Kernel-Centric Translation

Each benchmark instance in PARBENCH is encoded as a JSON specification (spec) that serves as a declarative contract defining what "correct execution" means for that kernel–API variant. The spec encodes all information required to build, run, and verify a benchmark without manual intervention, enabling fully automated evaluation. The schema is formally defined in JSON Schema and enforced by an offline validator. A spec declares the kernel's identity, specifies the target API, and includes provenance details to ensure reproducibility by including the pinned repository and commit. The spec partitions source files into `prompt_payload` (files provided to the LLM during translation), `support_files` (build infrastructure such as Makefiles and shared headers, provided as read-only reference), and `verification_only` (reference implementations never provided to the LLM). This field is central to the kernel-centric translation methodology described in Section 2.2. An example spec and the full structure are provided in Appendix C.1. **Build, run, and verification.** The `build` block specifies compiler commands, environment variables, and expected executable path. The `run` block defines input configurations with arguments and timeouts. The `verification` block declares an ordered list of verification strategies evaluated in conjunction. All must pass for an overall PASS verdict (Section 2.4). Strategy types range from `exit_code` and `stdout_pattern` checks to quantitative `numeric_comparison` and `file_hash` verification (Section 2.4).

Baseline results. The `baseline_results` block records the expected output from running the original implementation on a reference platform, providing ground truth for correctness evaluation.

The evaluation corpus comprises 96 specs spanning five benchmark suites: 60 from Rodinia Che et al. [2009], 4 from XSBench Tramm et al. [2014], 4 from RSBench, 3 from mixbench, and 25 from a curated subset of HeCBench Jin and Vetter [2023]. Of these, 87 pass the baseline build-run-verify pipeline; the remaining 9 (KNOWN_FAIL specs) fail due to platform-specific issues, such as CUDA 12 API deprecations, missing system libraries, or pre-existing runtime errors, and are excluded from evaluation (Section 3). Listing 1 in Appendix C illustrates the spec structure using Rodinia's BFS kernel.

2.2 Evaluation Pipeline: Kernel-Centric Translation

The key design choice is **kernel-centric translation**: the LLM is asked to modify only the `translation_targets` files and produce functionally equivalent target-API code, while Makefiles, host scaffolding, and verification logic remain fixed. This isolates parallel API translation from repository reconstruction—the confound that drives ParEval-Repo's 0% pass rate on applications above 133 SLoC Davis et al. [2025]. The evaluation pipeline constructs a structured prompt containing the source kernel code, the target API and output filenames, the build command, and read-only target infrastructure context, with prompt anonymization to reduce benchmark identity leakage. For cross-API translations, the pipeline handles argument and verification asymmetry automatically: full-program translations use a union of source and target verification patterns, whereas kernel-only translations, such as OpenCL `.cl` files, use the target spec directly. This design isolates evaluation of whether an LLM can correctly map parallel programming patterns from one API to another, independent of its ability to replicate project file organization. The full prompt template is in Appendix H.

Each LLM response is parsed to recover the translated source files using a multi-tier extraction strategy (Appendix C) that progressively relaxes matching criteria from explicit filename annotations to fuzzy and elimination-based heuristics. If any expected target file cannot be recovered or is empty, the task is classified as EXTRACT_FAIL. To enable stratified analysis of translation difficulty, each source–target pair is classified by cardinality of the translation: `single_file` ($1source \rightarrow 1target$), `multi_to_single` ($N \rightarrow 1$), `single_to_multi` ($1 \rightarrow N$), or `multi_to_multi` ($N \rightarrow M$).

2.3 Augmentation Engine: Probing Robustness via Code Perturbations

Widely used suites such as Rodinia Che et al. [2009] are likely present in LLM training data, so success on unmodified source may reflect memorization. The augmentation engine generates semantically equivalent but syntactically novel variants by applying six AST-level transforms backed by the `libclang` API: condition operand swapping (e.g., `a < b` $\rightarrow$ `b >= a`), arithmetic form conversion (e.g., `x += 1` $\leftrightarrow$ `x = x + 1`), scope-aware variable renaming, typedef expansion, pointer-to-array notation conversion (e.g., `*(arr + i)` $\rightarrow$ `arr[i]`), and static function renaming. All transforms preserve parallel structure (thread indexing, synchronization, memory management) while changing surface-level code features most likely to be memorized verbatim. Parallelism-aware transforms (e.g., loop reordering) are excluded because they risk altering observable output.

Five augmentation levels (L0–L4) control the density of applied transforms. L0 is unmodified source. L1 applies each applicable transform to a single candidate site. L2–L4 apply each transform to an increasing fraction of candidate sites (f=0.33, 0.66, 1.0). Baseline validation confirms semantic preservation: all 87 non-KNOWN_FAIL specs pass at L1–L2, and 80 of 87 pass at all levels through L4. The 7 high-intensity failures occur exclusively in `omp_target` variants, where condition swapping (e.g., `a < b` $\rightarrow$ `b > a`) causes the GPU-offloading compiler to generate device code that produces different numerical output under strict verification. Variable renaming is already disabled for files containing OpenMP pragmas. Level definitions and per-transform details are in Appendix Table 8 and Appendix F.1.

2.4 Harness: Build, Run, Verify

The harness is a three-stage pipeline where failure at any stage short-circuits subsequent stages. The **build stage** resolves the spec's working directory, substitutes platform-specific paths, and invokes compilation. The **run stage** executes the compiled binary with spec-defined arguments and timeout. The **verify stage** applies an ordered list of verification strategies in conjunction—all must pass for a PASS verdict. Five strategy types are implemented: `exit_code`, `stdout_pattern`, `stdout_exclude_pattern`, `numeric_comparison`, and `file_hash` (Section 2.2).

Conjunctive verification is essential. For example, a CUDA translation of Gaussian elimination compiles, runs to completion with exit code zero, and produces partial stdout—but omits the expected timing summary. This indicates a systematic translation defect rather than a transient failure. Compile-only TehraniJamsaz et al. [2024] or exit-code-only verification would misclassify this as successful. Each outcome receives one of four failure classifications—BUILD_FAIL, RUN_FAIL, VERIFY_FAIL, or EXTRACT_FAIL—or PASS, enabling systematic failure-mode analysis (Section 5.4).

Table 1: Evaluation corpus. SLoC is measured on the CUDA source variant when available. Multi-File denotes specs requiring more than one translated source file. KF = KNOWN_FAIL.

Suite	Kernels	Specs	PASS	KF	APIs	SLoC Range	Med.	Multi-File
Rodinia	22	60	53	7	CUDA, OMP, OCL	195–3,304	334	21/60 (35%)
HeCBench	10	25	23	2	CUDA, OMP, OMP-tgt	80–235	180	0/25 (0%)
XSBench	1	4	4	0	CUDA, OMP, OCL, OMP-tgt	1,390	–	2/4 (50%)
RSBench	1	4	4	0	CUDA, OMP, OCL, OMP-tgt	1,016	–	1/4 (25%)
mixbench	1	3	3	0	CUDA, OMP, OCL	312	–	0/3 (0%)
Total	**35**	**96**	**87**	**9**		**80–3,304**	**271**	**24/96 (25%)**

3 Benchmark Curation

The benchmark corpus was assembled through a four-stage systematic process: surveying open-source HPC benchmark repositories, quantifying kernel-level translation opportunities across parallel APIs, filtering candidate kernels through automated build-run-verify checks, and verifying each selected kernel through the complete pipeline on the evaluation platform. This curation populates the Source Code component of the PARBENCH architecture (Figure 1, orange) with kernel files from five benchmark suites spanning four parallel APIs—CUDA, OpenMP, OpenCL, and OpenMP target offload—with the first three serving as standard translation directions and OMP-target as a case-study extension (Section 4).

3.1 Suite and Kernel Selection

We screened 71 open-source HPC repositories spanning benchmark suites, mini-applications, proxy applications, libraries, and microbenchmarks across 29 parallel programming APIs, retaining 35 that met inclusion criteria for detailed kernel-level analysis (Appendix D.1).

A central finding is that **repository-level counting dramatically understates the available material for parallel translation evaluation**: although only 6 repositories contain both CUDA and OpenMP implementations, kernel-level analysis reveals 472 independent CUDA–OpenMP translation pairs—a 79× multiplier (Appendix Figure 14). This gap motivates PARBENCH's individual kernel-centric evaluation design, rather than evaluating repository-level translation.

Five suites were selected based on their open-source availability, multi-API kernel equivalence, build-run-verify automation, self-checking correctness labels, and domain diversity; detailed criteria and exclusion rationale are provided in Appendix E.2–E.6. **Rodinia** Che et al. [2009][2] is the largest contributor, providing 60 specs across 22 kernels over CUDA, OpenMP, and OpenCL; 53 pass baseline verification and 7 are KNOWN_FAIL. **HeCBench** Jin and Vetter [2023][3] provides the largest initial multi-API pool: from 522 kernels, a structured funnel yields 10 curated kernels—five with CUDA, OpenMP, and OMP-target variants, and five with CUDA and OMP-target only—totaling 25 specs. **XSBench** Tramm et al. [2014][4] and **RSBench** Tramm and Siegel [2014][5] contribute nuclear-physics proxy kernels, each with 4 specs, while **mixbench** Konstantinidis and Cotronis [2017][6] contributes 3 GPU roofline micro-benchmark specs. All XSBench, RSBench, and mixbench specs pass baseline verification.

3.2 Evaluation Corpus

The 87 verified-PASS specs constitute the evaluation corpus (Table 1). Kernel complexity spans more than an order of magnitude, from 80 to 3,304 SLoC with a median of 271. Overall, 31 of 35 kernels (89%) exceed 133 SLoC, the scale beyond which prior repository-level translation evaluation reports 0% pass@1 Davis et al. [2025]. Multi-file translations account for 25% of specs; among the three standard APIs, CUDA exhibits the highest rate (51%) due to separate host and kernel source files, compared with 12% for OpenMP and 13% for OpenCL. These properties make the corpus substantially more complex than single-function code-generation benchmarks while preserving fixed infrastructure for controlled kernel-level evaluation. A full quantitative characterization by suite is provided in Appendix Table 9.

[2] Commit `9c10d3ea` [3] Commit `22785cdd` [4] Commit `ba08e522` [5] Commit `34b64478` [6] Commit `32edeca9`

4 Experimental Setup

Models. We evaluate three models: Qwen 3.5 397B-A17B, a 397B-parameter MoE open-weight model accessed via Together AI; GPT-5.4, a general-purpose proprietary model accessed via Azure OpenAI; and GPT-5.3-codex, a code-specialized model optimized via reinforcement learning on software engineering tasks OpenAI [2026], also accessed via Azure OpenAI. All three expose provider-specific reasoning modes, configured at provider-recommended settings (`enable_thinking` for Qwen 3.5, `reasoning_effort=medium` for GPT-5.4 and GPT-5.3-codex). Because these thinking modes differ in mechanism, observed performance gaps may reflect reasoning-mode effects in addition to base capability. Detailed model configurations, hardware specifications, and cost accounting are provided in Appendix C.5.

Translation Protocol and Scope. From the 96-spec corpus (Section 3), we exclude 9 specifications classified as KNOWN_FAIL due to pre-existing toolchain incompatibilities (Appendix C.5), leaving 87 verified-PASS specs that yield 142 unique translation tasks across ten directions: six standard bidirectional directions among CUDA, OpenMP, and OpenCL, plus four OMP-target case-study directions. Translations follow the kernel-centric protocol (Section 2): only kernel source files are translated while host and build infrastructure remain fixed. We evaluate functional correctness only; performance speedup is out of scope.

Evaluation Phases. The canonical campaign (L0) evaluates the 142 non-KNOWN_FAIL pairs from unmodified source code for each model, drawing $n = 3$ independent samples per task in single-attempt mode (no iterative self-repair). Sampling uses temperature 0.7 for Qwen 3.5; for GPT-5.4 and GPT-5.3-codex, temperature is provider-controlled. Across three models this produces 1,278 L0 records. The augmentation applies an L0-conditional filter, retaining only pairs where at least one L0 sample passes, and evaluates $n = 1$ sample at each augmentation level L1–L4. Extended sampling configurations and deterministic seed derivation are described in Appendix C.5.

Metrics. The primary metric is **pass@k**, using the unbiased estimator from Chen et al. [2021] with normal-approximation confidence intervals on the task-level mean. Separately, we report **per-record pass rates** treating each record as an independent Bernoulli trial, computed with Wilson 95% confidence intervals. Full metric derivations are in Appendix C.5.

With the methodology established, Section 5 presents the pass rates, failure taxonomy, direction asymmetry, and augmentation robustness results.

5 Results

5.1 Overall Performance

Kernel-centric evaluation isolates executable kernel translation from repository reconstruction. Table 2 aggregates all valid records, including L0 and L0-conditional augmentation-ablation records; task-level L0 pass@k is reported separately in Section 5.2. As shown in Table 2, Qwen 3.5 passes 36.7% of its valid records, while GPT-5.4 passes 75.5%. GPT-5.3-codex passes 74.2% of its valid records, making it statistically indistinguishable from GPT-5.4 (Fisher's exact $p = 1.0$, OR = 0.93 [0.74, 1.16], Cohen's $h = -0.031$). Despite code-specialized reinforcement learning training on an earlier base model, GPT-5.3-codex does not surpass the more recent GPT-5.4, suggesting that general-purpose model scaling already captures the parallel programming capabilities tested here. Prior work such as ParEval-Repo reports 0% pass@1 on repository-level applications larger than 133 SLoC Davis et al. [2025]; 89% of our kernels exceed that threshold, yet PARBENCH's kernel isolation enables Qwen 3.5 to achieve 40.3% on CUDA-to-OpenMP—demonstrating that repository reconstruction, not translation itself, is the primary obstacle in full-application benchmarks.

5.2 pass@k Analysis

Restricting to the 142 L0 tasks, Qwen 3.5 achieves pass@1 = 23.9% and pass@3 = 35.2%, while GPT-5.4 achieves pass@1 = 62.7% and pass@3 = 69.7%. GPT-5.3-codex matches GPT-5.4's pass@1 exactly (62.7%) with a slightly narrower pass@1-to-pass@3 gap (5.6 pp versus 7.0 pp for GPT-5.4; Appendix F.4). Stochastic resampling provides diminishing returns: for Qwen 3.5, 64.8% of tasks produce no passing sample across three attempts, while only 13.4% pass all three. GPT-5.4 and GPT-5.3-codex show the inverse profile—53.5% and 57.0% pass deterministically—yet 30.3% and

Table 2: Overall pass rates across three models and 2,262 valid translation records. Qwen 3.5 covers 626 valid records (426 L0 + 200 augmentations) after excluding 82 records involving 9 KNOWN_FAIL specs. GPT-5.4 covers 822 valid records (426 L0 + 396 augmentations) and GPT-5.3-codex covers 814 valid records (426 L0 + 388 augmentations); KNOWN_FAIL specs were pre-excluded from both GPT evaluation batches.

Model	PASS	BUILD_FAIL	RUN_FAIL	VERIFY_FAIL	EXTRACT_FAIL	Total	Rate [95% Wilson CI]
Qwen 3.5 397B-A17B	230	245	121	29	1	626	36.7% [33.1%, 40.6%]
GPT-5.4	621	123	43	32	3	822	75.5% [72.5%, 78.4%]
GPT-5.3-codex	604	139	44	27	0	814	74.2% [71.1%, 77.1%]

Table 3: Pass rates by translation direction (L0 records, all three samples per task). Six standard directions form the core matrix; four OMP-target directions are case studies with fewer participating kernels (m). Here n denotes total L0 records. Wilson 95% CIs in brackets.

Direction	Qwen 3.5	GPT-5.4	GPT-5.3-codex	n
CUDA→OMP	40.3% [29.7%, 51.8%]	83.3% [73.1%, 90.2%]	76.4% [65.4%, 84.7%]	72
OMP→OpenCL	33.3% [22.0%, 47.0%]	72.5% [59.1%, 82.9%]	82.3% [69.8%, 90.4%]	51
OMP→CUDA	25.0% [16.4%, 36.1%]	55.6% [44.1%, 66.5%]	55.6% [44.1%, 66.5%]	72
OpenCL→OMP	9.8% [4.3%, 21.0%]	41.2% [28.8%, 54.8%]	39.2% [27.0%, 52.9%]	51
CUDA→OpenCL	7.0% [2.8%, 16.7%]	59.7% [46.7%, 71.4%]	57.9% [45.0%, 69.8%]	57
OpenCL→CUDA	0.0% [0.0%, 6.3%]	19.3% [11.1%, 31.3%]	19.3% [11.1%, 31.3%]	57
OMP-target→OMP (m=3)	100% [70.1%, 100%]	100% [70.1%, 100%]	100% [70.1%, 100%]	9
OMP-target→CUDA (m=8)	66.7% [46.7%, 82.0%]	95.8% [79.8%, 99.3%]	100% [86.2%, 100%]	24
OMP→OMP-target (m=3)	44.4% [18.9%, 73.3%]	100% [70.1%, 100%]	100% [70.1%, 100%]	9
CUDA→OMP-target (m=8)	0.0% [0.0%, 13.8%]	95.8% [79.8%, 99.3%]	100% [86.2%, 100%]	24

31.7% still hard-fail on every attempt. Across all models, failures are predominantly systematic (incorrect API-surface mapping) rather than stochastic; resampling cannot rescue them.

5.3 Direction Dependence

Per-record pass rates vary strongly across directions (Table 3), reflecting structural API asymmetries. Translating from CUDA to OpenMP replaces explicit device memory and launch configuration with compiler directives. The reverse requires introducing explicit synchronization and memory management. OpenCL translations introduce further complexity with separate host/kernel source and JIT compilation, leading to 0% success for OpenCL-to-CUDA under Qwen 3.5. GPT-5.3-codex's direction profile nearly mirrors GPT-5.4 across all six standard directions (within 2–10 pp), reinforcing that direction difficulty is a property of the task, not the model. At the kernel level, performance is highly heterogeneous (full breakdown in Appendix F.4). Simple kernels pass at high rates, while those with irregular memory access or multi-file dependencies frequently fail entirely.

GPT-5.3-codex's direction profile closely tracks GPT-5.4 across five of six standard directions (maximum gap 7 pp, with two directions matching exactly). The exception is OMP-to-OpenCL, where GPT-5.3-codex outperforms GPT-5.4 by 9.8 pp (82.3% versus 72.5%). Overall, direction difficulty is predominantly a property of the translation task, not the model.

5.4 Failure Taxonomy

Build-stage adaptation is the dominant failure mode, accounting for 39.1% of all records and 61.9% of failures for Qwen 3.5 (Figure 2). These indicate incomplete API-surface adaptation (e.g., retained CUDA identifiers in OpenMP targets). VERIFY_FAIL accounts for 4.6% of records, proving that compile-only or zero-exit-code checks are insufficient for rigorous evaluation. RUN_FAIL represents 19.3% of records, frequently driven by OpenCL JIT compilation errors occurring at runtime. GPT-5.3-codex's failure distribution is intermediate: BUILD_FAIL accounts for 17.1% of records, between Qwen 3.5's 39.1% and GPT-5.4's 15.0%.

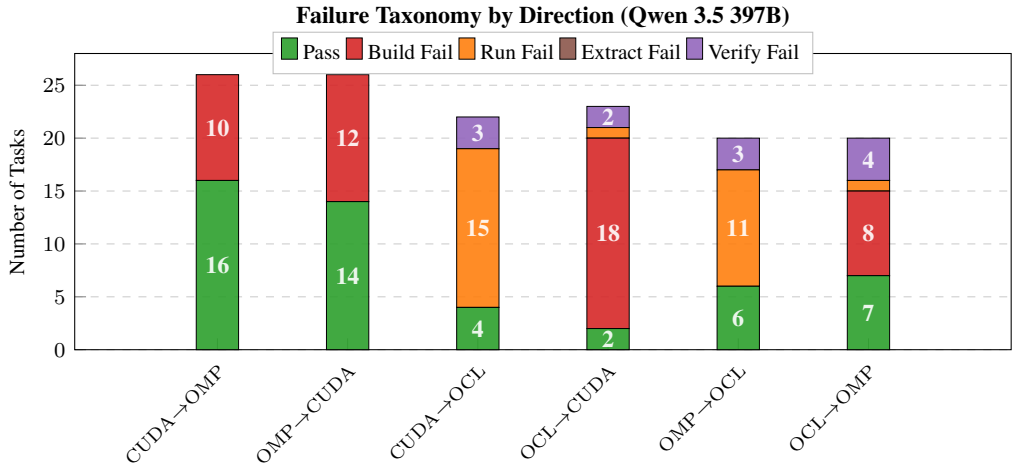

Figure 2: Qwen 3.5 outcome taxonomy (L0, first sample per task, 6 standard directions, 137 tasks drawn from 626 valid records). Build-stage failures are the largest non-pass category. GPT-5.4 and GPT-5.3-codex taxonomies appear in Appendices I and J, respectively.

5.5 Augmentation Robustness

To assess whether baseline success is driven primarily by surface-form memorization, we evaluated augmented source variants (L1–L4) on L0-conditional subsets (detailed in Appendix F.4). GPT-5.3-codex shows the same plateau pattern as GPT-5.4: 85.6%–88.7% at L1–L4 (χ^2 independence test, $p = 1.0$ after Bonferroni correction), in contrast to Qwen 3.5's peak-then-decline (74.0% at L1 to 56.0% at L4, $p = 0.005$). This stability across both GPT models is compatible with robustness to surface-form perturbation, though survivorship bias (L0-conditional filtering) prevents ruling out deeper structural memorization entirely.

Per-kernel analysis reveals exceptions: on `stencil1d`, Qwen 3.5 produces a correct parallel loop at L0 but reverts to a non-portable CUDA-style tiling pattern at L1 (BUILD_FAIL) and L2 (VERIFY_FAIL), and introduces a boundary-condition error at L4 (VERIFY_FAIL)—illustrating how surface-form perturbation can expose fragile pattern-matching rather than robust semantic reasoning.

6 Discussion, Limitations, and Conclusion

PARBENCH establishes that LLMs can perform non-trivial parallel code translation when evaluation isolates the kernel and preserves executable correctness. The benchmark surfaces three structural properties of this task: strong direction dependence (CUDA-to-OpenMP passes at 40–83% while OpenCL-to-CUDA yields 0–19%), build-stage API adaptation as the dominant failure mode, and preliminary robustness to surface-form augmentation on the tested subset. An omnibus chi-squared test across all 2,262 records confirms significant model differences ($\chi^2(2) = 287.27$, $p < 10^{-10}$, Cramér's $V = 0.356$). However, pairwise analysis on the 142 balanced L0 tasks reveals two distinct tiers: GPT-5.4 and GPT-5.3-codex are statistically indistinguishable (Bonferroni-corrected Fisher's exact $p = 1.0$, OR = 0.93 [0.74, 1.16], Cohen's $h = -0.031$; McNemar concordance: 94 both-pass, 40 both-fail, 5 GPT-5.4-only, 3 GPT-5.3-codex-only), while both surpass Qwen 3.5 by approximately $5\times$ odds ratio (Cohen's $h = 0.80$ and 0.77, respectively). Because the two providers use unmatched sampling conditions (Section C.5), the Qwen–GPT gap cannot be causally attributed to model capability alone; the within-provider GPT-5.4–GPT-5.3-codex comparison controls for this confound and suggests that code-specialized training does not measurably improve parallel translation.

The gap is concentrated in deterministic outcomes: 53.5% of GPT-5.4 tasks pass all three samples versus 13.4% for Qwen 3.5, and hard failures (0/3) account for 30.3% versus 64.8%, yielding a smaller pass@1-to-pass@3 gap (7.0 pp versus 11.3 pp). GPT-5.3-codex mirrors GPT-5.4: 57.0% always-pass, 31.7% hard-fail, 5.6 pp pass@1-to-pass@3 gap. All three models share the same dominant failure mode, incomplete API-surface adaptation at build time, but the GPT models reduce

build failures from 50.0% (Qwen 3.5) to 22.8% (GPT-5.4) and 25.1% (GPT-5.3-codex) of L0 records. Direction-level results are consistent across all three models: CUDA-to-OpenMP is the most tractable standard direction (GPT-5.4 83.3%, GPT-5.3-codex 76.4%, Qwen 3.5 40.3%), while OpenCL-to-CUDA remains the hardest (GPT-5.4 and GPT-5.3-codex 19.3%, Qwen 3.5 0%). Augmentation robustness analysis on L0-conditional subsets shows stable pass rates for both GPT models (87–91% for GPT-5.4, 86–89% for GPT-5.3-codex at L1–L4), while Qwen 3.5 peaks at L1 (74%) then declines to 56% at L4.

Limitations. The benchmark is correctness-focused: it does not measure speedups, occupancy, memory bandwidth, or efficiency. Correct translated code may still be unusably slow, a concern also raised by TRACY Gong et al. [2025]. The corpus is finite; some directions have small paired samples. The per-model augmentation analysis covers 22–30 kernels across directions, but the balanced cross-model comparison is limited to 12 common kernels in a single direction. Small per-level sample sizes and the L0-conditioned selection of augmentation subsets complicate formal trend interpretation. Ten specs across six kernels were downgraded from strong or medium oracles to weak (stdout pattern plus exit code): six due to cross-API floating-point reduction-order divergence (cfd, hotspot, myocyte) and four due to synthesis asymmetry between reference implementations (bfs, nw, nn). This limits oracle strength but correctly avoids penalizing faithful translations. The stochastic evaluation (temperature 0.7, three samples) captures variability but does not measure iterative repair or agentic translation strategies. The three models use different reasoning mechanisms and sampling configurations (Section C.5). The Qwen–GPT gap is large (Cohen's $h = 0.80$) but cannot be fully attributed to model capability given unmatched sampling conditions. The within-provider GPT-5.4–GPT-5.3-codex comparison controls for provider-side differences but both share the same Azure OpenAI infrastructure, limiting generalizability beyond this provider. GPT-5.3-codex's code specialization targets general software engineering tasks; the null result for parallel translation may not generalize to models specifically fine-tuned on HPC code.

References

Tomer Bitan, Tal Kadosh, Erel Kaplan, Shira Meiri, Le Chen, Peter Morales, Niranjan Hasabnis, and Gal Oren. UniPar: A unified LLM-based framework for parallel and accelerated code translation in HPC. *arXiv preprint arXiv:2509.12136*, 2025. doi: 10.48550/arXiv.2509.12136. URL https://arxiv.org/abs/2509.12136.

Aman Chaturvedi, Daniel Nichols, Siddharth Singh, and Abhinav Bhatele. HPC-Coder-V2: Studying code LLMs across low-resource parallel languages. In *ISC High Performance 2025 Research Paper Proceedings*, pages 1–14. IEEE, 2025. doi: 10.23919/ISC.2025.11017585. URL https://doi.org/10.23919/ISC.2025.11017585.

Shuai Che, Michael Boyer, Jiayuan Meng, David Tarjan, Jeremy W. Sheaffer, Sang-Ha Lee, and Kevin Skadron. Rodinia: A benchmark suite for heterogeneous computing. In *2009 IEEE International Symposium on Workload Characterization (IISWC)*, pages 44–54. IEEE, 2009. doi: 10.1109/IISWC.2009.5306797. URL https://doi.org/10.1109/IISWC.2009.5306797.

Hao Chen, Qian Liu, and Zhong Ming. Memorize or generalize? an empirical study on code LLMs with semantic-preserving code rewriting, 2025a. URL https://arxiv.org/abs/2503.02296.

Le Chen, Arijit Bhattacharjee, Nesreen K. Ahmed, Niranjan Hasabnis, Gal Oren, Tal Kadosh, and Ali Jannesari. OMPGPT: A generative pre-trained transformer model for OpenMP. In *Proceedings of the 30th International European Conference on Parallel and Distributed Computing (Euro-Par)*, volume 14801 of *Lecture Notes in Computer Science*, pages 121–134. Springer, 2024. doi: 10.1007/978-3-031-69577-3_9. URL https://doi.org/10.1007/978-3-031-69577-3_9.

Le Chen, Nesreen K. Ahmed, Mihai Capotă, Ted Willke, Niranjan Hasabnis, and Ali Jannesari. PCEBench: A multi-dimensional benchmark for evaluating large language models in parallel code generation. In *2025 IEEE International Parallel and Distributed Processing Symposium*, pages 546–557. IEEE, 2025b. doi: 10.1109/IPDPS64566.2025.00055. URL https://doi.org/10.1109/IPDPS64566.2025.00055.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374, 2021. URL https://arxiv.org/abs/2107.03374.

Joshua H. Davis, Daniel Nichols, Ishan Khillan, and Abhinav Bhatele. On the limits of LLM-based repository-level HPC code translation. In *Proceedings of the 54th International Conference on Parallel Processing (ICPP)*, pages 94–103. ACM, 2025. doi: 10.1145/3754598.3754669. URL https://doi.org/10.1145/3754598.3754669.

Matthew T. Dearing, Yiheng Tao, Xingfu Wu, Zhiling Lan, and Valerie Taylor. LASSI: An LLM-based automated self-correcting pipeline for translating parallel scientific codes. In *2024 IEEE International Conference on Cluster Computing Workshops (CLUSTER Workshops)*, pages 136–143. IEEE, 2024. doi: 10.1109/CLUSTERWorkshops61563.2024.00029. URL https://doi.org/10.1109/CLUSTERWorkshops61563.2024.00029.

Mingzhe Du, Anh Tuan Luu, Bin Ji, and See-Kiong Ng. Mercury: An efficiency benchmark for LLM code synthesis. *arXiv preprint arXiv:2402.07844*, 2024. doi: 10.48550/arXiv.2402.07844. URL https://arxiv.org/abs/2402.07844.

Nichole Etienne, Simon Garcia de Gonzalo, and Dorian Arnold. Openmp-annotated code dataset for large language model fine-tuning on parallel programming tasks. *Frontiers in High Performance Computing*, 4:1771927, 2026. doi: 10.3389/fhpcp.2026.1771927. URL https://doi.org/10.3389/fhpcp.2026.1771927.

Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. Datasheets for datasets. *Communications of the ACM*, 64(12): 86–92, 2021. doi: 10.1145/3458723. URL https://doi.org/10.1145/3458723.

William Godoy, Pedro Valero-Lara, Keita Teranishi, and Jeffrey S. Vetter. VibeCodeHPC: An agent-based iterative prompting auto-tuner for HPC code generation using LLMs, 2025. URL https://arxiv.org/abs/2510.00031.

William F. Godoy, Pedro Valero-Lara, Keita Teranishi, Prasanna Balaprakash, and Jeffrey S. Vetter. Large language model evaluation for high-performance computing software development. *Concurrency and Computation: Practice and Experience*, 36(26):e8269, 2024. doi: 10.1002/cpe.8269. URL https://doi.org/10.1002/cpe.8269.

Zhihao Gong, Zeyu Sun, Dong Huang, Qingyuan Liang, Jie M. Zhang, and Dan Hao. TRACY: Benchmarking execution efficiency of LLM-based code translation. arXiv preprint arXiv:2508.11468, 2025. URL https://arxiv.org/abs/2508.11468.

Re'em Harel, Yuval Pinter, and Gal Oren. Learning to parallelize in a shared-memory environment with transformers. In *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*, pages 450–452. ACM, 2023. doi: 10.1145/3572848.3582565. URL https://doi.org/10.1145/3572848.3582565.

Re'em Harel, Tal Kadosh, Niranjan Hasabnis, Timothy G. Mattson, Yuval Pinter, and Gal Oren. PragFormer: Data-driven parallel source code classification with transformers. *International Journal of Parallel Programming*, 53(1):2, 2025. doi: 10.1007/s10766-024-00778-9. URL https://doi.org/10.1007/s10766-024-00778-9.

Ali Reza Ibrahimzada, Kaiyao Ke, Mrigank Pawagi, Muhammad Salman Abid, Rangeet Pan, Saurabh Sinha, and Reyhaneh Jabbarvand. AlphaTrans: A neuro-symbolic compositional approach for repository-level code translation and validation. *Proceedings of the ACM on Software Engineering*, 2(FSE):2454–2476, 2025. doi: 10.1145/3729379. URL https://doi.org/10.1145/3729379.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *Proceedings of the Twelfth International Conference on Learning Representations (ICLR)*, 2024. doi: 10.48550/arXiv.2310.06770. URL https://openreview.net/forum?id=VTF8yNQM66.

Zheming Jin and Jeffrey S. Vetter. A benchmark suite for improving performance portability of the SYCL programming model. In *2023 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 325–327. IEEE, 2023. doi: 10.1109/ISPASS57527.2023.00041. URL https://doi.org/10.1109/ISPASS57527.2023.00041.

Zheming Jin, Swaroop Pophale, and Keita Teranishi. Enhancing ChatPORT with CUDA-to-SYCL kernel translation capability. In *Proceedings of the SC '25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 524–533. ACM, 2025. doi: 10.1145/3731599.3767398. URL https://doi.org/10.1145/3731599.3767398.

Tal Kadosh, Niranjan Hasabnis, Timothy Mattson, Yuval Pinter, and Gal Oren. Quantifying OpenMP: Statistical insights into usage and adoption. In *2023 IEEE High Performance Extreme Computing Conference*, pages 1–7. IEEE, 2023a. doi: 10.1109/HPEC58863.2023.10363459. URL https://doi.org/10.1109/HPEC58863.2023.10363459.

Tal Kadosh, Nadav Schneider, Niranjan Hasabnis, Timothy Mattson, Yuval Pinter, and Gal Oren. Advising OpenMP parallelization via a graph-based approach with transformers. In *OpenMP: Advanced Task-Based, Device and Compiler Programming (IWOMP 2023)*, volume 14114 of *Lecture Notes in Computer Science*, pages 3–17. Springer, 2023b. doi: 10.1007/978-3-031-40744-4_1. URL https://doi.org/10.1007/978-3-031-40744-4_1.

Tal Kadosh, Nadav Schneider, Niranjan Hasabnis, Timothy G. Mattson, Yuval Pinter, and Gal Oren. Advising OpenMP parallelization via a graph-based approach with transformers. In *OpenMP: Advanced Task-Based, Device and Compiler Programming*, volume 14114 of *Lecture Notes in Computer Science*, pages 3–17. Springer, 2023c. doi: 10.1007/978-3-031-40744-4_1. URL https://doi.org/10.1007/978-3-031-40744-4_1.

Tal Kadosh, Niranjan Hasabnis, Prema Soundararajan, Vy A. Vo, Mihai Capotă, Nesreen K. Ahmed, Yuval Pinter, and Gal Oren. OMPar: Automatic parallelization with ai-driven source-to-source compilation. *arXiv preprint arXiv:2409.14771*, 2024a. doi: 10.48550/arXiv.2409.14771. URL https://arxiv.org/abs/2409.14771.

Tal Kadosh, Niranjan Hasabnis, Vy A. Vo, Nadav Schneider, Neva Krien, Mihai Capotă, Abdul Wasay, Guy Tamir, Ted Willke, Nesreen K. Ahmed, Yuval Pinter, Timothy G. Mattson, and Gal Oren. MonoCoder: Domain-specific code language model for HPC codes and tasks. In *2024 IEEE High Performance Extreme Computing Conference*, pages 1–7. IEEE, 2024b. doi: 10.1109/HPEC62836.2024.10938441. URL https://doi.org/10.1109/HPEC62836.2024.10938441.

Erel Kaplan, Tomer Bitan, Lian Ghrayeb, Le Chen, Tom Yotam, Niranjan Hasabnis, and Gal Oren. ParaCodex: A profiling-guided autonomous coding agent for reliable parallel code generation and translation. *arXiv preprint arXiv:2601.04327*, 2026. doi: 10.48550/arXiv.2601.04327. URL https://arxiv.org/abs/2601.04327.

Elias Konstantinidis and Yiannis Cotronis. A quantitative roofline model for GPU kernel performance estimation using micro-benchmarks and hardware metric profiling. *Journal of Parallel and Distributed Computing*, 107:37–56, 2017. doi: 10.1016/j.jpdc.2017.04.002. URL https://doi.org/10.1016/j.jpdc.2017.04.002.

Shanchao Liang, Spandan Garg, and Roshanak Zilouchian Moghaddam. The SWE-Bench illusion: When state-of-the-art LLMs remember instead of reason. arXiv preprint arXiv:2506.12286, 2025. URL https://arxiv.org/abs/2506.12286.

Quazi Ishtiaque Mahmud, Ali TehraniJamsaz, Hung D. Phan, Le Chen, Mihai Capotă, Theodore L. Willke, Nesreen K. Ahmed, and Ali Jannesari. AutoParLLM: GNN-guided context generation for zero-shot code parallelization using LLMs. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 11821–11841. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.naacl-long.593. URL https://aclanthology.org/2025.naacl-long.593/.

Marko Mišić and Matija Dodović. An assessment of large language models for OpenMP-based code parallelization: A user perspective. *Journal of Big Data*, 11:161, 2024. doi: 10.1186/s40537-024-01019-z. URL https://doi.org/10.1186/s40537-024-01019-z.

Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency*, FAT* '19, pages 220–229. ACM, 2019. doi: 10.1145/3287560.3287596. URL https://doi.org/10.1145/3287560.3287596.

Pei Mu, Yifan Zhu, Dongning Ma, and Xipeng Shen. QiMeng-MuPa: Multi-paradigm parallel code generation with large language models, 2025. URL https://arxiv.org/abs/2506.11153.

Daniel Nichols, Joshua H. Davis, Zhaojun Xie, Arjun Rajaram, and Abhinav Bhatele. Can large language models write parallel code? In *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing (HPDC)*, pages 281–294. ACM, 2024a. doi: 10.1145/3625549.3658689. URL https://doi.org/10.1145/3625549.3658689.

Daniel Nichols, Aniruddha Marathe, Harshitha Menon, Todd Gamblin, and Abhinav Bhatele. HPC-Coder: Modeling parallel programs using large language models. In *ISC High Performance 2024 Research Paper Proceedings*, pages 1–12. IEEE, 2024b. doi: 10.23919/ISC.2024.10528929. URL https://doi.org/10.23919/ISC.2024.10528929.

OpenAI. GPT-5.3-codex system card, 2026. URL https://openai.com/index/gpt-5-3-codex-system-card/. Accessed 2026-05-01.

Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. KernelBench: Can LLMs write efficient GPU kernels? In *Proceedings of the 42nd International Conference on Machine Learning*, 2025. doi: 10.48550/arXiv.2502.10517. URL https://arxiv.org/abs/2502.10517.

Swaroop Pophale, Zheming Jin, and Keita Teranishi. ChatPORT: Fine-tuned LLM for easy code PORTing. In *OpenMP: Balancing Productivity and Performance Portability*, volume 16123 of *Lecture Notes in Computer Science*, pages 197–211. Springer, 2025. doi: 10.1007/978-3-032-06343-4_13. URL https://doi.org/10.1007/978-3-032-06343-4_13.

Baptiste Rozière, Marie-Anne Lachaux, Loïk Chanussot, and Guillaume Lample. Unsupervised translation of programming languages. In *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020. doi: 10.5555/3495724.3497454. URL https://proceedings.neurips.cc/paper/2020/hash/ed23fbf18c2cd35f8c7f8de44f85c08d-Abstract.html.

Nadav Schneider, Tal Kadosh, Niranjan Hasabnis, Timothy Mattson, Yuval Pinter, and Gal Oren. MPI-RICAL: Data-driven MPI distributed parallelism assistance with transformers. In *Proceedings of the SC '23 Workshops of the International Conference on High Performance Computing, Network, Storage, and Analysis*, pages 2–10. ACM, 2023. doi: 10.1145/3624062.3624063. URL https://doi.org/10.1145/3624062.3624063.

Nadav Schneider, Niranjan Hasabnis, Vy A. Vo, Tal Kadosh, Neva Krien, Mihai Capotă, Guy Tamir, Theodore L. Willke, Nesreen K. Ahmed, Yuval Pinter, Timothy G. Mattson, and Gal Oren. MPIrigen: MPI code generation through domain-specific language models. In *Proceedings of the 2024 Workshop on AI for Systems*, pages 1–6. ACM, 2024. doi: 10.1145/3660605.3660944. URL https://doi.org/10.1145/3660605.3660944.

Ali TehraniJamsaz, Arijit Bhattacharjee, Le Chen, Nesreen K. Ahmed, Amir Yazdanbakhsh, and Ali Jannesari. CodeRosetta: Pushing the boundaries of unsupervised code translation for parallel programming. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024. doi: 10.52202/079017-3202. URL https://openreview.net/forum?id=V6hrg4O9gg.

John R. Tramm and Andrew R. Siegel. RSBench – a mini-app for the multipole method of cross section lookups. Technical report, Argonne National Laboratory, 2014. https://github.com/ANL-CESAR/RSBench.

John R. Tramm, Andrew R. Siegel, Tanzima Islam, and Martin Schulz. XSBench – the development and verification of a performance abstraction for Monte Carlo reactor analysis. In *Proceedings of the International Conference on the Physics of Reactors (PHYSOR)*, Kyoto, Japan, 2014. URL https://www.mcs.anl.gov/papers/P5064-0114.pdf.

Pedro Valero-Lara, Aaron Young, Thomas III Naughton, Christian Engelmann, Al Geist, Jeffrey S. Vetter, Keita Teranishi, and William F. Godoy. ChatMPI: LLM-driven MPI code generation for HPC workloads. In *Proceedings of Supercomputing Asia and International Conference on High Performance Computing in Asia Pacific Region*, pages 19–30. ACM, 2026. doi: 10.1145/3773656.3773659. URL https://doi.org/10.1145/3773656.3773659.

Yanli Wang, Yanlin Wang, Suiquan Wang, Daya Guo, Jiachi Chen, John Grundy, Xilin Liu, Yuchi Ma, Mingzhi Mao, Hongyu Zhang, and Zibin Zheng. RepoTransBench: A real-world multilingual benchmark for repository-level code translation. *IEEE Transactions on Software Engineering*, 52 (2):675–690, 2026. doi: 10.1109/TSE.2025.3645056.

Yifan Zhang, Yuqi Zhu, Yibo Liu, Jiawei Liu, Ge Li, and Zhi Jin. CodeMorph: Semantic-preserving code transformation for data leakage mitigation in code LLM evaluation. In *Proceedings of the 47th International Conference on Software Engineering (ICSE), Industry Track*, 2025. URL https://arxiv.org/abs/2506.17627.

A Related Work

The main text positions PARBENCH through the evaluation gap it addresses. This appendix expands that comparison in Tables 4–7, which organize prior work by evaluation level and role: general code and repository-level benchmarks, OpenMP/MPI and local parallel-code corpora, LLM-based parallel-code translation and porting systems, and efficiency or robustness benchmarks. The comparison separates five questions that are often conflated in prior work: whether the task is generation, parallelization, porting, or translation; what unit of code is evaluated; whether the corpus is a reusable benchmark or a paper-specific local evaluation set; whether correctness is checked by execution; and whether robustness to surface-form variation is measured. This distinction is important because recent LLM-for-parallel-programming work has produced many useful local corpora, benchmark subsets, and task-specific evaluation protocols, but comparatively few reusable executable frameworks for controlled cross-API translation of existing parallel kernels.

Local parallel-code corpora and paper-specific benchmarks. A recurring pattern in recent LLM-assisted parallel-programming work is that evaluation is often built around a corpus or benchmark subset introduced for the specific paper. Earlier OpenMP and MPI work constructed curated datasets for source-to-source parallelization, pragma recommendation, parallel-region classification, MPI API assistance, and domain-specific model training Harel et al. [2023], Schneider et al. [2023], Kadosh et al. [2023b,c], Chen et al. [2024], Schneider et al. [2024], Kadosh et al. [2024b], Harel et al. [2025], Etienne et al. [2026]. Prior corpus studies also show that OpenMP usage itself has been systematically characterized at scale Kadosh et al. [2023a], reinforcing that parallel-programming corpora are valuable but do not by themselves define a controlled LLM translation benchmark. Later LLM-based systems for automatic parallelization, porting, and HPC code modeling continued this pattern, evaluating on paper-specific mixtures of OpenMP, MPI, CUDA, mini-applications, or benchmark-suite kernels Kadosh et al. [2024a], Mahmud et al. [2025], Mišić and Dodović [2024], Nichols et al. [2024b], Chaturvedi et al. [2025], Godoy et al. [2024], Valero-Lara et al. [2026], Godoy et al. [2025]. These studies are directly relevant because they show sustained demand for parallel-code evaluation, but they do not provide a shared measurement substrate for controlled cross-API translation of existing parallel kernels.

Generation, parallelization, porting, and translation measure different capabilities. Parallel code generation benchmarks such as ParEval and PCEBench ask whether a model can synthesize a parallel program from a natural-language description Nichols et al. [2024a], Chen et al. [2025b]. This is an important capability, but it differs from source-to-source API translation. In generation, the model may choose its own decomposition, data layout, and implementation strategy; in translation, it must preserve the structure and behavior of an existing implementation while adapting thread indexing, synchronization, memory movement, host–device coordination, and API-specific launch structure. Automatic-parallelization systems such as OMPar, AutoParLLM, and related OpenMP-focused assessments measure yet another capability: identifying and introducing parallelism into serial or partially parallel code Kadosh et al. [2024a], Mahmud et al. [2025], Mišić and Dodović [2024]. Porting frameworks such as ChatPORT and CUDA-to-SYCL translation work are closer to PARBENCH in spirit, but are typically narrower in API direction, corpus scope, or evaluation protocol Pophale et al. [2025], Jin et al. [2025]. PARBENCH instead targets cross-API translation of already-parallel kernels, where correctness depends on maintaining the semantics of the original parallel implementation rather than merely producing some correct parallel solution.

Repository-level realism versus kernel-level attribution. Repository-level benchmarks expose realistic integration barriers. SWE-bench and RepoTransBench demonstrate the importance of repository-level executable evaluation in general software settings Jimenez et al. [2024], Wang et al. [2026], while ParEval-Repo shows that full-repository HPC translation can be dominated by build-system and scaffolding failures Davis et al. [2025]. Those barriers matter for deployment, but they can obscure the narrower scientific question of whether the model can translate the computational kernel itself. PARBENCH occupies the intermediate granularity between single-function benchmarks and full repositories. It uses real kernels from established HPC suites, but fixes the surrounding build, run, and verification infrastructure through declarative specifications. This design preserves executable evaluation while making failure attribution sharper: a failed task is more directly tied to API adaptation, multi-file kernel coordination, or semantic preservation, rather than to missing scaffolding or repository reconstruction.

Table 4: Representative related work, Part I-a. General code benchmarks and repository-level translation benchmarks.

Work	Primary task / granularity	Corpus and verification basis	Relation to PARBENCH
HumanEval Chen et al. [2021]	General code generation; function level	Public benchmark with unit tests	Establishes execution-based evaluation, but does not exercise parallel API semantics.
TransCoder Rozière et al. [2020]	General source-to-source translation; function level	Public multilingual corpus with unit tests	Provides a code-translation precedent, but not for HPC or parallel APIs.
SWE-bench Jimenez et al. [2024]	Repository issue repair; repository level	Public GitHub-issue benchmark with repository tests	Demonstrates executable repository evaluation, but is mostly non-HPC and non-parallel.
RepoTransBench Wang et al. [2026]	Repository-level code translation	Public repository benchmark with build/test signals	Shows repository translation complexity, but confounds kernel translation with scaffolding.
AlphaTrans Ibrahimzada et al. [2025]	Repository translation and validation	Repository-scale translation tasks with validation/tests	Adjacent repository-level translation work, but not focused on parallel API semantics.

Table 5: Representative related work, Part I-b. OpenMP/MPI/local parallel-code corpora and generation benchmarks.

Work	Primary task / granularity	Corpus and verification basis	Relation to PARBENCH
Learning to Parallelize Harel et al. [2023]	OpenMP parallelization prediction; loop/region level	Curated OpenMP training and evaluation corpus	Shows local corpus construction for OpenMP assistance, not executable cross-API translation.
MPI-RICAL Schneider et al. [2023]	MPI assistance; function/region level	Curated MPI-focused corpus with recommendation metrics	Relevant as MPI-specific parallel-code assistance using a local dataset.
OpenMP Graph Transformer Kadosh et al. [2023c]	OpenMP parallelization advice; loop/region level	Curated OpenMP corpus with recommendation/classification metrics	Relevant to pragma advice, but not translation evaluation.
OMPGPT Chen et al. [2024]	OpenMP-oriented code modeling; function/task level	OpenMP-oriented model corpus and generation metrics	Domain-specific modeling work, but not reusable translation infrastructure.
MPIrigen Schneider et al. [2024]	MPI code generation; function/-task level	MPI-specific curated corpus with generation checks	Generation-oriented MPI work, not API-to-API kernel translation.
MonoCoder Kadosh et al. [2024b]	HPC code modeling; task/function level	HPC-oriented model corpus and task metrics	Relevant HPC specialization work, but method/corpus-specific.
PragFormer Harel et al. [2025]	Parallel-code classification; loop/region level	Curated classification corpus	Relevant representation work, but not executable translation.
OMPar Kadosh et al. [2024a]	AI-driven OpenMP source-to-source parallelization; program/loop level	Paper-specific OpenMP evaluation set	Automatic parallelization rather than translation of existing parallel kernels.
AutoParLLM Mahmud et al. [2025]	Zero-shot code parallelization; loop/function level	Local parallelization evaluation set	Measures introducing parallelism, not preserving existing parallel semantics across APIs.
OpenMP Assessment Mišić and Dodović [2024]	LLMs for OpenMP parallelization; snippet/program level	Paper-specific assessment set	Shows the need for OpenMP-specific evaluation, but not cross-API translation.
ParEval Nichols et al. [2024a]	Parallel code generation from natural language; task level	Public benchmark with correctness checks	Strong parallel generation benchmark, but evaluates synthesis rather than source translation.
PCEBench Chen et al. [2025b]	Multi-dimensional parallel code generation; task level	Public benchmark with correctness/task metrics	Complements PARBENCH by evaluating generation rather than existing-kernel translation.

Table 6: Representative related work, Part II. LLM-based parallel-code translation, porting, and HPC-code modeling systems.

Work	Primary task / granularity	Corpus and verification basis	Relation to ParBench
LASSI Dearing et al. [2024]	Agentic parallel-code translation; kernel level	Curated HeCBench subset with build-run-verify	Closest in verification style, but smaller in scope and focused on iterative repair.
CodeRosetta TehraniJamsaz et al. [2024]	Parallel code translation and fine-tuning; function/kernel level	Model-specific C++/CUDA corpus with compile/similarity/local checks	Relevant translation work, but lacks fixed multi-API executable specifications.
HPC-Coder Nichols et al. [2024b]	HPC code modeling; task/function level	HPC-oriented corpus and model-evaluation metrics	Domain-specific HPC modeling, not benchmark-infrastructure focused.
HPC-Coder-V2 Chaturvedi et al. [2025]	Code LLMs across low-resource parallel languages; task level	Parallel-language tasks with pass@k and task checks	Relevant to parallel-code LLMs, but not controlled executable kernel translation.
ParEval-Repo Davis et al. [2025]	Repository-level HPC translation	Public HPC repository benchmark with build and functional tests	Motivates ParBench by showing that repository integration can dominate failures.
UniPar Bitan et al. [2025]	Parallel and accelerated code translation; task/program level	System-specific evaluation corpus with local correctness checks	Relevant multi-paradigm translation, but not primarily a reusable benchmark substrate.
QiMeng-MuPa Mu et al. [2025]	Sequential-to-parallel translation; task/program level	Curated training/evaluation corpus with local correctness checks	Related parallelization/translation work, but different from API-to-API translation of existing kernels.
ChatPORT Pophale et al. [2025]	LLM-assisted code porting; kernel/program level	Local porting evaluation with build/run checks	Relevant porting work, but method-specific.
ChatPORT CUDA-to-SYCL Jin et al. [2025]	CUDA-to-SYCL translation; kernel level	Local CUDA-to-SYCL evaluation with build/run checks	Directly relevant API migration, but narrower in direction and scope.
ChatMPI Valero-Lara et al. [2026]	MPI code generation; function/-workload level	MPI-focused local evaluation with functional checks	Distributed-parallel generation, not cross-API kernel translation.
Large LLM Evaluation for HPC Godoy et al. [2024]	Evaluating LLMs for HPC software development; task/program level	HPC-oriented evaluation suite with task-specific checks	Broad HPC LLM context, but not controlled cross-API translation.
VibeCodeHPC Godoy et al. [2025]	Agentic HPC code generation/-tuning; application/kernel level	Small local HPC benchmark set with build/run/performance signals	Agentic generation and optimization, not existing-kernel API translation.
ParaCodex Kaplan et al. [2026]	Profiling-guided agentic parallel generation/translation; kernel/program level	Curated suite mixture with build-run-verify and profiling feedback	Directly relevant method-side work; ParBench can evaluate such systems under fixed specs.

Table 7: Representative related work, Part III. Efficiency, robustness, and benchmark-validity studies relevant to ParBench.

Work	Primary task / granularity	Corpus and verification basis	Relation to ParBench
TRACY Gong et al. [2025]	Efficiency of LLM-based code translation; function/task level	Efficiency benchmark with correctness and runtime	Complementary performance axis; ParBench focuses first on correctness.
KernelBench Ouyang et al. [2025]	Efficient GPU-kernel generation; kernel level	GPU-kernel benchmark with correctness and speed	Accelerator-kernel generation, not translation of existing parallel APIs.
Mercury Du et al. [2024]	Efficiency of LLM code synthesis; function/task level	Efficiency benchmark with runtime/resource metrics	Evaluates efficiency rather than parallel API semantic preservation.
CodeMorph Zhang et al. [2025]	Robustness and leakage via code transformation; function level	Existing benchmarks with transformed variants and unit tests	Provides the source-perturbation principle adapted by ParBench to HPC translation.
Semantic Code Rewrite Chen et al. [2025a]	Memorization versus generalization under code rewriting; function level	Rewritten code benchmarks with unit tests	Motivates source rewriting as a contamination and robustness probe.
SWE-bench Illusion Liang et al. [2025]	Benchmark memorization analysis; repository level	SWE-bench-style evaluation	Supports robustness checks for public code benchmarks.
ParBench	**Cross-API translation of existing parallel kernels; kernel level**	**Reusable executable specs with conjunctive build-run-verify**	**Fixes infrastructure, isolates parallel API translation, and adds L0–L4 AST augmentation.**

Agentic repair and specialized models are complementary to benchmark design. Several systems improve parallel-code generation or translation through fine-tuning, prompting, or iterative agentic repair. CodeRosetta and HPC-Coder-style models demonstrate the value of domain-specific corpora and model adaptation for HPC and parallel code TehraniJamsaz et al. [2024], Nichols et al. [2024b], Chaturvedi et al. [2025]. LASSI evaluates an automated self-correcting pipeline for parallel scientific-code translation on a curated HeCBench subset Dearing et al. [2024]. UniPar, QiMeng-MuPa, VibeCodeHPC, and ParaCodex similarly explore multi-paradigm generation, sequential-to-parallel transformation, agentic HPC code generation, or profiling-guided repair using local or mixed benchmark suites Bitan et al. [2025], Mu et al. [2025], Godoy et al. [2025], Kaplan et al. [2026]. These systems answer questions about method design: how far can feedback, fine-tuning, or agentic iteration push performance? PARBENCH is complementary: it provides an evaluation substrate on which such methods can be compared under a controlled kernel-centric translation protocol, including first-attempt evaluation, fixed infrastructure, and uniform failure taxonomy.

Verification rigor. Verification practices vary substantially across the literature. Some works report prediction accuracy, pragma classification, compilation success, similarity, or pass@k on task-specific checks Harel et al. [2023], Kadosh et al. [2023b], TehraniJamsaz et al. [2024], Chaturvedi et al. [2025]. Others incorporate build, run, and functional validation, especially in agentic or repository-level settings Dearing et al. [2024], Davis et al. [2025], Kaplan et al. [2026]. PARBENCH adopts conjunctive build-run-verify evaluation at the kernel level: a translation must compile, execute, and satisfy all declared correctness checks. This matters because parallel translations can compile and even terminate successfully while still violating output constraints, omitting required host–device transfers, mishandling reductions, or preserving only superficial API structure. Compile-only or exit-code-only evaluation would therefore overestimate translation capability.

Efficiency, performance reasoning, and correctness. A separate line of work evaluates whether generated or translated code is efficient rather than merely correct. TRACY, KernelBench, Mercury, and related GPU-kernel or efficiency benchmarks study runtime, kernel quality, optimization, or resource behavior Gong et al. [2025], Ouyang et al. [2025], Du et al. [2024]. These works are complementary because functional correctness and performance are separable objectives: a translation can be correct but slow, or fast but semantically wrong. PARBENCH deliberately scopes to correctness in order to isolate the translation-capability question before adding performance tuning as a second axis.

Robustness and benchmark validity. Public code benchmarks face contamination and memorization risks, and recent work shows that surface-form perturbations can reveal brittle pattern matching Liang et al. [2025], Zhang et al. [2025], Chen et al. [2025a]. CodeMorph applies semantics-preserving transformations to standard code benchmarks to diagnose leakage and robustness. PARBENCH applies the same principle to HPC translation, where contamination risk is especially plausible because suites such as Rodinia and HeCBench appear in public repositories and prior papers. Its AST-driven augmentation engine creates L0–L4 source variants using semantics-preserving transformations such as identifier renaming, arithmetic rewriting, pointer/array interchange, typedef expansion, and condition rewriting. The goal is not only to detect possible memorization, but to measure whether translation success survives controlled perturbations of the source surface form.

B Future Work

Benchmark longevity depends on headroom. Both GPT models achieve pass@1 = 62.7% at L0, so the current 142-task corpus is not yet saturated, but rapid model improvement will narrow that margin. Three extension mechanisms are available: adding kernels through the declarative spec format (JSON plus build recipe, no framework code), composing augmentation transformations beyond L4, and upgrading weak oracles to numeric verification to catch currently undetected semantic errors.

Additional directions include self-repair experiments using the framework's built-in feedback loop, evaluation of agentic systems such as LASSI Dearing et al. [2024] and ParaCodex Kaplan et al. [2026], fine-tuned HPC models Chaturvedi et al. [2025], efficiency-aware evaluation following TRACY Gong et al. [2025], and expansion of the augmentation campaign to more kernels and directions.

Artifact Availability. An anonymized artifact snapshot is submitted with this paper and contains the curated PARBENCH specifications, the build-run-verify harness, the evaluation and augmentation pipelines, and per-record result JSONs for all three evaluated models. The release contains 206 generated JSON specifications, including the 96 curated specifications used in the paper and the 87 eval-eligible specifications that pass the baseline build-run-verify pipeline. Each kernel specification embeds its provenance: the upstream suite, repository URL, pinned commit hash, license, build recipe, run configuration, and verification oracle (Section 3.1). Upon acceptance, the artifact will be de-anonymized, mirrored to GitHub under the MIT license for the PARBENCH framework, and archived on Zenodo to provide a persistent DOI; upstream suites retain their original licenses: Rodinia (BSD-like), XSBench (MIT), RSBench (MIT), HeCBench (MIT), and mixbench (GPL-3.0). A datasheet following Gebru et al. [2021] and a JSON schema accompany the release; Croissant export for ML dataset registries is planned as a post-acceptance extension.

Appendix L provides a structured Evaluation Card declaring PARBENCH's measurement scope, assumptions, valid and invalid claims, and a recommended reporting protocol for future users.

PARBENCH is evaluation infrastructure rather than a new translation method. Its contribution is to make the measurement problem sharper: kernel-centric isolation avoids repository-level build confounds, declarative specs make correctness reproducible, augmentation probes robustness, and the harness exposes where translations fail. The initial results show that current LLMs have real but uneven parallel translation capability, with build adaptation, direction asymmetry, and multi-file coordination as central barriers to reliable use. The benchmark code, all 96 curated JSON specs (87 eval-eligible), and per-record result files for all three models are included in the supplementary material to enable independent verification and extension.

C Framework and Evaluation Details

This appendix expands the framework description in Section 2. It records the spec structure, augmentation level definitions, and implementation details that are useful for reproducing the benchmark but too detailed for the main narrative.

C.1 Specification Schema

Each benchmark/API variant is encoded by a JSON specification. The schema defines five primary field groups (with additional `implementation`, `hardware`, `metadata`, `baseline_results`, and `performance` fields defined in the schema):

- **Identity and provenance**: globally unique identifier, source suite, kernel name, target API, repository URL, and commit.
- **File partitioning**: prompt payload files, translation targets, support files, and verification-only files.
- **Build**: build system, clean/configure/build commands, environment variables, and expected executable path.
- **Run**: executable, input configuration, command-line arguments, timeouts, and optional environment variables.
- **Verification**: ordered correctness strategies applied conjunctively. Five types are implemented: `exit_code`, `stdout_pattern` `stdout_exclude_pattern`, `numeric_comparison`, and `file_hash`.

C.2 Augmentation Levels

Table 8 gives the exact L0–L4 augmentation definitions.

C.3 Prompt Anonymization

Before prompt construction, PARBENCH applies six anonymization measures: it removes the kernel name and description, strips C/C++ comments from all files shown to the LLM, replaces source filenames with generic labels (`Source File 1`, etc.), genericizes target filenames (`translated_0.ext`,

```
{
  "identity": {
    "unique_id": "rodinia-bfs-cuda",
    "parallel_api": "cuda",
    "source_suite": "rodinia"
  },
  "files": {
    "prompt_payload": ["bfs.cu", "kernel.cu", "kernel2.cu"],
    "support_files": ["Makefile"],
    "translation_targets": ["bfs.cu", "kernel.cu", "kernel2.cu"]
  },
  "build": {
    "commands": {"build": "make CUDA_DIR=..."},
    "outputs": {"executable": "./bfs.out"}
  },
  "run": {
    "executable": "./bfs.out",
    "timeout_seconds": 300,
    "input_configurations": {
      "correctness": {
        "arguments": ["../../data/bfs/graph1MW_6.txt"]
      }
    }
  },
  "verification": {
    "strategies": [
        {"type": "stdout_pattern",
        "pattern": "Result stored in result\\.txt"},
        {"type": "exit_code", "expected": 0}
        ]
    }
}
```

Listing 1: Condensed specification structure for `rodinia-bfs-cuda`.

Table 8: Augmentation level definitions.

Level	Transform selection	Candidate fraction	Description
L0	None	0%	Unmodified source code baseline
L1	1 transform	1 site	Minimal source perturbation
L2	33% of transforms	33% of candidates	Moderate perturbation
L3	66% of transforms	66% of candidates	Heavy perturbation
L4	All transforms	100% of candidates	Maximum configured perturbation

etc.) and support-file names (`Header File` N, `Code File` N, `Infrastructure File` N), and anonymizes kernel identifiers in build commands. These measures complement source augmentation: anonymization reduces benchmark identity leakage, while augmentation changes the source surface form. The complete prompt template is reproduced in Appendix H.

C.4 Response Extraction Strategy

Each LLM response is parsed to recover the translated source files using a multi-tier extraction strategy that progressively relaxes matching criteria. The tiers are applied in order until all expected target files are recovered:

1. **Explicit filename match.** The parser searches for markdown code fences annotated with the expected target filename (e.g., ‘‘‘`cpp filename=translated_0.cpp`). This is the highest-confidence tier.
2. **Extension-based match.** If explicit filenames are absent, the parser matches code fences by file extension (e.g., `.cu`, `.cl`, `.cpp`) against the expected target file types.
3. **Fuzzy match.** Partial filename matches and common naming variants are attempted (e.g., matching `kernel.cu` to an expected `translated_0.cu`).

4. **Elimination-based assignment.** When multiple code blocks remain unmatched, they are assigned to the remaining expected files by elimination order.

If any expected target file cannot be recovered or the extracted content is empty after all tiers, the task is classified as EXTRACT_FAIL. Across the 2,262 valid records, extraction failures are rare (1 for Qwen 3.5, 3 for GPT-5.4, 0 for GPT-5.3-codex), indicating that the structured prompt format (Appendix H) effectively guides models to produce parseable output.

C.5 Extended Experimental Setup Details

This section expands on the experimental setup described in Section 4.

KNOWN_FAIL Exclusion Policy. Nine specs are classified as KNOWN_FAIL due to toolchain incompatibilities or pre-existing runtime failures unrelated to LLM translation quality: seven Rodinia specs affected by deprecated CUDA 12 texture APIs, missing system libraries, pre-existing segmentation faults, or silent OpenCL runtime failures, and two HeCBench `omp_target` specs with build or verification failures on the evaluation platform. A record is excluded when *either* the source or the target spec is KNOWN_FAIL. Source-side exclusion is necessary because source correctness is unverified on the failing platform; target-side exclusion is necessary because the target build or verification infrastructure is broken. For Qwen 3.5, this policy excludes 82 of 708 total records, leaving 626 valid records (426 L0 + 200 augmentations). The GPT-5.4 and GPT-5.3-codex evaluation batches pre-exclude KNOWN_FAIL specs, so all 822 GPT-5.4 and 814 GPT-5.3-codex on-disk records are valid.

Sampling Configuration. For Qwen 3.5, we set temperature 0.7 with top-p = 1.0 (full-distribution sampling). For GPT-5.4 and GPT-5.3-codex, the Azure reasoning API does not accept explicit temperature or top-p parameters—these are controlled internally by the provider and cannot be set by the caller. Cross-model differences in pass rates may therefore partly reflect unmatched sampling conditions rather than model capability alone; the confound is between Qwen 3.5 (explicit temperature) and both Azure models (provider-controlled). A deterministic seed derived from SHA-256 of the concatenation `source_spec|target_spec|sample_id` (pipe-delimited, truncated to 31 bits) is passed to Qwen 3.5 (Together AI) and GPT-5.4 (Azure Chat Completions). The Responses API used by GPT-5.3-codex does not accept a seed parameter. Seeds are recorded for auditing but do not guarantee bitwise-identical outputs: Together AI treats seeds as advisory for MoE inference, and Azure provides no determinism guarantee for reasoning models. Each sample receives one LLM call with no iterative feedback, isolating first-attempt translation capability.

Canonical Evaluation and Augmentation. The canonical campaign evaluates all 142 unique pairs at augmentation level L0 (unmodified source code) with three samples each, producing 426 L0 records per model and 1,278 total. Each record passes through the full build–run–verify pipeline with conjunction semantics. Most specs verify via exit-code and stdout-pattern checks; a subset additionally declares numeric-comparison or file-hash oracles. For the augmentation, an L0-conditional filter is applied: a pair is included only if any of its three canonical L0 samples passes. This avoids spending budget on pairs the model cannot translate even from unmodified source. For Qwen 3.5, 50 of the 142 pairs qualify (35%); for GPT-5.4, 99 pairs qualify (70%); for GPT-5.3-codex, 97 pairs qualify (68%). Each qualifying pair is evaluated once (k = 1) at each of levels L1 through L4.

Metrics Details. For pass@k, following Chen et al. [2021], we use the unbiased estimator $1 - \binom{n-c}{k} / \binom{n}{k}$, where c is the number of correct samples out of n total. We report pass@1 (expected single-sample success rate, equivalent to c/n averaged across tasks) and pass@3 (fraction of tasks where any sample passes). Separately, we report per-record pass rates with Wilson 95% confidence intervals. Wilson intervals are preferred over the Wald interval for their superior coverage at extreme proportions. Since three samples per task share the same kernel and direction, the independence assumption is anti-conservative and the Wilson CI understates the true uncertainty. For augmentation analysis, we apply the Cochran–Armitage trend test where per-level sample sizes are adequate, with Cohen's h for adjacent-level effect sizes.

D API Selection Rationale

This appendix provides the complete quantitative evidence underlying ParBench's selection of CUDA, OpenMP, and OpenCL as target parallel programming APIs. Section 3.1 in the main paper summarizes the corpus selection process; here we present the expanded survey data and kernel-level analysis that informed the final selection.

D.1 Full API Co-Occurrence Data

The benchmark landscape survey initially covered 71 repositories spanning benchmark suites, mini-applications, proxy applications, libraries, and full applications across 29 distinct parallel programming APIs. Of these, 35 met the inclusion criteria for detailed analysis (Section E.2). The main paper (Section 3.1) reports statistics for these 35 qualifying repositories.

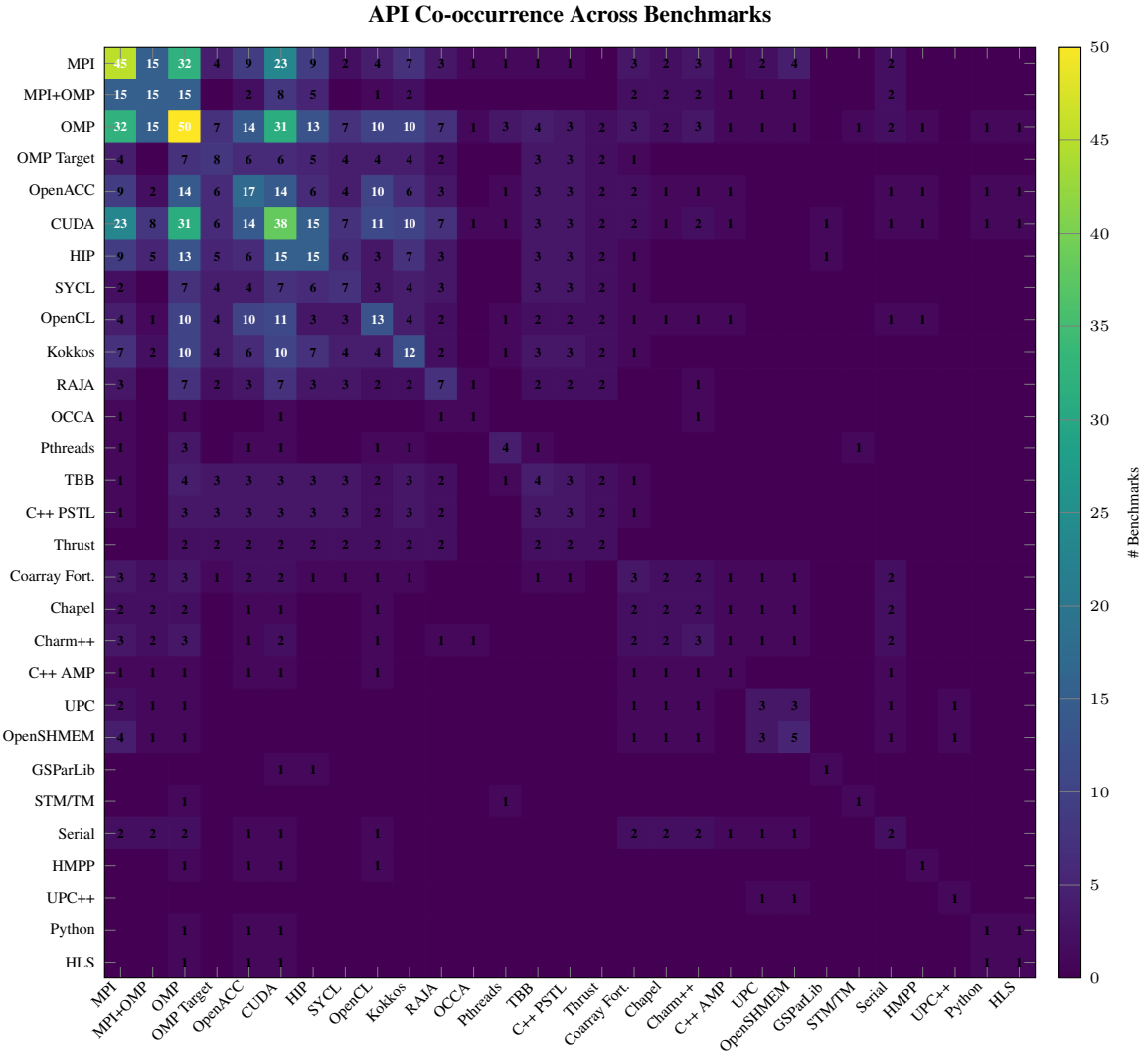

Figure 3: API co-occurrence heatmap across all 71 surveyed repositories. Cell values indicate the number of repositories providing implementations in both the row and column APIs.

Figure 3 shows the full survey. The dominant pattern is a dense co- occurrence cluster among MPI, OpenMP, CUDA, and HIP, with sparser connectivity among directive-based (OpenACC, OpenMP

Target) and portability (Kokkos, RAJA, SYCL) APIs. Peripheral APIs such as Chapel, STM/TM, HLS, and Python show near-zero co-occurrence.

We classify the co-occurrence structure into three groupings:

Core cluster. MPI, OpenMP, CUDA, HIP, and OpenACC co-occur frequently across the suites with the richest multi-API material (HeCBench, Rodinia, RAJAPerf, BabelStream). OpenCL co-occurs with fewer core APIs (appearing in 13 of 40 repositories, versus 38–50 for the top three) but is included in PARBENCH's target API set because it exercises a qualitatively distinct programming model (Section D.3).

Portability periphery. SYCL, Kokkos, RAJA, OpenMP Target, and MPI+OpenMP connect to the core cluster with moderate co-occurrence, concentrated in a smaller number of suites (primarily RAJAPerf, BabelStream, and HeCBench). Pthreads, TBB, and C++ PSTL appear in 3–4 repositories each and occupy a similar peripheral role.

Isolated APIs. OCCA, Thrust, Chapel, C++ AMP, HMPP, Python, HLS, GSParLib, STM/TM, Serial, and UPC++ have maximum pairwise co-occurrence at most 2. Coarray Fortran, Charm++, UPC, and OpenSHMEM reach co-occurrence of 3–4 with MPI or with each other but remain disconnected from the GPU-centric core.

Figure 4 presents the per-API coverage in a ranked bar chart, making the quantitative dominance of the top-3 APIs clear.

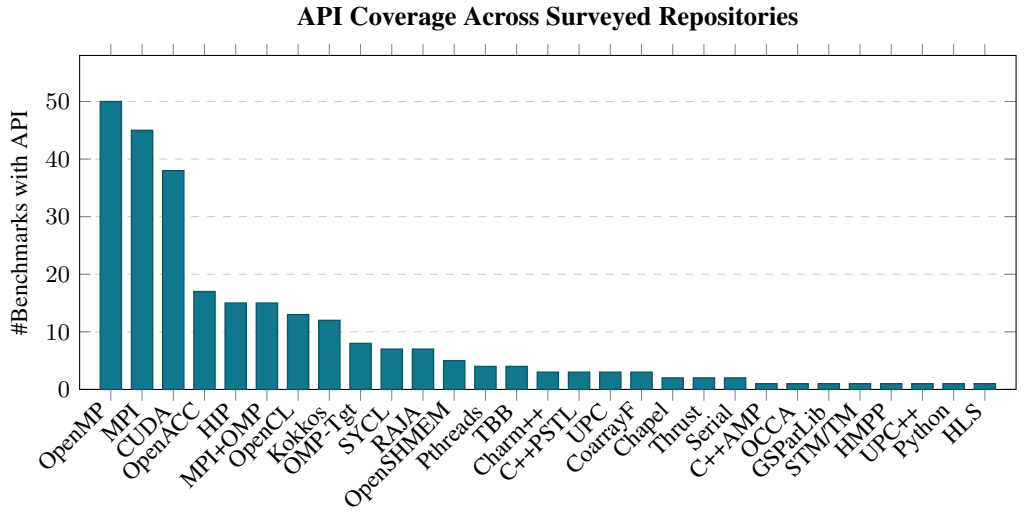

Figure 4: API coverage across the benchmark survey. OpenMP (50 benchmarks), MPI (45), and CUDA (38) dominate, with OpenACC (17), HIP (15), and MPI+OpenMP (15) forming a second tier. OpenCL (13) and Kokkos (12) bridge the second and third tiers. OpenMP Target (8), SYCL (7), and RAJA (7) form a third tier.

APIs below 5 benchmarks were excluded from evaluation consideration due to insufficient benchmark material for statistical evaluation.

Kernel-Level Co-Occurrence

The repository-level analysis in Figures 3–4 counts how many *repositories* provide implementations in a given API pair, but does not capture the depth of coverage within each repository. A suite like HeCBench provides 522 individual kernels, each potentially implemented in multiple APIs, while a single-kernel mini-application like miniBUDE provides only 1 kernel across 12+ APIs. For translation evaluation, the kernel count is the relevant unit: it determines how many independent evaluation instances are available per direction.

Figure 5 provides the strongest quantitative evidence for API selection. The kernel-level data reveals a clear hierarchy:

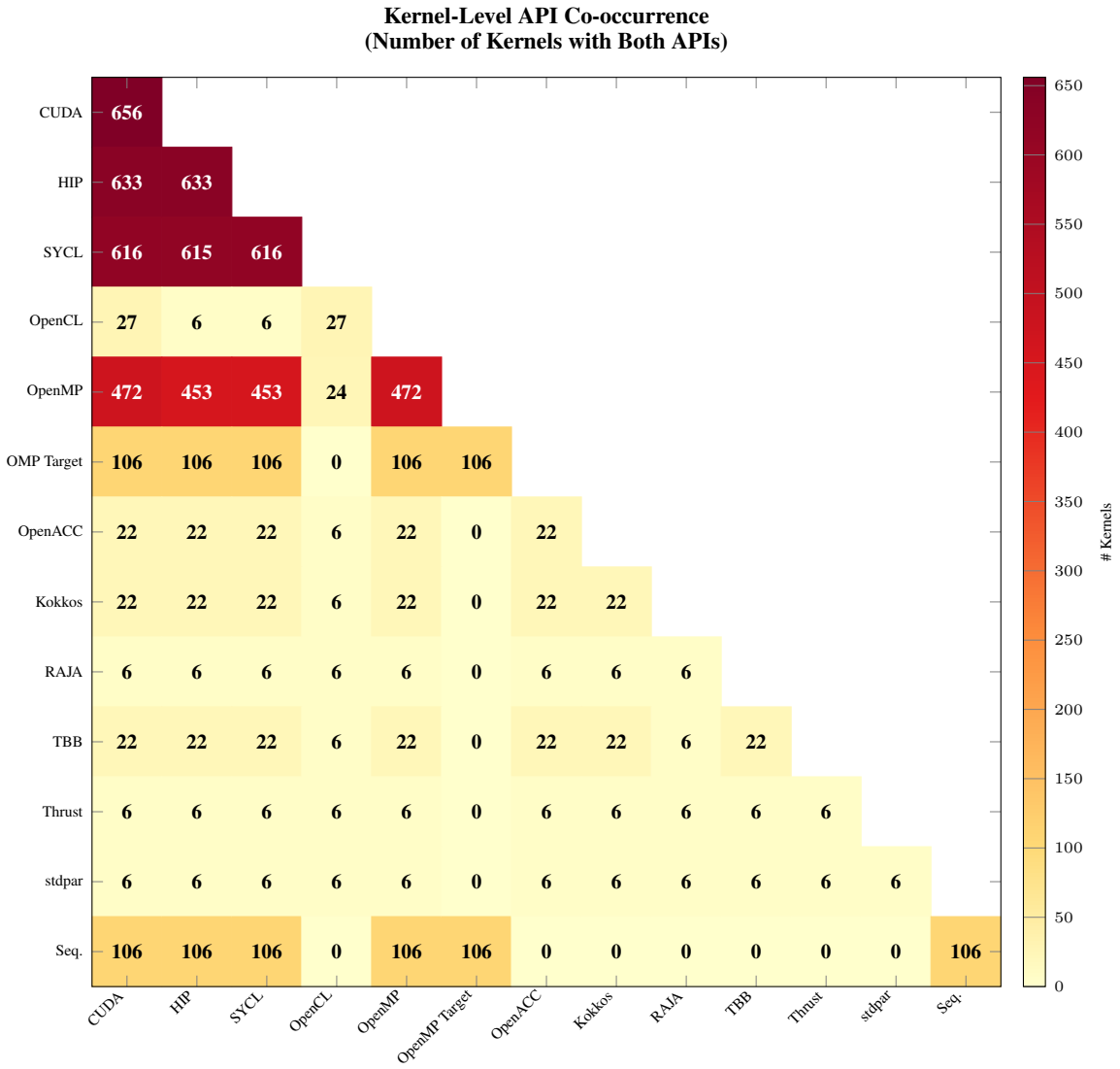

Figure 5: Kernel-level API co-occurrence matrix. Each cell counts the number of individual kernels with implementations in both the row and column APIs. The CUDA–HIP pair leads with 633 shared kernels, followed by CUDA–SYCL (616), HIP–SYCL (615), and CUDA–OpenMP (472). OpenCL has dramatically lower kernel-level coverage: only 27 kernels share implementations with CUDA, and 24 with OpenMP. OpenMP Target provides 106 shared kernels with each of CUDA, HIP, SYCL, and OpenMP, concentrated predominantly in HeCBench. Sequential reference implementations exist for 106 kernels.

- **Tier 1 (>600 kernel pairs):** CUDA–HIP (633), CUDA–SYCL (616), HIP–SYCL (615). These pairs are dominated by HeCBench's systematic multi-API coverage.
- **Tier 2 (∼450 kernel pairs):** CUDA–OpenMP (472), HIP–OpenMP (453), SYCL–OpenMP (453). OpenMP's CPU threading model provides the deepest pool for *paradigm-crossing* translation evaluation.
- **Tier 3 (∼100 kernel pairs):** OpenMP Target and Sequential at 106 each, concentrated in HeCBench.
- **Tier 4 (<30 kernel pairs):** OpenCL at 27 (CUDA–OpenCL) and 24 (OpenMP–OpenCL), with an OpenCL diagonal of 27 total kernels.

The network visualization in Figure 6 makes this tiered structure visually apparent.

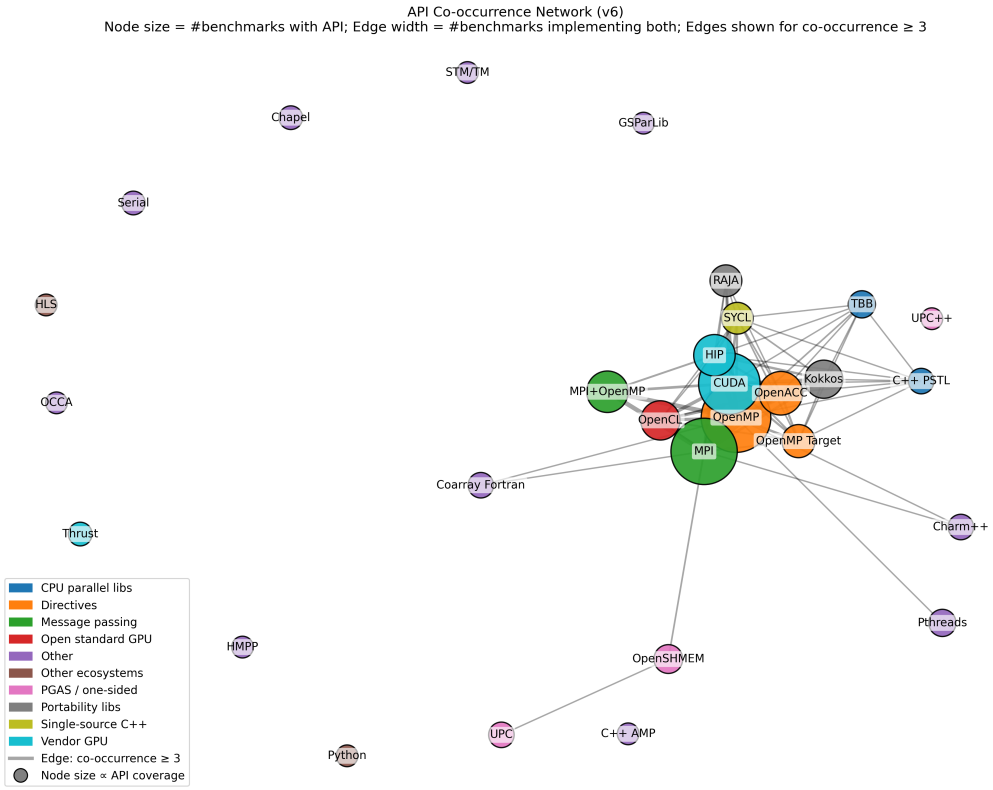

Figure 6: Kernel-level API co-occurrence network. Edges shown for ≥ 20 kernel pairs; edge width proportional to count. The tight CUDA/HIP/SYCL/OpenMP cluster reflects HeCBench and RAJAPerf's uniform multi-API coverage. OpenCL appears as a peripheral node with weak connections (27 CUDA–OpenCL kernel pairs, just above the 20-kernel threshold). RAJA, stdpar, and Thrust appear as isolated nodes with fewer than 20 kernel-level pairs.

D.2 Translation Difficulty Taxonomy

The preceding analysis establishes which translation directions are feasible at scale. The choice of API pairs for evaluation is motivated not only by data availability (Section D.1) but also by the *structural transformation complexity* each direction demands. We classify translation directions into four categories:

Syntactic renaming (e.g., CUDA to HIP): Near-1:1 API call mapping. `cudaMalloc` becomes `hipMalloc`; kernel launch semantics are preserved with minor syntax changes; thread-index arithmetic (`threadIdx.x`, `blockIdx.x`) maps directly to HIP equivalents. This category primarily tests API surface adaptation rather than parallel reasoning.

```
// CUDA                              // HIP (syntactic rename)
cudaMalloc(&d_a, size);              hipMalloc(&d_a, size);
kernel<<<grid, block>>>(d_a);        hipLaunchKernelGGL(kernel,
                                       grid, block, 0, 0, d_a);
cudaMemcpy(h_a, d_a, ...);           hipMemcpy(h_a, d_a, ...);
```

Listing 2: CUDA to HIP: syntactic renaming.

Paradigm translation (e.g., CUDA to OpenMP): SPMD to fork-join model transformation. Explicit GPU thread indexing must be converted to loop-based parallelism with OpenMP directives. Shared memory and synchronization primitives (`__shared__`, `__syncthreads()`) must be replaced with stack-allocated arrays and implicit barrier semantics. Memory management (`cudaMalloc`/`cudaMemcpy`/`cudaFree`) is entirely eliminated.

```
// CUDA                                    // OpenMP
__global__ void kern(float* a, int n) {    void kern(float* a, int n) {
  int i = blockIdx.x * blockDim.x           #pragma omp parallel for
          + threadIdx.x;                     for (int i = 0; i < n; i++) {
  if (i < n) a[i] = a[i] * 2.0f;              a[i] = a[i] * 2.0f;
}                                            }
                                           }
```

Listing 3: CUDA to OpenMP: paradigm translation.

Architecture split (e.g., CUDA to OpenCL): CUDA's single-source compilation model (host and device code in one `.cu` file) must be split into separate host code (OpenCL runtime API calls for platform/device/context/queue management) and kernel source (`.cl` files with OpenCL C syntax). The memory model changes from CUDA's unified virtual addressing to OpenCL's explicit buffer objects. Kernel launch syntax transforms from `«<grid, block»>` to `clEnqueueNDRangeKernel` with explicit work-group sizing.

Directive insertion (e.g., sequential C to OpenMP): Adding parallelism to sequential code by identifying parallelizable loops and inserting appropriate `#pragma omp` directives with correct data-sharing clauses (`shared`, `private`, `reduction`). This is the inverse of the paradigm translation direction and tests whether the LLM can *discover* parallelism rather than *translate* it.

PARBENCH targets the middle of this difficulty spectrum: paradigm translation and architecture split (CUDA/OpenMP/OpenCL cross-translations). Syntactic renaming (CUDA to HIP) is too easy to be informative, while directive insertion requires sequential baselines outside the current corpus. The six directions among CUDA, OpenMP, and OpenCL span paradigm translation (CUDA↔OpenMP) and architecture split (CUDA↔OpenCL, OpenMP↔OpenCL), maximizing the semantic challenge while remaining tractable for automated verification.

D.3 HIP and SYCL Exclusion Rationale

Despite the large kernel-level co-occurrence counts for CUDA–HIP (633) and CUDA–SYCL (616), these pairs were excluded from the primary evaluation for distinct reasons.

HIP exclusion: CUDA-to-HIP translation is predominantly a syntactic renaming task. The AMD HIP API was explicitly designed as a drop-in replacement for CUDA, preserving the SPMD programming model, kernel launch syntax, and memory management API. Automated tools such as `hipify-perl` and `hipify-clang` achieve near-perfect conversion rates. Evaluating LLMs on CUDA-to-HIP translation would measure API name lookup ability rather than parallel programming reasoning—the core capability PARBENCH is designed to assess.

SYCL exclusion: While SYCL introduces a genuinely different programming model (single-source C++ with lambda-based kernel dispatch and buffer/accessor memory management), it preserves the GPU execution model. CUDA-to-SYCL translation is more complex than CUDA-to-HIP but remains within the same architectural paradigm (GPU offload). Furthermore, SYCL benchmark coverage is concentrated almost entirely in HeCBench, limiting the diversity of evaluation instances. The CUDA-to-OpenMP direction provides a more fundamental paradigm crossing (GPU SPMD to CPU fork-join) with better benchmark diversity across Rodinia, HeCBench, XSBench, and RSBench.

D.4 OpenACC Exclusion

OpenACC was excluded despite appearing in 17 benchmarks in the repository-level survey (Figure 4), of which 12 also provide both CUDA and OpenMP implementations.

Three factors motivated this decision:

1. **Low kernel-level coverage**: At the kernel level, only 22 kernels provide OpenACC implementations across the surveyed repositories (Figure 5)—fewer than OpenCL (27) and substantially fewer than the primary APIs (CUDA: 656, OpenMP: 472). While repository-level co-occurrence with CUDA and OpenMP is high, the kernel-level material is insufficient for the multi-direction evaluation design that PARBENCH requires.
2. **Paradigm overlap with OpenMP target**: OpenACC's directive-based model (`#pragma acc parallel loop`) occupies a similar conceptual niche to OpenMP's target offload directives (`#pragma omp target teams distribute`). Including both would provide diminishing returns in programming-model diversity. OpenCL, despite comparable kernel counts, is retained because it exercises a qualitatively distinct programming model (explicit host/kernel separation and JIT compilation) not covered by any other included API.
3. **Compiler availability**: OpenACC compilation requires the NVIDIA HPC SDK (`nvc/nvc++`) or GCC with `-fopenacc`. This is a less universally available toolchain than CUDA (`nvcc`) or OpenMP (any modern C/C++ compiler), introducing a confounding variable between compiler availability and LLM translation capability.

D.5 OpenMP Target as Case Study

OpenMP target offload (`#pragma omp target`) occupies a middle ground between CPU OpenMP and GPU-native APIs. It uses the OpenMP directive model but targets GPU execution, requiring the programmer to specify data mapping (`#pragma omp target data map(...)`) and work distribution (`#pragma omp teams distribute parallel for`) explicitly.

PARBENCH evaluates OpenMP target as a case study rather than a primary direction for three reasons:

1. **Compiler requirement**: OpenMP target compilation for NVIDIA GPUs requires the NVIDIA HPC compiler (`nvc/nvc++`, part of the NVIDIA HPC SDK 24.3). Default GCC and Clang installations support only CPU-threaded OpenMP; GPU offloading requires custom builds with target-offload support enabled. This introduces a confounding variable: a failed build may indicate compiler unavailability rather than translation quality.
2. **Limited benchmark coverage**: The kernel-level survey identifies 106 kernels with both OpenMP target and CUDA implementations (Figure 5), concentrated predominantly in HeCBench. Within PARBENCH's curated corpus, OpenMP target implementations are available for 12 kernels: 10 from HeCBench, 1 from XSBench, and 1 from RSBench. Rodinia does not provide OpenMP target variants.
3. **Compilation model difference**: OpenMP target generates GPU offload code through the compiler, while CPU OpenMP generates threaded CPU code. A translation from CUDA to OpenMP target preserves GPU execution semantics but changes the syntax entirely; a translation from CUDA to CPU OpenMP changes both the execution model and the syntax. These are qualitatively different evaluation targets that should not be conflated in aggregate statistics.

The case study results for OpenMP target are reported separately in the direction analysis (Table 3) to enable direct comparison with the primary CUDA/OpenMP/OpenCL directions without confounding the aggregate pass rates.

E Benchmark Kernel Survey

This appendix documents the systematic survey and multi-stage selection process that produced PARBENCH's evaluation corpus. The main paper (Section 3) summarizes this process; here we provide the complete data, selection criteria, and exclusion rationale.

E.1 Full Repository Inventory

The benchmark landscape survey covered 71 open-source HPC repositories (Appendix D.1). After applying the inclusion criteria described in Section E.2, 35 repositories remained as candidates for kernel extraction, spanning five categories of HPC benchmark software (Figure 7).

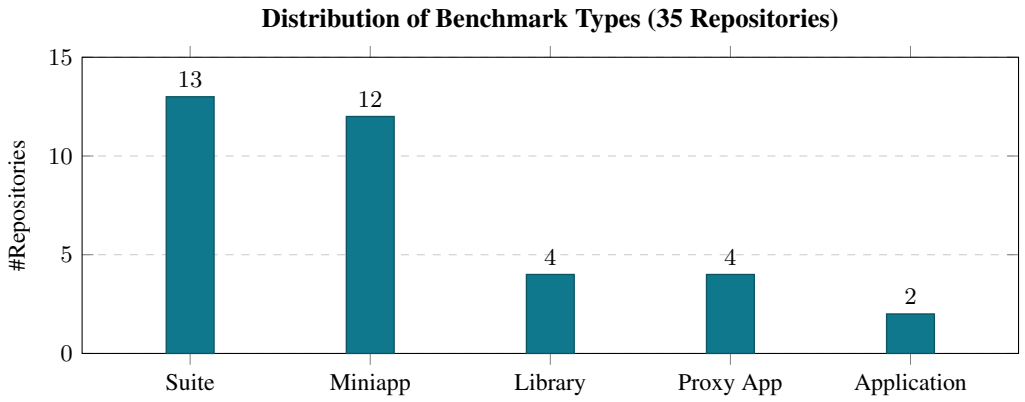

Figure 7: Distribution of benchmark types across the 35 repositories that met inclusion criteria. Suites (13, 37%) and mini-applications (12, 34%) dominate, accounting for 71% of included repositories. Proxy applications (4, 11%), libraries (4, 11%), and full applications (2, 6%) complete the distribution. Suites and mini-applications yield the richest kernel-level material for translation evaluation: they contain multiple independent computational kernels with self-checking verification.

API coverage varies widely across repositories: some implement a single API (e.g., SPEC OMP, PARSEC, LonestarGPU), while others provide implementations across 10+ APIs (BabelStream and miniBUDE each with 12).

Table 9: Quantitative characterization of the evaluation corpus. SLoC counts physical source lines (non-blank, non-comment) in each kernel's CUDA prompt payload, the code provided to the LLM as translation input. Multi-File gives specs requiring translation of >1 source file. Std. shows the dominant language standard per suite.

Suite	SLoC Range	Med.	Multi-File	Std.
Rodinia	195–3,304	334	21/60 (35%)	C++14
HeCBench (curated)	80–235	180	0/25 (0%)	C++17
XSBench	1,390	–	2/4 (50%)	C11
RSBench	1,016	–	1/4 (25%)	C11
mixbench	312	–	0/3 (0%)	C++11
Total	**80–3,304**	**271**	**24/96 (25%)**	

The API breadth distribution across the full 71-repository survey reveals the selection opportunity (Figure 8).

Taken together, these distributions show that kernel-level translation material is concentrated in a small number of large suites with broad API coverage—a Pareto distribution that guided suite selection. HeCBench and Rodinia were selected as the primary contributors due to their kernel richness and three-API coverage; XSBench, RSBench, and mixbench were added to extend domain coverage to nuclear physics simulation and GPU roofline characterization.

E.2 Quality Tier Classification

All 35 repositories that met inclusion criteria were classified into two quality tiers based on three criteria: build documentation, verification capability, and maintenance status.

Tier A criteria (all three required):

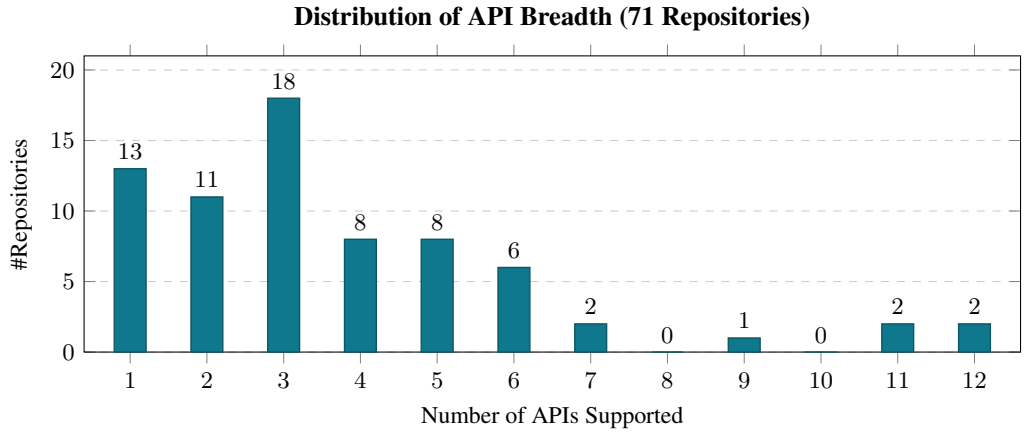

Figure 8: Distribution of API breadth across all 71 surveyed repositories. The mode is 3 APIs (18 repositories); a long tail extends to 12 APIs (BabelStream and miniBUDE). 24 repositories support only 1–2 APIs and are unsuitable for cross-API translation evaluation. Repositories supporting $\geq$3 APIs constitute the candidate pool for multi-direction evaluation.

- Documented build process (Makefile, CMake, or equivalent with clear instructions)
- Automated verification: self-checking output (print PASS/FAIL, compute checksum, compare against reference) or reference output file comparison
- Active maintenance: commits within 2 years of the survey date, or stable release with no known build failures on modern toolchains

Tier B criteria (any one triggers downgrade):

- Partial verification only (e.g., visual output inspection, manual comparison)
- Limited API coverage (single API, reducing cross-translation utility)
- Unmaintained but still buildable (no commits in 2+ years, but Makefiles still produce working binaries)

Tier B repositories were not excluded from the survey but were deprioritized in kernel selection. Their kernels were considered only when no Tier A alternative existed for a given computational domain.

E.3 Candidate Ranking and HeCBench Selection Funnel

The selection of individual kernels for ParBench's evaluation corpus required balancing two competing objectives: *kernel richness* (maximizing the number of independent evaluation instances) and *API breadth* (maximizing the number of translation directions per kernel). Figure 9 plots these two dimensions for all surveyed repositories.

Figure 10 presents the top candidates ranked by their combined kernel-count and API-breadth scores, revealing the fundamental tradeoff in corpus construction.

HeCBench Selection Funnel

HeCBench's 522 kernels were filtered through a five-stage funnel to identify the curated evaluation set:

- **Stage 1 (API completeness):** Of 522 total kernels in the HeCBench repository, 329 provide implementations in all 4 primary APIs (CUDA, HIP, SYCL, OpenMP).
- **Stage 2 (Build infrastructure and verification):** Of these 329 kernels, 2 lack Makefiles and were excluded. Of the remaining 327, only 242 produce self-checking output (printing PASS/FAIL, computing checksums, or comparing against reference values). These 242 candidates proceed to complexity filtering.

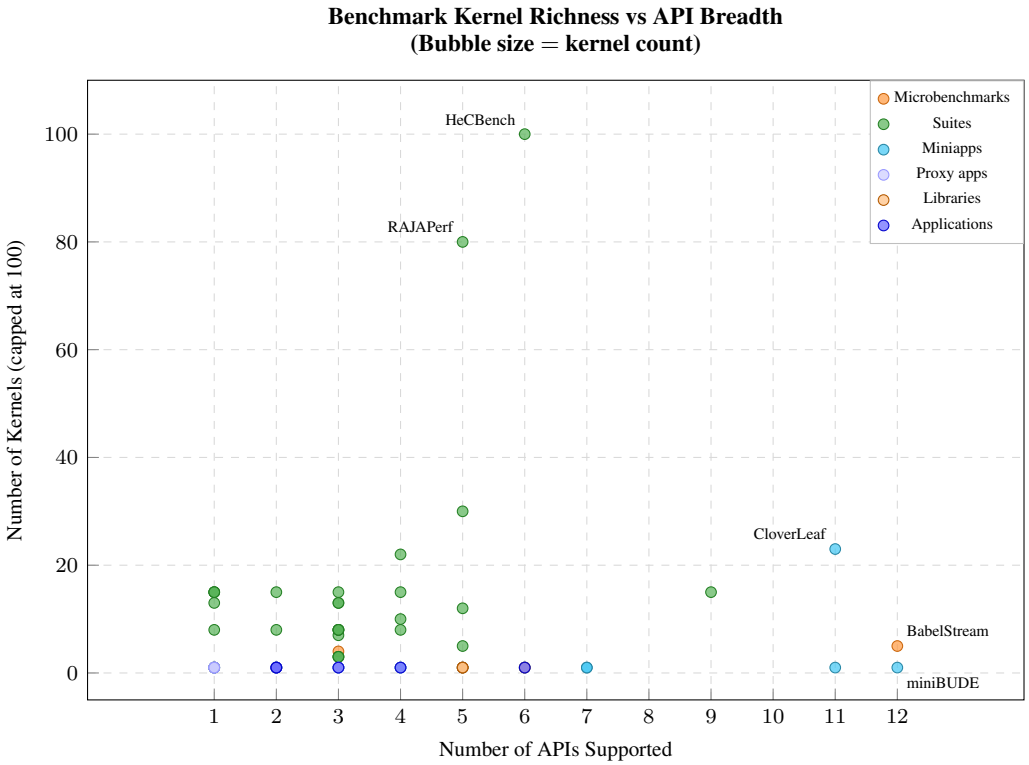

Figure 9: Benchmark kernel richness (y-axis, capped at 100 for readability) vs. API breadth (x-axis, number of APIs supported). Bubble size proportional to total kernel count; color indicates benchmark type. HeCBench (522 kernels, 6 APIs[8]) dominates the upper region. RAJAPerf (80 kernels, 5 APIs) is the second-largest. CloverLeaf (23 kernels, 11 APIs) and BabelStream (5 kernels, 12 APIs) occupy the high-API-breadth region but with far fewer kernels. The optimal candidates for corpus construction lie in the high-kernel, moderate-API quadrant (upper-center).

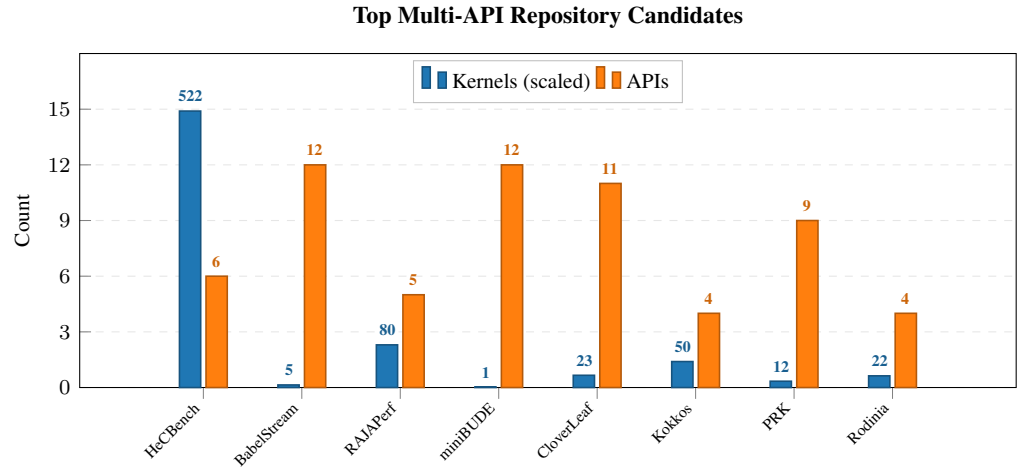

Figure 10: Top multi-API repository candidates ranked by kernel count (blue bars, scaled for readability; true counts annotated above) and API breadth (orange bars). HeCBench provides the largest kernel pool (522 kernels, 6 APIs spanning CUDA, HIP, SYCL, OpenMP, OpenMP-target offloading, and MPI). BabelStream and miniBUDE provide the broadest API coverage (12 APIs each) but with few kernels. RAJAPerf (80 kernels, 5 APIs) offers a balanced middle ground. Rodinia (22 kernels, 3 APIs) provides established, well-understood benchmarks with strong community recognition.

- **Stage 3 (Complexity filter):** The 242 candidates were filtered for evaluation tractability. Kernels with more than 15 source files or fewer than 2 were excluded (the former are too complex for single-prompt translation; the latter are trivial boilerplate). Kernels requiring external input data files were also excluded to ensure self-contained evaluation.
- **Stage 4 (Domain diversity):** From the complexity-filtered set, 60 kernels were manually selected to ensure coverage across 10 computational categories: linear algebra, stencil computation, graph algorithms, machine learning, physics simulation, sorting, reduction, scan, image processing, and cryptography.
- **Stage 5 (Curation):** The 60-kernel working set was narrowed to a final curated set of 20 kernels based on manual inspection of build reliability, output determinism, and domain representativeness. Of these, 10 kernels were included in the current PARBENCH evaluation corpus, with the remaining 10 reserved for future expansion.

The 10 curated kernels included in the current corpus are: `convolution1d`, `floydwarshall`, `heat2d`, `iso2dfd`, `jacobi`, `md`, `nqueen`, `page-rank`, `scan`, and `stencil1d`. Each provides a CUDA implementation paired with one or more OpenMP/OpenMP-target variants, totaling 25 specs. Two of these are KNOWN_FAIL: `stencil1d-omp_target` (build failure due to target-offloading compilation) and `scan-omp_target` (output mismatch on the CPU offload target), leaving 23 in the active evaluation corpus.

E.4 Rodinia Kernel Inventory

Rodinia Che et al. [2009] provides the foundation of PARBENCH's evaluation corpus. As one of the most widely cited heterogeneous computing benchmark suites, it offers well-understood computational characteristics across CUDA, OpenMP, and OpenCL. PARBENCH includes all 22 Rodinia kernels, yielding 60 total specs; five kernels lack one or more API variants because the upstream repository does not provide implementations for all three APIs (six API variants absent in total, as one kernel—`huffman`—lacks both OpenMP and OpenCL). Of these 60 specs:

- **53 PASS**: Verified via conjunctive verification—exit code 0 *and* stdout pattern match must both hold. Five of these employ stronger oracle methods: three (`hotspot3d`) use numeric comparison against baseline output within a fixed tolerance, and two (`bptree`) use cryptographic file hashing for bitwise-exact output matching.
- **7 KNOWN_FAIL**: Excluded from evaluation due to pre-existing platform failures unrelated to LLM translation quality: 2 specs use deprecated CUDA `texture<>` references removed in CUDA 12 (`kmeans-cuda`, `mummergpu-cuda`); 1 requires OpenGL (`hybridsort-cuda`); 1 has CUDA API dependencies in its OpenMP variant (`mummergpu-omp`); 2 exhibit pre-existing OpenCL segmentation faults (`nn-opencl`, `kmeans-opencl`); and 1 has a silent `clEnqueueReadBuffer` error masked by exit code 0 (`backprop-opencl`).

E.5 XSBench, RSBench, and mixbench Profiles

Three additional benchmark suites complement Rodinia and HeCBench in the evaluation corpus:

XSBench Tramm et al. [2014] (4 specs: CUDA, OpenMP, OpenCL, OpenMP Target): A Monte Carlo neutron cross-section lookup proxy application representative of nuclear reactor simulation workloads. XSBench implements a unionized energy grid algorithm; the OpenMP variant runs in history-based mode (checksum 941,535) while the CUDA, OpenCL, and OpenMP Target variants run in event-based mode (checksum 945,990). Both checksums are self-verified as valid by XSBench's built-in verification. This mode asymmetry reflects algorithmic differences between CPU and GPU scheduling strategies rather than correctness issues. All 4 API variants pass baseline verification.

RSBench Tramm and Siegel [2014] (4 specs: CUDA, OpenMP, OpenCL, OpenMP Target): A proxy application for the multipole method of computing neutron cross sections on-the-fly, representing a complementary algorithm to XSBench's unionized energy grid approach. Including both suites enables controlled comparison within the same computational domain (Monte Carlo neutron transport) at different algorithmic complexity levels. All 4 API variants pass baseline verification.

mixbench Konstantinidis and Cotronis [2017] (3 specs: CUDA, OpenMP, OpenCL): A micro-benchmark that sweeps a range of compute-to-memory operation ratios, mapping the practical

roofline boundary of a compute device. Its parameterized kernel structure tests whether LLM translations preserve arithmetic intensity across API boundaries. All 3 API variants pass baseline verification.

E.6 Exclusion Log and Verification Methods

Figure 11 groups the surveyed benchmarks by their verification approach, illustrating the diversity of correctness checking methods across the survey and the basis for exclusion decisions.

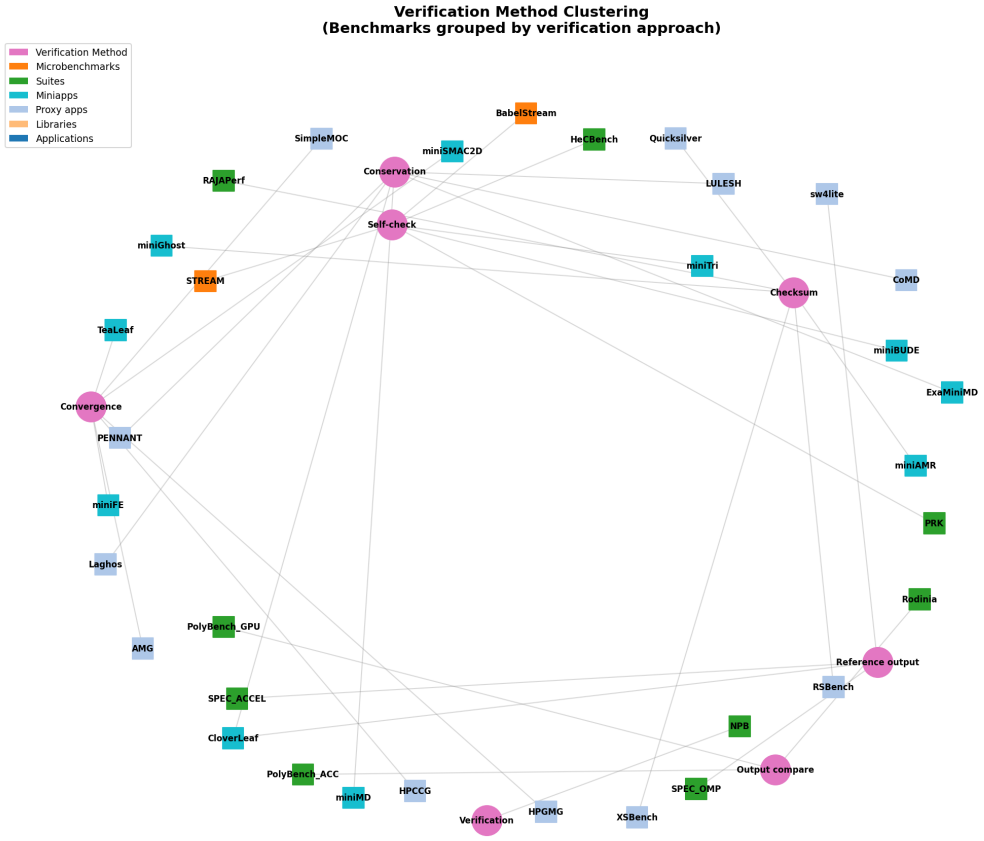

Figure 11: Benchmarks grouped by verification method. Large nodes represent verification approaches; benchmark nodes are connected to their method(s). Benchmarks without automated verification were excluded or downgraded to Tier B (Section E.2). The checksum and reference output clusters contain the majority of Tier A benchmarks.

Repository-Level Exclusions

Of the 71 surveyed repositories, 36 were excluded in two stages. First, 31 were removed during initial screening (single-API only, no HPC kernel content, or outside the CUDA/OpenMP/OpenCL scope), leaving 40 candidates. Of these, five were excluded during detailed evaluation:

- **Download failures** (2): Repository URLs were dead or access-restricted at the time of survey.
- **Insufficient documentation** (2): No build instructions, no Makefiles, or build system required proprietary dependencies not available on the evaluation platform.
- **Duplicate content** (1): One repository was a fork of an already-surveyed suite with no additional API variants.

Kernel-Level Exclusions

Within the selected suites, individual kernels were excluded for the following reasons:

- **Missing API variants**: Kernels available in only one API (no cross-translation possible).
- **No self-checking output**: Kernels that produce graphical output, require manual inspection, or have no deterministic expected output.
- **Excessive complexity**: Kernels requiring more than 15 source files, external data files not included in the repository, or runtime dependencies (e.g., OpenGL, MPI multi-node) not available on the single-GPU evaluation platform.
- **Non-deterministic output**: Kernels whose output depends on execution order, random seeds without fixed initialization, or floating-point non-associativity beyond the tolerance of the verification method.

The complete exclusion log with per-kernel justifications is recorded in PARBENCH's specification metadata (`specs/*.json`).

F Detailed Evaluation Findings

This appendix provides per-kernel and per-direction breakdowns of the evaluation results summarized in Section 5 of the main paper, including augmentation transform statistics, the HeCBench selection funnel, and a detailed case study of multi-file translation consistency failures.

F.1 Augmentation Transform Frequency Per Kernel

PARBENCH's code augmentation pipeline applies six AST-level transforms at four intensity levels (L1–L4). The transforms do not apply uniformly: some kernels have many candidate sites for a given transform, while others have few or none. Figure 12 presents the per-kernel, per-transform frequency heatmap.

Aggregate transform frequencies across the corpus reveal a clear dominance hierarchy. Below we report the number of Rodinia specs (out of 60) to which each transform applies at L4 (full intensity), followed by the total application count across all 240 augmentation tasks (60 specs $\times$ 4 levels):

- **SwapCondition** (59/60 specs at L4; 162 applications across all levels): The most broadly applicable transform. Swaps operands of comparison operators (e.g., `a < b` $\rightarrow$ `b > a`), with guards against expressions containing side effects. Its near-universal coverage reflects the ubiquity of comparison-based control flow in HPC kernels—boundary checks, convergence tests, and branch-based algorithm selection all contribute candidate sites.
- **ArithmeticTransform** (45/60 specs at L4; 69 applications): Converts compound assignment operators to their expanded forms and vice versa (e.g., `x += expr` $\leftrightarrow$ `x = x + (expr)`). Applies only to integer-typed scalar targets and excludes for-loop incrementors to avoid disrupting iteration patterns. Less broadly applicable than SwapCondition because it requires compound-assignment or assignment-with-matching-LHS patterns that not all kernels exhibit.
- **ChangeNames** (32/60 specs at L4; 55 applications): Consistently renames local variables and function parameters using symbol-aware reference collection, preserving semantic equivalence. Coverage is bounded by the number of renameable local declarations per kernel.
- **TypedefExpansion** (3/60 specs at L4; 7 applications): Inlines `typedef` aliases to their underlying types. Rare because most Rodinia kernels use primitive types directly rather than defining type aliases.
- **PointerArithmeticToArrayIndex** (3/60 specs at L4; 6 applications): Rewrites `*(ptr + i)` to `ptr[i]` and vice versa, with precedence-aware handling of struct member access. Limited by the prevalence of pointer arithmetic in the corpus—most kernels use array indexing natively.

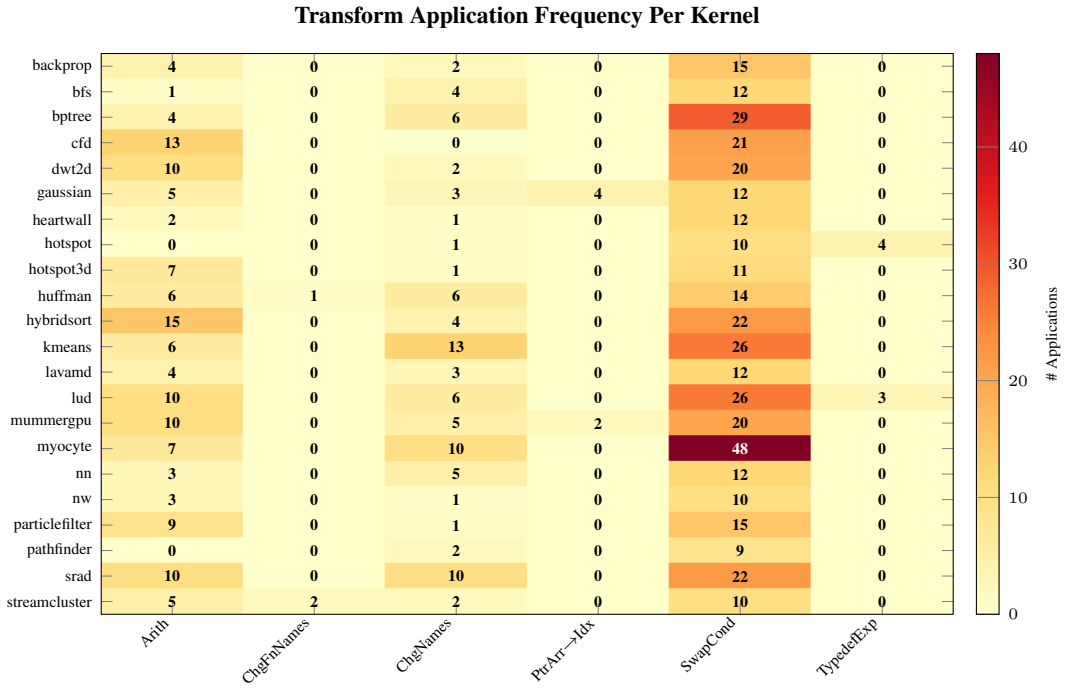

Figure 12: Per-kernel transform application frequency across the Rodinia corpus. Rows represent the 22 Rodinia kernels; columns represent the six AST-level transforms. Cell values give the total number of times each transform was applied to files belonging to that kernel, aggregated across all augmentation levels and API variants. The distribution is highly non-uniform: multi-file kernels such as `myocyte` (15 source files) and `bptree` accumulate many SwapCondition applications, while smaller kernels (e.g., `pathfinder`, `hotspot`) show lower counts. ChangeFunctionNames applies to only two kernels (`huffman`, `streamcluster`).

- **ChangeFunctionNames** (2/60 specs at L4; 2 applications): Renames user-defined functions and updates all call sites, skipping `main`, GPU kernel entry points, and identifiers appearing in string literals. The rarest transform, reflecting the small number of user-defined functions in single-kernel files.

The level-invariance property confirms that augmentation introduces no new correctness failures to the benchmark baseline: all 53 non-KNOWN_FAIL Rodinia specs pass at every augmentation level L1–L4. (Note that the per-transform applicability fractions above, e.g., 59/60 for SwapCondition, are computed over all 60 Rodinia specs including the 7 KNOWN_FAIL specs, since augmentation is applied to source files regardless of baseline status; the correctness validation uses only the 53 non-KNOWN_FAIL specs.) All transforms are designed to preserve semantic equivalence, verified by 15 unit tests covering each transform in isolation and by the end-to-end augmentation sweep across the full Rodinia corpus. Across all five suites, 80 of 87 non-KNOWN_FAIL specs pass at all levels L1–L4, with the 7 failures confined to `omp_target` variants where aggressive variable renaming interferes with OpenMP target data-mapping clauses (a known limitation of the current transform guards; all 87 specs pass at L1–L2).

From an evaluation perspective, augmentation levels serve as a robustness probe rather than as independent samples that inflate the effective sample size. The L0-conditional ablation (Section 5) confirms this discriminative role across all three models: GPT-5.4 and GPT-5.3-codex maintain high pass rates under augmentation (87–91% and 85–89% at L1–L4 among their respective L0-passing tasks), while Qwen 3.5 degrades from 74.0% at L1 to 56.0% at L4. The monotonic decline for Qwen 3.5 suggests that semantics-preserving source perturbations expose fragility in weaker models' translation capabilities—exactly the behavior a robustness probe should surface.

F.2 HeCBench Kernel Selection Funnel

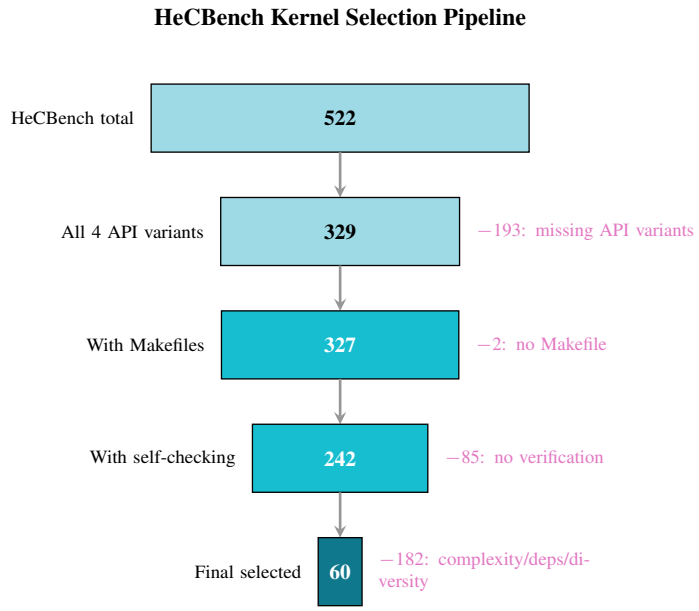

Figure 13: HeCBench kernel selection funnel. Starting from 522 kernels spanning CUDA, HIP, SYCL, and OpenMP at commit `22785cdd`, successive filters for API completeness (329 with all 4 variants), build infrastructure (327), self-checking output patterns (242), and complexity bounds yield a 60-kernel working set. Manual curation further narrows this to 10 evaluation kernels (25 specs; see Section 3).

HeCBench is the largest source of multi-API kernel implementations in the benchmark survey, with 522 kernels spanning CUDA, HIP, SYCL, and OpenMP (Figure 13). Its scale necessitated a structured selection funnel to identify kernels suitable for automated evaluation:

1. **API completeness** (522 → 329): Retained only kernels providing implementations in all 4 primary APIs. Kernels missing one or more API variants cannot support the full matrix of cross-API translation directions that ParBench evaluates.
2. **Build infrastructure** (329 → 327): Excluded 2 kernels whose CUDA variants lacked Makefiles or had broken build configurations. All retained kernels build successfully with `make` on the evaluation platform (NVIDIA HPC SDK 24.3, CUDA 12.3).
3. **Self-checking output** (327 → 242): Excluded kernels without deterministic, machine-parseable output. Retained kernels print checksums, PASS/FAIL indicators, or numerical results that can be matched against regular expression patterns for automated verification.
4. **Complexity bounds** (242 → 60): Excluded kernels with more than 15 source files (too complex for single-prompt LLM translation) or fewer than 2 source files (trivial boilerplate). Also excluded kernels requiring external input data files not bundled with the repository.
5. **Domain diversity and curation** (60 → 20): Manual inspection for build reliability, output determinism, and domain representativeness. The final 20 kernels span linear algebra, stencil computation, graph algorithms, machine learning, physics simulation, sorting, reduction, scan, image processing, and cryptography. Ten kernels are included in the current evaluation corpus; ten are reserved for future expansion.

This funnel illustrates a general challenge in constructing LLM code translation benchmarks: the vast majority of available HPC kernels are unsuitable for automated evaluation due to missing API variants, non-deterministic output, or excessive complexity. Only 3.8% of HeCBench's 522 kernels (20/522) survive the full selection pipeline, highlighting the gap between "available parallel code" and "evaluable parallel code."

F.3 Case Study: Multi-File Translation Consistency Failure

Full-program translations—where the LLM must rewrite both kernel code and host code—expose a class of errors absent from kernel-only translations. We document this failure class through a detailed case study of the `rodinia-lavamd` kernel, a molecular dynamics simulation with a multi-file build structure (kernel, wrapper, and supporting headers).

Observation

Across all six standard translation directions, Qwen 3.5 fails every `lavamd` translation: 0 of 18 records (6 directions $\times$ 3 independent samples at temperature 0.7) produce a correct program. The failure mode is not uniform—it varies systematically by translation direction rather than by stochastic sample, revealing direction-dependent structural barriers in multi-file coordination.

Per-Direction Error Taxonomy

Table 10: Multi-file translation error taxonomy for `lavamd` across all six standard directions (Qwen 3.5, L0, 3 samples each). The failure mode varies by direction, demonstrating that multi-file coordination errors are structurally determined by the source–target API pairing rather than stochastic.

Direction	Status ($\times$3)	Primary Failure Mode
OpenCL→CUDA	BUILD_FAIL	Phantom include (`timing.h`)
OMP→CUDA	BUILD_FAIL	Phantom include (`main.h` path)
CUDA→OMP	BUILD_FAIL	Phantom include (`./main.h`)
CUDA→OpenCL	RUN_FAIL	OpenCL kernel compilation failure
OMP→OpenCL	RUN_FAIL	OpenCL kernel compilation failure
OpenCL→OMP	Mixed	2 BUILD_FAIL + 1 VERIFY_FAIL

Table 10 reveals two structural patterns. First, all directions targeting CUDA fail at build time due to phantom include paths: the model hallucinates headers (`timing.h`, `./main.h`, `./util/timer/timer.h`) that exist in the source program's directory tree but are unavailable in the translated program's build context. All three samples within each CUDA-targeting direction fail on the same phantom header, indicating a deterministic error rather than stochastic variation. Second, directions targeting OpenCL pass host-code compilation but fail at runtime when `clBuildProgram` attempts to compile the translated OpenCL kernel source (exit code 245 = `CL_BUILD_PROGRAM_FAILURE`). The runtime compilation errors include incorrect `__local` variable scoping and undeclared identifiers—API-specific constraints that the model fails to satisfy in the translated kernel.

Cross-Model Persistence

The direction-dependent failure pattern persists across models. Only two of six directions yield any PASS outcomes across the three models: OMP→OpenCL (GPT-5.4 and GPT-5.3-codex each 3/3 PASS, Qwen 3.5 0/3) and CUDA→OMP (GPT-5.4 2/3 PASS, Qwen 3.5 and GPT-5.3-codex 0/3). The remaining four directions are universally failed across all models and samples (0/36 PASS). OpenCL→CUDA and OMP→CUDA fail at build time due to phantom includes (18/18 BUILD_FAIL), CUDA→OpenCL fails at runtime due to OpenCL kernel compilation errors (9/9 RUN_FAIL), and OpenCL→OMP fails with a mix of phantom includes and output mismatches. These asymmetries indicate that multi-file coordination difficulty is a property of the translation direction—not solely of the kernel complexity or the model.

Implications

This case study identifies three distinct error subcategories within the "multi-file translation consistency failure" class:

1. **Phantom dependencies**: The model hallucinates include files (`timing.h`, `./main.h`) or helper functions (`checkCUDAError`, `setdevice`) that may exist in the model's training data but are absent from the target program's file structure. This is the dominant build-time failure mode for CUDA-targeting directions across all three models.

2. **Cross-file identifier inconsistency**: In some translations (observed in GPT-5.3-codex and GPT-5.4 OpenCL→CUDA attempts), the kernel file uses the target API's naming convention while the wrapper file retains the source API's function name, producing linker errors (`undefined reference to 'kernel_gpu_cuda'`).
3. **Runtime kernel compilation failure**: For OpenCL-targeting directions, the host C/C++ code compile successfully, but the OpenCL kernel source—compiled at runtime by `clBuildProgram`—contains errors such as incorrect `__local` variable scoping and undeclared type aliases. This represents a disconnect between the C/C++ compiler's view of the host code and the OpenCL runtime compiler's requirements for device code, a failure mode invisible to static build checks.

These findings suggest that multi-file translation failures are not uniformly distributed across directions. Directions requiring the model to generate host-side boilerplate for a fundamentally different runtime (e.g., CUDA driver API, OpenCL `cl*` calls) are systematically harder than directions where the host code structure is similar between source and target. A hybrid pipeline—combining LLM translation for computational kernels with template-based scaffold generation for host infrastructure—may address subcategories 1 and 2 but would not resolve the runtime compilation failures in subcategory 3, which require correct translation of API-specific device-code constraints.

F.4 Extended Results Details

This section expands on the results presented in Section 5. Across all three models, we collected 2,262 valid translation records: 626 for Qwen 3.5 (426 L0 + 200 augmentations, after excluding 82 records involving 9 KNOWN_FAIL specs[9]), 822 for GPT-5.4 (426 L0 + 396 augmentations), and 814 for GPT-5.3-codex (426 L0 + 388 augmentations). KNOWN_FAIL specs were pre-excluded from both GPT evaluation batches.

Table 11: pass@k results for Qwen 3.5 (L0 only, temperature 0.7, three independent samples per task, 142 unique source-target pairs). CIs are normal-approximation intervals on the macro-averaged Chen et al. estimator (distinct from the Wilson binomial CIs used for per-record pass rates in Table 2)

Metric	Value [95% CI]
pass@1	23.9% [17.9%, 30.0%]
pass@3	35.2% [27.3%, 43.1%]
Always pass (3/3)	19 tasks (13.4%)
Noisy (1–2/3)	31 tasks (21.8%)
Hard fail (0/3)	92 tasks (64.8%)

pass@k Analysis. The 36.7% per-record rate (Table 2) treats each sample and augmentation level independently. The pass@k analysis (Table 11) restricts to the 142 L0 tasks and macro-averages over tasks to estimate single-sample and three-sample success probabilities, with each task generating three independent samples at temperature 0.7. The gap between pass@1 (23.9%) and pass@3 (35.2%) is 11.3%: only 31 of 142 tasks show within-task variability (1 or 2 of 3 samples pass); none of the 92 hard failures (0/3) benefit from resampling. This confirms that failures are predominantly systematic rather than stochastic.

Representative Per-Kernel Pass Rates. Table 12 highlights the easiest and hardest kernels for Qwen 3.5.

Augmentation Pass Rates. If baseline success were driven primarily by surface-form memorization, augmented source variants should degrade pass rates. On a balanced 12-kernel CUDA-to-OpenMP subset present at all five augmentation levels (Table 13), pass rates remain between 75% and 83% at L1–L4. The L0 rate on this subset (80.6%) exceeds the overall CUDA-to-OpenMP L0 rate (40.3%) because the ablation filter selects kernels that already pass at L0, creating a survivorship bias toward easier kernels. We computed Cochran–Armitage trend tests, but two confounds undermine their

[9] The Qwen 3.5 canonical total is 626, not 630, because four `rodinia-backprop-opencl` L1–L4 augmentation records are excluded via the KNOWN_FAIL policy (the OpenCL baseline is broken).

Table 12: Representative per-kernel pass rates for Qwen 3.5 across all directions and augmentation levels (626 records, 31 evaluated kernels—4 curated kernels have no evaluable translation pairs).

Suite	Kernel	Total	PASS	Fail	Rate
Easiest kernels					
HeCBench	iso2dfd	38	32	6	84.2%
HeCBench	floydwarshall	38	29	9	76.3%
HeCBench	heat2d	38	29	9	76.3%
HeCBench	stencil1d	14	9	5	64.3%
HeCBench	page-rank	10	6	4	60.0%
Hardest kernels (0% pass rate, $n \geq 18$)					
Rodinia	heartwall	18	0	18	0.0%
Rodinia	myocyte	18	0	18	0.0%
Rodinia	bptree	18	0	18	0.0%
RSBench	rsbench	18	0	18	0.0%
Rodinia	lavamd	18	0	18	0.0%

Table 13: Augmentation pass rates on the common 12-kernel CUDA-to-OpenMP subset present at all five levels. These 12 kernels were selected by the L0-conditional ablation filter ($\geq$1 of 3 L0 samples passes), so L0 rates exceed the overall CUDA-to-OpenMP rate of 40.3%.

Level	Pass / Total	Rate	Kernels with any pass
L0	29/36	80.6%	12/12 (100%)
L1	9/12	75.0%	9/12 (75%)
L2	10/12	83.3%	10/12 (83%)
L3	9/12	75.0%	9/12 (75%)
L4	10/12	83.3%	10/12 (83%)

interpretability: the 3:1 ratio of L0 to L1–L4 sample sizes on this subset, and the survivorship selection inherent in L0-conditional ablation. The observed stability is compatible with some robustness to surface-form perturbation, though it does not rule out memorization at other levels of abstraction.

G Supplementary Tables and Figures

This appendix contains detailed evidence tables and figures referenced from the main text. Floats are ordered by their original section appearance. Per-kernel translation heatmaps appear in Figures 17, 24, and 26. Pass@k by direction appears in Figures 18, 19, and 20. Cross-suite comparisons appear in Figures 21, 22, and 23. Table 19 summarizes the direction asymmetry and augmentation trend tests for Qwen 3.5.

Table 14: Programming model characteristics of the three primary translation APIs. These differences define the structural transformation challenges that LLM-based translation must address.

Property	CUDA	OpenMP	OpenCL
Execution model	SIMT (warps of 32 threads)	Fork-join thread teams	NDRange work-groups
Memory management	Explicit device allocation and copies	Implicit shared CPU memory	Explicit buffer objects and command queues
Kernel specification	Compiled with host code	Pragma directives over loops	Runtime string compilation
Source structure	Single or multi-file (18 of 35 kernels are multi-file)	Single file (directives annotate sequential code)	Separate host program + `.cl` kernel files
Thread indexing	Block/thread indices	Loop variable or thread query	Global/local ID queries

Table 17 reports pass rates across all augmentation levels. GPT-5.4 and GPT-5.3-codex both maintain >85% pass rates across L1–L4 on all directions, while Qwen 3.5 shows a monotonic decline from 74.0% at L1 to 56.0% at L4 (Cochran–Armitage $z = -1.84$, $p = 0.065$). On CUDA-to-OMP specifically, both GPT models sustain 81–96% across augmentation levels, whereas Qwen 3.5 fluctuates between 75% and 83% on a smaller conditional subset ($n = 12$). This pattern suggests that weaker models are more sensitive to surface-form perturbation, though the small L0-conditional sample sizes—particularly for Qwen 3.5—limit the strength of this conclusion. Figure 15 plots the

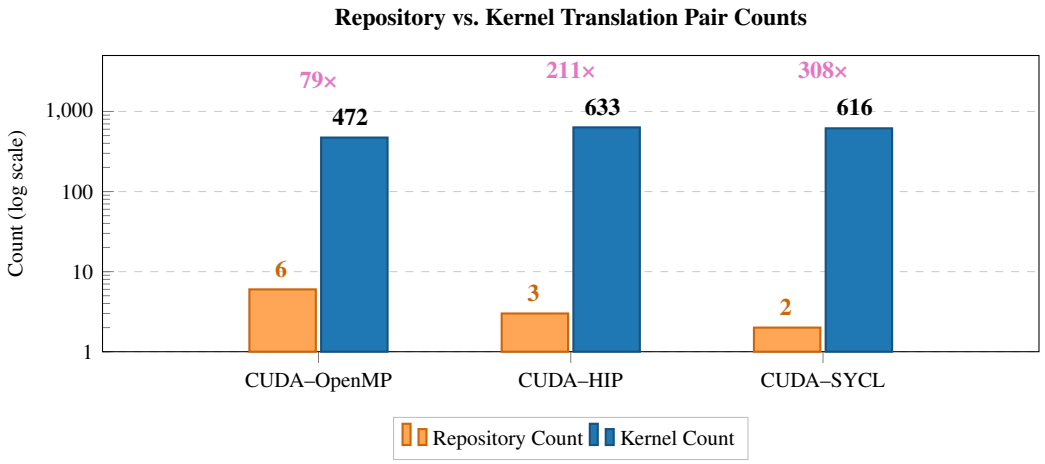

Figure 14: Repository-level vs. kernel-level translation pair counts. Left bars show the number of repositories containing both APIs; right bars show the number of independent kernel-level translation pairs. The multipliers (79×–308×) demonstrate that repository-level counting substantially underestimates the translation evaluation opportunity.

Table 15: Model configurations. †Azure reasoning models do not accept explicit temperature or top-p parameters; sampling is controlled internally by the provider.

Model	Provider	Arch.	Parameters	Reasoning	Temp.
Qwen 3.5 397B-A17B	Together AI	MoE	397B / 17B active	`enable_thinking`	0.7
GPT-5.4	Azure OpenAI	proprietary	proprietary	`reasoning_effort=medium`	N/A†
GPT-5.3-codex	Azure OpenAI	proprietary	proprietary	`reasoning_effort=medium`	N/A†

augmentation robustness trend across all three models; Figure 16 shows the per-kernel augmentation status for the CUDA-to-OMP subset under Qwen 3.5.

The full pass@k estimates appear in Table 18. GPT-5.4 and GPT-5.3-codex share identical pass@1 rates (62.7%) but differ in consistency: GPT-5.3-codex has more deterministic behavior, with 57.0% of pairs always passing (vs. 53.5% for GPT-5.4) and fewer noisy pairs (11.3% vs. 16.2%). The pass@1-to-pass@3 gap is correspondingly narrower for GPT-5.3-codex (5.6 pp vs. 7.0 pp). This pattern suggests code-specialized training yields more consistent but not more capable translations at this task difficulty. Across all three models, the majority of failures are hard failures (0/3 samples pass), confirming that incorrect translations are systematic—typically wrong API mappings—rather than stochastic errors recoverable by resampling.

Table 16: Hardware and software configuration (shared across all model evaluations).

Component	Specification
GPU	NVIDIA GeForce RTX 4070 (12 GB, Ada Lovelace, sm_89)
CPU	AMD Ryzen 9 7900X (12 cores / 24 threads, 4.7 GHz)
RAM	32 GB DDR5
OS	Ubuntu 24.04 LTS
CUDA toolkit	NVIDIA HPC SDK 24.3 (nvcc 12.3)
C/C++ compiler	GCC 12.4.0
OpenMP	GCC `-fopenmp`; nvc++ 24.3 for target offload
OpenCL runtime	NVIDIA via HPC SDK 24.3
Python	3.12.3

Table 17: LLM pass rates across augmentation levels. L0 covers all 142 direction–kernel pairs ($n = 426$ tasks, temperature 0.7, three samples each). L1–L4 are evaluated deterministically (temperature 0, one sample each) on L0-conditional subsets: pairs where $\geq$1 of 3 L0 samples passed (Qwen 3.5: 50 pairs, GPT-5.4: 99 pairs, GPT-5.3-codex: 97 pairs).

Level	Qwen 3.5 (all dirs)	Qwen 3.5 (C→OMP)	GPT-5.4 (all dirs)	GPT-5.4 (C→OMP)	GPT-5.3-codex (all dirs)	GPT-5.3-codex (C→OMP)
L0	23.9% (n=426)	40.3% (n=72)	62.7% (n=426)	83.3% (n=72)	62.7% (n=426)	76.4% (n=72)
L1	74.0% (n=50)	75.0% (n=12)	88.9% (n=99)	90.9% (n=22)	86.6% (n=97)	85.0% (n=20)
L2	64.0% (n=50)	83.3% (n=12)	90.9% (n=99)	95.5% (n=22)	88.7% (n=97)	85.0% (n=20)
L3	62.0% (n=50)	75.0% (n=12)	86.9% (n=99)	81.8% (n=22)	86.6% (n=97)	85.0% (n=20)
L4	56.0% (n=50)	83.3% (n=12)	90.9% (n=99)	90.9% (n=22)	85.6% (n=97)	90.0% (n=20)

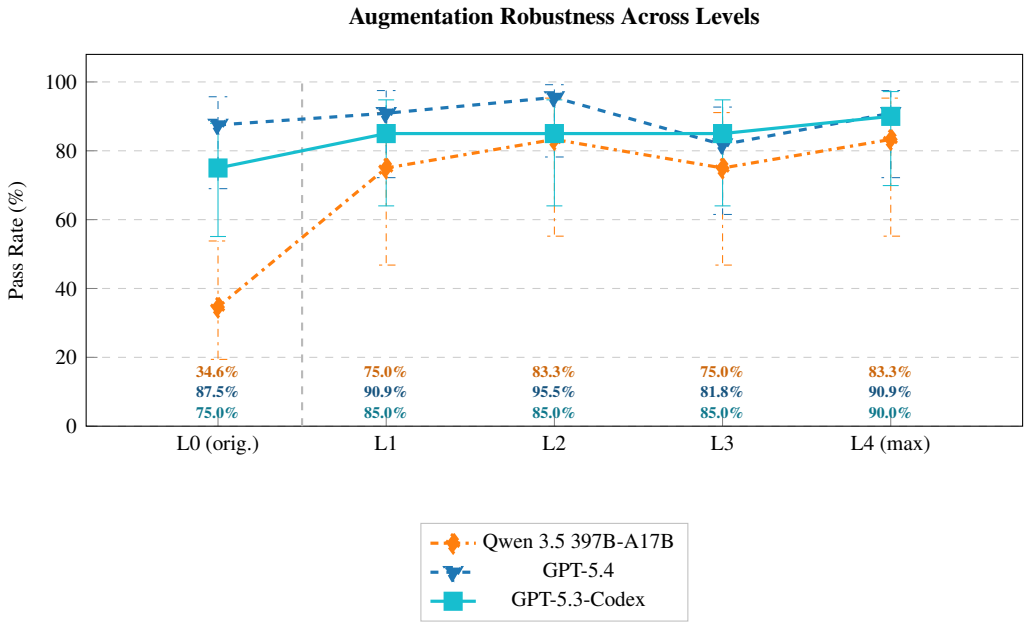

Figure 15: Augmentation robustness across all three models. First-sample per-kernel pass rate (%) at each augmentation level (L0–L4) on the L0-conditional CUDA-to-OMP subset. Error bars show Wilson 95% CIs.

Table 18: Pass@k estimates (L0, temperature 0.7, three independent samples per direction–kernel pair, zero-shot single-generation). Hard Fail = 0/3 samples pass; Always Pass = 3/3; Noisy = 1–2/3.

Model	Pairs	pass@1	pass@3	Hard Fail	Noisy	Always Pass
Qwen 3.5	142	23.9%	35.2%	92/142 (64.8%)	31/142 (21.8%)	19/142 (13.4%)
GPT-5.4	142	62.7%	69.7%	43/142 (30.3%)	23/142 (16.2%)	76/142 (53.5%)
GPT-5.3-codex	142	62.7%	68.3%	45/142 (31.7%)	16/142 (11.3%)	81/142 (57.0%)

Table 19: Statistical summary (Qwen 3.5 canonical evaluation). All confidence intervals are Wilson 95% CIs. McNemar tests use Bonferroni correction ($\alpha = 0.05/5 = 0.01$).

Test	Statistic	p-value	Effect Size	Interpretation
Qwen augmentation (Cochran–Armitage)	$z = 7.65$	$p < 0.001$	$h_{L0\to L1} = 1.05$	Significant increasing trend (L0 dominated; see text)
Direction: CUDA↔OMP (McNemar)	$n_{\text{paired}} = 24$, exact	$p = 0.180$	$\lvert h\rvert = 0.44$	No significant asymmetry ($\alpha = 0.01$)
Direction: CUDA↔OpenCL (McNemar)	$n_{\text{paired}} = 19$, exact	$p = 1.000$	$\lvert h\rvert = 0.46$	No significant asymmetry ($\alpha = 0.01$)
Direction: OMP↔OpenCL (McNemar)	$n_{\text{paired}} = 17$, exact	$p = 0.289$	$\lvert h\rvert = 0.53$	No significant asymmetry ($\alpha = 0.01$)
Direction: CUDA↔OMP-target (McNemar)	$n_{\text{paired}} = 8$, exact	$p = 0.031$	$\lvert h\rvert = 2.09$	Large effect but not significant ($\alpha = 0.01$; n too small)
Direction: OMP↔OMP-target (McNemar)	$n_{\text{paired}} = 3$, exact	$p = 1.000$	$\lvert h\rvert = 1.23$	Underpowered ($n = 3$)

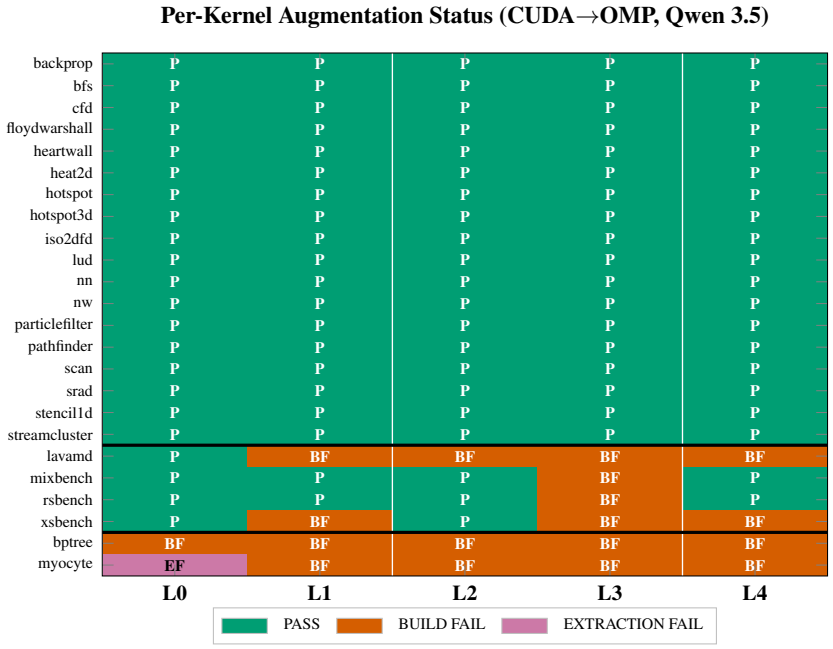

Figure 16: Per-kernel augmentation status across levels L0–L4 (CUDA-to-OMP, Qwen 3.5). Cell color indicates pass/fail status at each level.

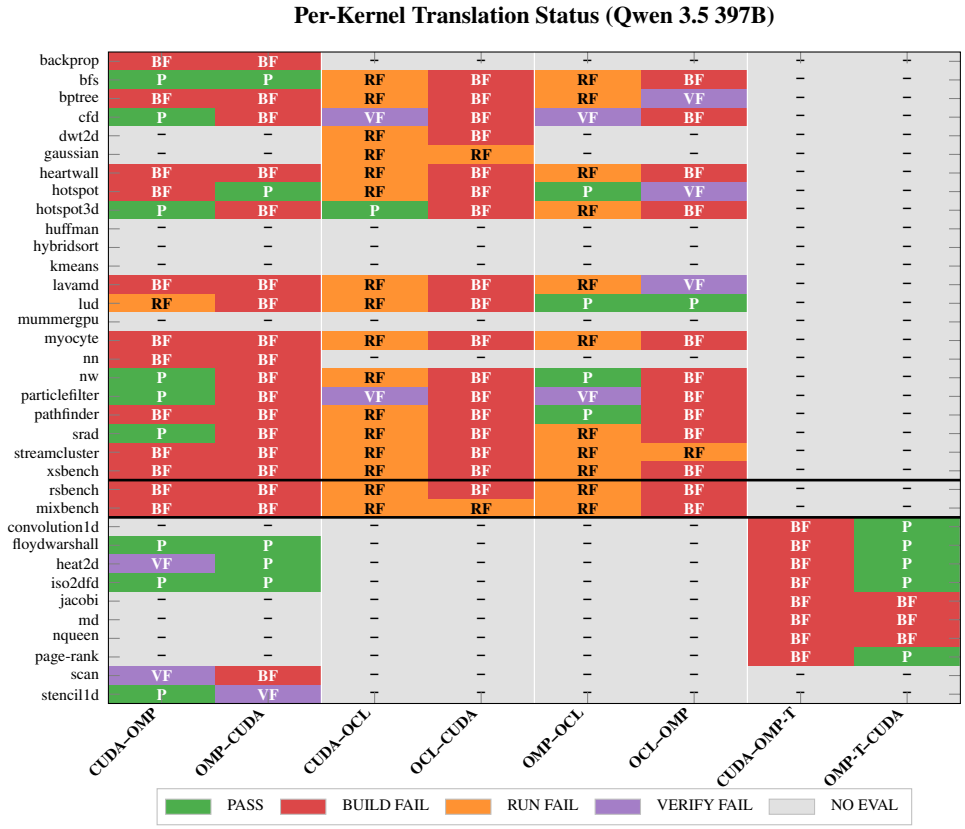

Figure 17: Per-kernel translation status across all directions (Qwen 3.5, L0, first sample per task, 31 evaluated kernels × up to 8 directions). Rows with all grey cells correspond to kernels whose specs are entirely KNOWN_FAIL or phantom. GPT-5.4 and GPT-5.3-codex heatmaps appear in Figures 24 and 26.

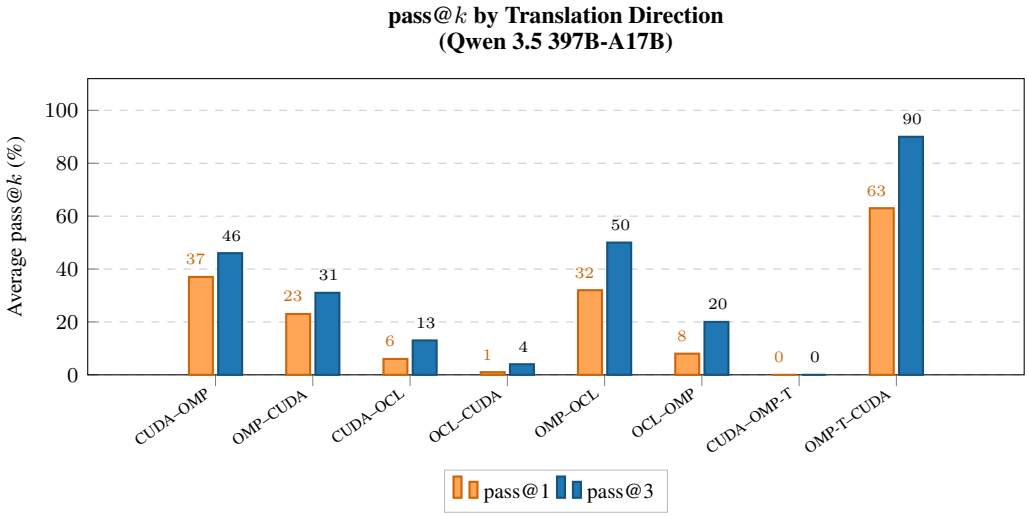

Figure 18: Pass@k by translation direction (Qwen 3.5, L0, three samples per task).

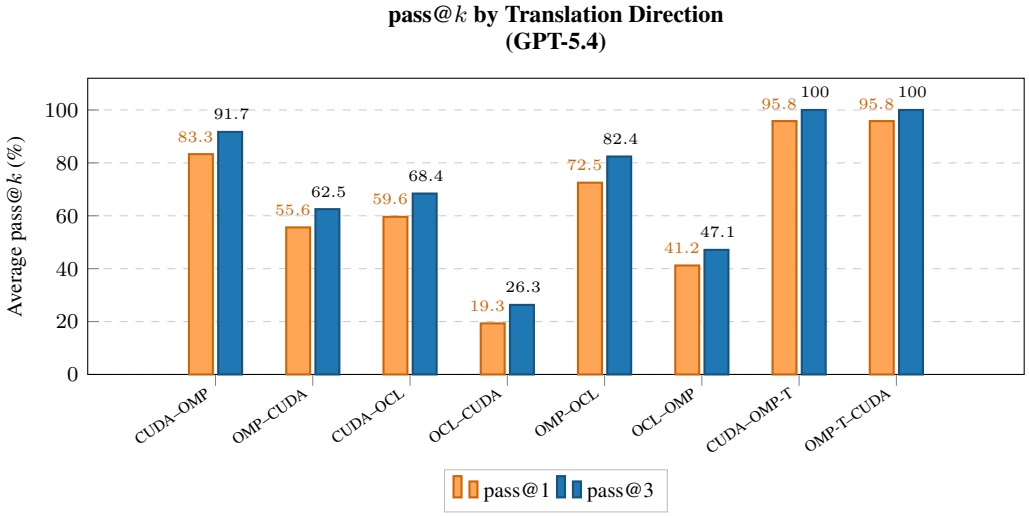

Figure 19: Pass@k by translation direction (GPT-5.4, L0, three samples per task).

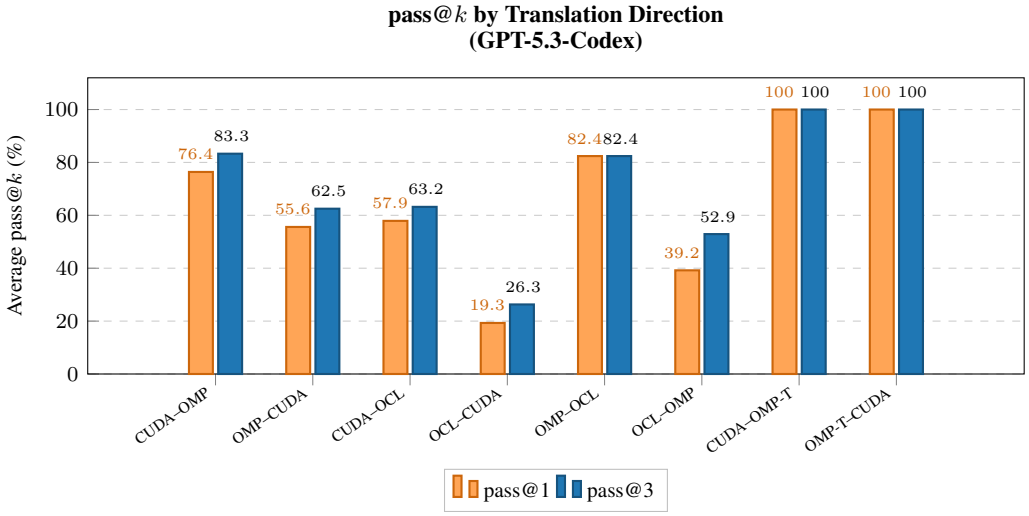

Figure 20: Pass@k by translation direction (GPT-5.3-codex, L0, three samples per task).

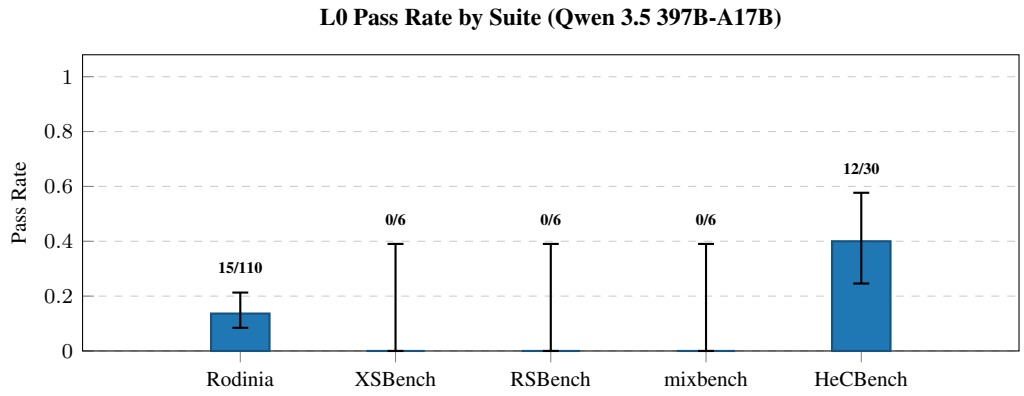

Figure 21: Cross-suite pass rate comparison (L0, Qwen 3.5). Per-suite aggregate pass rates with Wilson 95% CIs across all five benchmark suites.

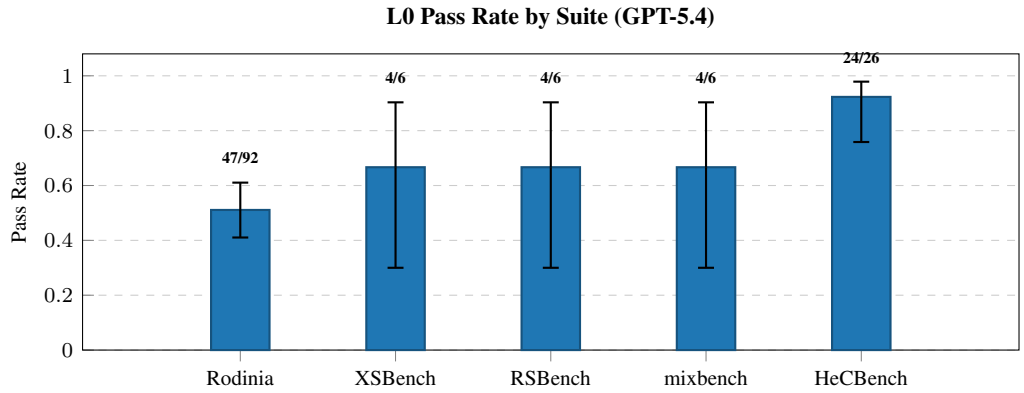

Figure 22: Cross-suite pass rate comparison (L0, GPT-5.4).

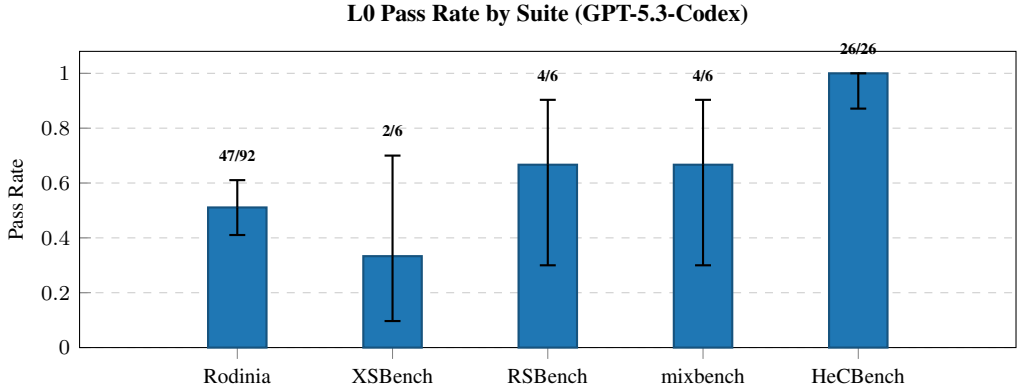

Figure 23: Cross-suite pass rate comparison (L0, GPT-5.3-codex).

Table 20: Full per-kernel pass rates across all 31 evaluated kernels (Qwen 3.5, 626 valid records including L0 and augmentation levels, 31 kernels across 5 suites). KNOWN_FAIL source specs excluded.

Suite	Kernel	Total	PASS	Fail	Rate	95% Wilson CI
HeCBench	iso2dfd	38	32	6	84.2%	[69.6%, 92.6%]
HeCBench	floydwarshall	38	29	9	76.3%	[60.8%, 87.0%]
HeCBench	heat2d	38	29	9	76.3%	[60.8%, 87.0%]
HeCBench	stencil1d	14	9	5	64.3%	[38.8%, 83.7%]
HeCBench	nqueen	10	6	4	60.0%	[31.3%, 83.2%]
HeCBench	page-rank	10	6	4	60.0%	[31.3%, 83.2%]
Rodinia	hotspot3d	34	20	14	58.8%	[42.2%, 73.6%]
Rodinia	hotspot	30	14	16	46.7%	[30.2%, 63.9%]
Rodinia	lud	30	14	16	46.7%	[30.2%, 63.9%]
Rodinia	bfs	34	15	19	44.1%	[28.9%, 60.5%]
HeCBench	md	10	4	6	40.0%	[16.8%, 68.7%]
Rodinia	nw	30	11	19	36.7%	[21.9%, 54.5%]
Rodinia	particlefilter	26	9	17	34.6%	[19.4%, 53.8%]
Rodinia	srad	26	9	17	34.6%	[19.4%, 53.8%]
HeCBench	jacobi	10	3	7	30.0%	[10.8%, 60.3%]
Rodinia	cfd	22	6	16	27.3%	[13.2%, 48.2%]
Rodinia	pathfinder	30	8	22	26.7%	[14.2%, 44.4%]
HeCBench	convolution1d	10	2	8	20.0%	[5.7%, 51.0%]
mixbench	mixbench	22	2	20	9.1%	[2.5%, 27.8%]
Rodinia	streamcluster	22	1	21	4.5%	[0.8%, 21.8%]
XSBench	xsbench	22	1	21	4.5%	[0.8%, 21.8%]
Rodinia	backprop	6	0	6	0.0%	[0.0%, 39.0%]
Rodinia	bptree	18	0	18	0.0%	[0.0%, 17.6%]
Rodinia	dwt2d	6	0	6	0.0%	[0.0%, 39.0%]
Rodinia	gaussian	6	0	6	0.0%	[0.0%, 39.0%]
Rodinia	heartwall	18	0	18	0.0%	[0.0%, 17.6%]
Rodinia	lavamd	18	0	18	0.0%	[0.0%, 17.6%]
Rodinia	myocyte	18	0	18	0.0%	[0.0%, 17.6%]
Rodinia	nn	6	0	6	0.0%	[0.0%, 39.0%]
RSBench	rsbench	18	0	18	0.0%	[0.0%, 17.6%]
HeCBench	scan	6	0	6	0.0%	[0.0%, 39.0%]

Table 21: Full per-kernel pass rates across all 31 evaluated kernels (GPT-5.4, 822 valid records including L0 and augmentation levels, 31 kernels across 5 suites). KNOWN_FAIL specs pre-excluded from evaluation batch.

Suite	Kernel	Total	PASS	Fail	Rate
HeCBench	convolution1d	14	14	0	100.0%
HeCBench	floydwarshall	42	41	1	97.6%
HeCBench	heat2d	42	40	2	95.2%
HeCBench	iso2dfd	42	42	0	100.0%
HeCBench	jacobi	14	14	0	100.0%
HeCBench	md	14	13	1	92.9%
HeCBench	nqueen	14	14	0	100.0%
HeCBench	page-rank	14	12	2	85.7%
HeCBench	scan	14	14	0	100.0%
HeCBench	stencil1d	14	14	0	100.0%
Rodinia	backprop	10	7	3	70.0%
Rodinia	bfs	38	35	3	92.1%
Rodinia	bptree	26	13	13	50.0%
Rodinia	cfd	34	28	6	82.4%
Rodinia	dwt2d	10	7	3	70.0%
Rodinia	gaussian	6	0	6	0.0%
Rodinia	heartwall	22	6	16	27.3%
Rodinia	hotspot	34	28	6	82.4%
Rodinia	hotspot3d	38	26	12	68.4%
Rodinia	lavamd	26	8	18	30.8%
Rodinia	lud	34	27	7	79.4%
Rodinia	myocyte	22	5	17	22.7%
Rodinia	nn	14	14	0	100.0%
Rodinia	nw	34	23	11	67.6%
Rodinia	particlefilter	42	33	9	78.6%
Rodinia	pathfinder	34	27	7	79.4%
Rodinia	srad	38	28	10	73.7%
Rodinia	streamcluster	22	7	15	31.8%
mixbench	mixbench	38	27	11	71.1%
RSBench	rsbench	38	29	9	76.3%
XSBench	xsbench	38	25	13	65.8%

Table 22: Full per-kernel pass rates across all 31 evaluated kernels (GPT-5.3-codex, 814 valid records including L0 and augmentation levels, 31 kernels across 5 suites). KNOWN_FAIL specs pre-excluded from evaluation batch.

Suite	Kernel	Total	PASS	Fail	Rate
HeCBench	convolution1d	14	14	0	100.0%
HeCBench	floydwarshall	42	40	2	95.2%
HeCBench	heat2d	42	39	3	92.9%
HeCBench	iso2dfd	42	42	0	100.0%
HeCBench	jacobi	14	14	0	100.0%
HeCBench	md	14	14	0	100.0%
HeCBench	nqueen	14	13	1	92.9%
HeCBench	page-rank	14	13	1	92.9%
HeCBench	scan	14	14	0	100.0%
HeCBench	stencil1d	14	14	0	100.0%
Rodinia	backprop	6	0	6	0.0%
Rodinia	bfs	38	35	3	92.1%
Rodinia	bptree	30	13	17	43.3%
Rodinia	cfd	34	27	7	79.4%
Rodinia	dwt2d	10	6	4	60.0%
Rodinia	gaussian	6	0	6	0.0%
Rodinia	heartwall	22	3	19	13.6%
Rodinia	hotspot	34	28	6	82.4%
Rodinia	hotspot3d	38	33	5	86.8%
Rodinia	lavamd	22	6	16	27.3%
Rodinia	lud	34	28	6	82.4%
Rodinia	myocyte	22	7	15	31.8%
Rodinia	nn	14	14	0	100.0%
Rodinia	nw	38	26	12	68.4%
Rodinia	particlefilter	38	29	9	76.3%
Rodinia	pathfinder	34	28	6	82.4%
Rodinia	srad	34	26	8	76.5%
Rodinia	streamcluster	22	7	15	31.8%
mixbench	mixbench	42	32	10	76.2%
RSBench	rsbench	38	20	18	52.6%
XSBench	xsbench	34	19	15	55.9%

H Translation Prompt Template

This appendix reproduces the translation prompt template used by the evaluation pipeline (`build_translation_prompt()` in `llm_evaluate.py`). All prompts follow this structure; only the source code content, target API, and file lists vary between tasks.

H.1 System Message

The system message is identical for all translation tasks:

```
You are a parallel programming expert
specializing in {src_api} to {tgt_api}
translation. Translate the provided source
code to {tgt_api}. Output ONLY the translated
code, no explanations. For each file, output
a markdown code fence with the filename on
the opening line:

```{tgt_lang} filename={filename}
<code>
```

Preserve the algorithm, I/O behavior, data
formats, and output format exactly. The
translated code must compile with the
provided build command.
```

Listing 4: System message template. `{src_api}` and `{tgt_api}` are replaced with API display names (e.g., "CUDA", "OpenMP"). `{tgt_lang}` is the target language hint for the code fence (e.g., "cuda", "cpp", "c").

H.2 User Message Structure

The user message is assembled from the following sections, in order:

1. **Translation Task** — source and target API display names. The kernel name and benchmark description are *not* included (anonymization).
2. **Target Files to Produce** — list of genericized target filenames (e.g., `translated_0.cpp`, `translated_1.cl`). When target infrastructure context is provided, an explanatory note clarifies that only these files replace existing project files.
3. **Build Command** — the anonymized compilation command in a code fence, so the LLM can ensure API and flag compatibility.
4. **Build Environment** — system dependencies from the target spec (e.g., "CUDA Toolkit $\geq$ 11.0", "GCC $\geq$ 9.0").
5. **Source Code** — each source file presented as a numbered subsection (`Source File 1`, `Source File 2`, ...) with all C/C++ comments stripped. The section heading includes the source API name (e.g., "Source Code (CUDA)").
6. **Support / Header Files** (optional) — source-directory headers and code files, genericized as `Header File` N and `Code File` N, with an instruction to inline definitions rather than emit unresolvable `#include` directives. Code files are included up to a cumulative 50,000-character limit.
7. **Target Infrastructure Context** (optional) — non-kernel target files (prompt payload entries not in `translation_targets`, plus target support headers) provided as read-only reference so the LLM can match expected function signatures and data structures. Headed "DO NOT MODIFY — for reference only" and genericized as `Infrastructure File` N. Omitted when no non-kernel target files or target support headers remain after filtering.

H.3 Anonymization Details

Six anonymization measures are applied during prompt construction to reduce the risk that benchmark-specific identifiers trigger memorized translations. These measures complement the AST-driven source augmentation (Section 2.3): anonymization targets prompt *metadata*, while augmentation modifies source code *structure*.

1. **Kernel identity omission.** The kernel name and benchmark description are excluded from the prompt entirely.
2. **Comment stripping.** All C/C++ line (`//`) and block (`/* */`) comments are removed from every file shown to the LLM—source files, support files, and target infrastructure files alike. The stripper uses a state-machine parser that preserves string, character, and raw string literals.
3. **Source and support filename genericization.** Source files are labeled `Source File 1`, `Source File 2`, etc. Support files are labeled `Header File` N or `Code File` N by extension. Target infrastructure files are labeled `Infrastructure File` N.
4. **Target filename genericization.** Target output filenames are replaced with `translated_0.ext`, `translated_1.ext`, etc., preserving original file extensions. An internal mapping restores original filenames when writing LLM output to disk, including any cross-file `#include` references the LLM emits using the generic names.
5. **Build command anonymization.** Kernel-specific identifiers are removed from the build command: `make` *`target`* is reduced to `make` (relying on the Makefile default target), and kernel names in other command strings are replaced with a generic placeholder.
6. **API-name retention.** The source and target API names (e.g., "CUDA", "OpenMP") are intentionally *not* anonymized, as they are essential to the translation task specification.

I GPT-5.4 Per-Model Figures

The following figures present per-kernel and per-direction breakdowns for GPT-5.4 (822 valid records: 426 L0 + 396 augmentation across 31 evaluated kernels). Both use L0 first-sample outcomes to match the visualization convention in Figure 2. Pass@k by direction and cross-suite comparisons appear in Appendix G (Figures 19 and 22). The full per-kernel table is in Table 21.

Compared with Qwen 3.5, the most salient differences are: (i) overall pass rate rises from 36.7% to 75.5%; (ii) BUILD_FAIL drops from 39.1% to 15.0% of records, indicating substantially better API-surface adaptation; and (iii) 11 of 31 kernels achieve $\geq$80% pass rate across their evaluable directions (vs. 0 for Qwen 3.5).

J GPT-5.3-codex Per-Model Figures

The following figures present per-kernel and per-direction breakdowns for GPT-5.3-codex (814 valid records: 426 L0 + 388 augmentation across 31 evaluated kernels). Both use L0 first-sample outcomes, matching the visualization convention in Figure 2. Pass@k by direction and cross-suite comparisons appear in Appendix G (Figures 20 and 23).The full per-kernel table is in Table 22. The failure taxonomy appears in Figure 27.

GPT-5.3-codex achieves a 74.2% overall pass rate with a failure profile intermediate between Qwen 3.5 and GPT-5.4: BUILD_FAIL accounts for 17.1% of records (vs. 39.1% for Qwen 3.5 and 15.0% for GPT-5.4). Compared with GPT-5.4, GPT-5.3-codex shows higher kernel-level consistency—13 of 31 kernels achieve $\geq$80% pass rate (vs. 11 for GPT-5.4)—despite identical macro pass@1 rates (62.7%).

K Evaluation Cost Summary

Table 23 reports the computational cost of each evaluation campaign. Token counts and response times are aggregated from per-result metadata recorded during evaluation; task counts include all

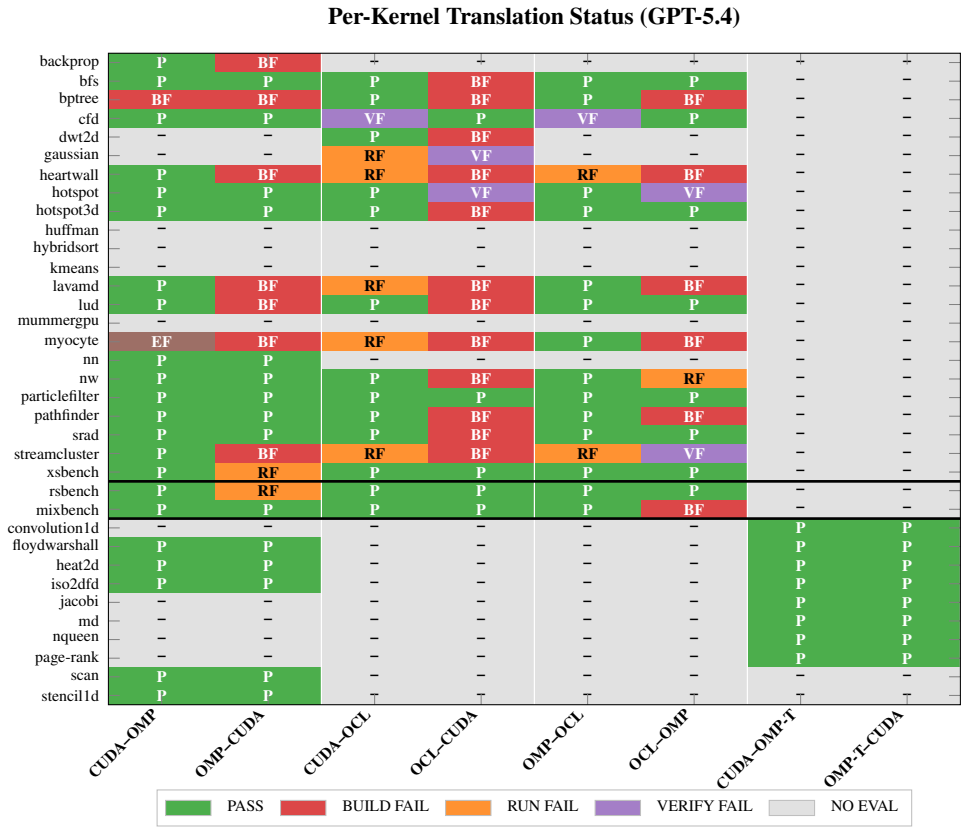

Figure 24: Per-kernel translation status across all directions (GPT-5.4, L0, first sample per task, 31 evaluated kernels × up to 8 directions). The corresponding Qwen 3.5 data is reported in Table 20; GPT-5.3-codex heatmap in Figure 26.

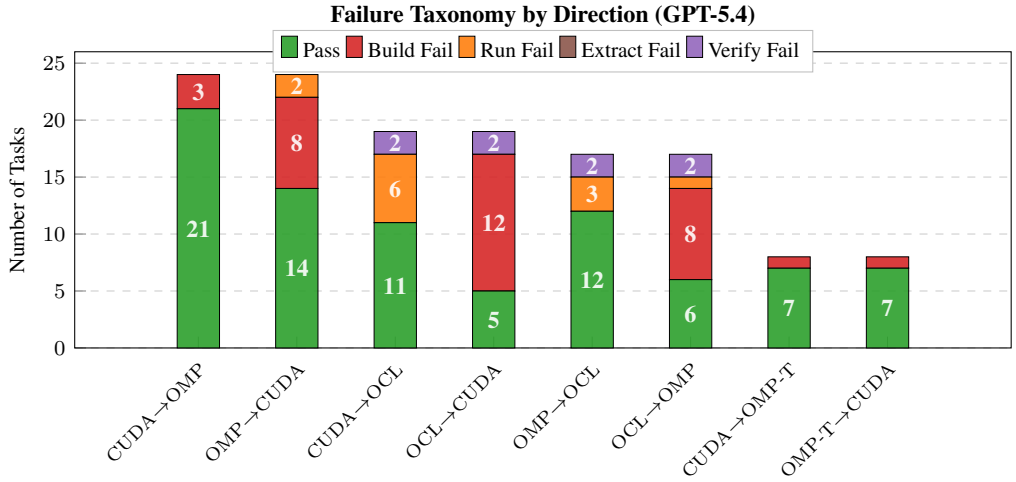

Figure 25: Failure taxonomy by translation direction (GPT-5.4, L0, first sample per task, 136 tasks across 8 directions including case-study pairs). Compare with Figure 2 (Qwen 3.5, 137 tasks across 6 standard directions; the count difference reflects KNOWN_FAIL pre-exclusion in the GPT batch). BUILD_FAIL concentrates in OMP→CUDA and OCL→OMP directions, consistent with the cross-file identifier inconsistency pattern documented in Appendix F.3.

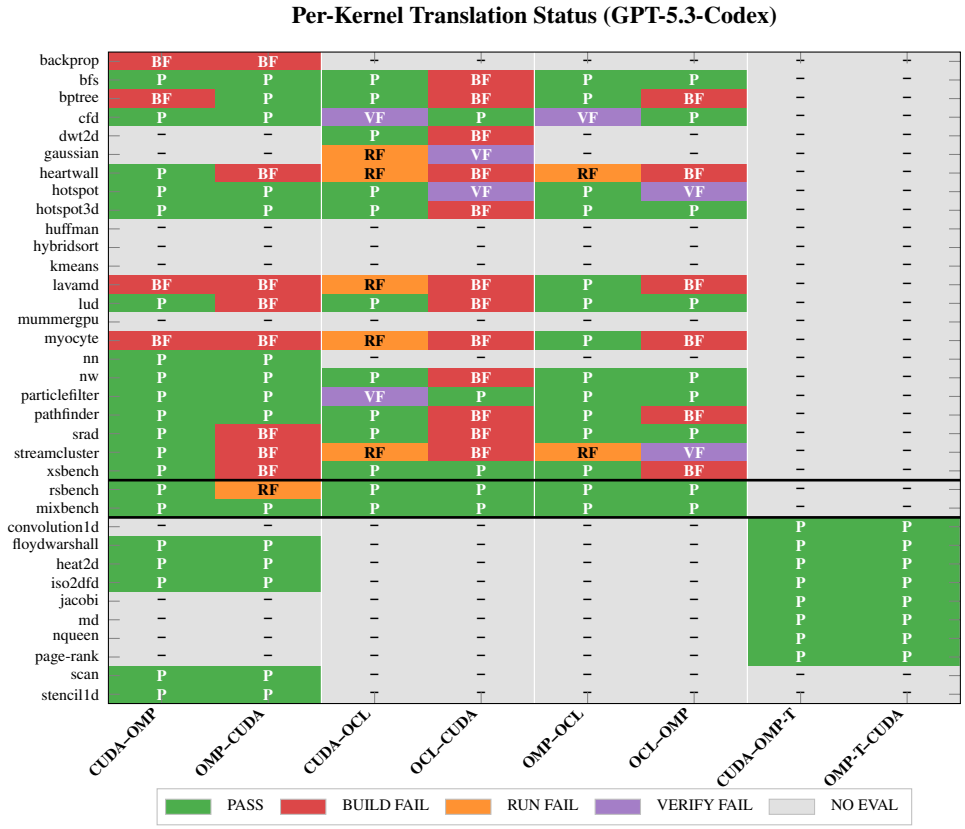

Figure 26: Per-kernel translation status across all directions (GPT-5.3-codex, L0, first sample per task, 31 evaluated kernels $\times$ up to 8 directions). The corresponding Qwen 3.5 data is reported in Table 20; GPT-5.4 heatmap in Figure 24.

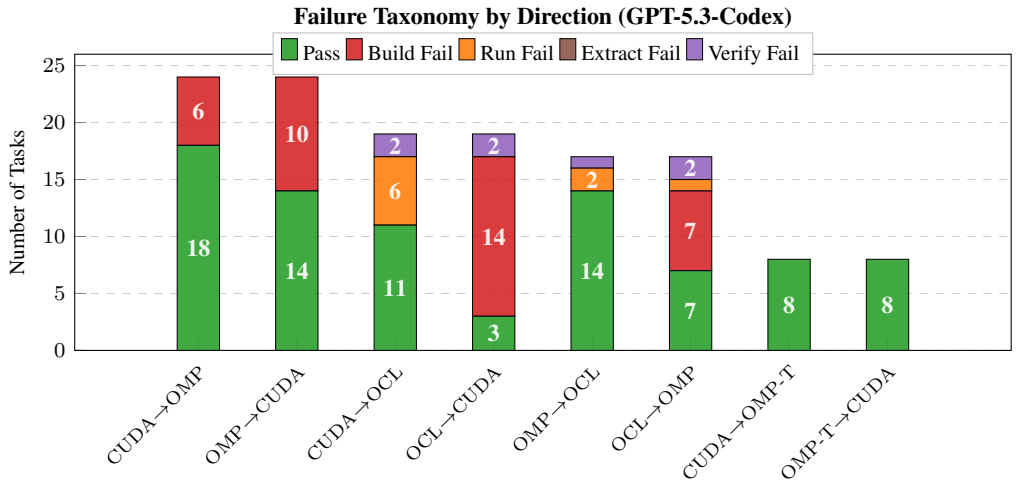

Figure 27: Failure taxonomy by translation direction (GPT-5.3-codex, L0, first sample per task, 136 tasks across 8 directions). Compare with Figure 2 (Qwen 3.5) and Figure 25 (GPT-5.4).

records (canonical L0 and augmentation). The three campaigns ran sequentially on a single GPU workstation over a nine-day window.

Table 23: Evaluation campaign cost summary. All metrics are aggregated from per-result JSON metadata. Token counts reflect only the tokens recorded per evaluation call (see text).

Metric	**Qwen 3.5**	**GPT-5.4**	**GPT-5.3-codex**
Tasks evaluated	708	822	814
Total tokens	13.6M	14.9M	13.2M
Input	8.9M	9.3M	9.2M
Output	4.7M	5.6M	4.0M
Avg. tokens per task	19,149	18,138	16,204
Total LLM wall time (h)	14.0	27.4	11.9
Campaign dates (2026)	Apr 21–24	Apr 25–27	Apr 30–May 1
All models	2,344 tasks, 41.7M tokens, 53.4 h total		

Per-result token counts undercount total provider-billed consumption because they exclude system-prompt overhead, HTTP-level retries, and provider-side formatting tokens. Actual billed totals may substantially exceed the reported figures. Researchers budgeting a replication should consult current provider pricing and account for this overhead. The Qwen 3.5 task count includes 82 records involving KNOWN_FAIL specs that are excluded from pass-rate denominators (Section C.5); the GPT-5.4 and GPT-5.3-codex batches pre-excluded such specs, so all on-disk records are valid.

L Evaluation Card

Following Model Cards Mitchell et al. [2019] and Datasheets Gebru et al. [2021], we provide an *Evaluation Card* for PARBENCH that states what it measures, what it does not measure, the claims it supports, the assumptions it relies on, and how results should be reported. Table 24 summarizes the scope; the subsections below justify each point.

L.1 What PARBENCH Measures

PARBENCH measures the following properties of LLM-based parallel code translation:

1. **Binary functional correctness of kernel-level translation.** The harness evaluates whether LLM-translated code compiles, executes, and produces output matching the spec's verification strategies. Verification applies a conjunction of strategies: all declared checks (exit code, stdout pattern, and optionally numeric comparison or file hash) must pass for a record to receive PASS status. The result is PASS or one of four failure statuses (EXTRACT_FAIL, BUILD_FAIL, RUN_FAIL, VERIFY_FAIL); timeouts are classified under the relevant pipeline stage (BUILD_FAIL or RUN_FAIL).
2. **Translation capability across three primary APIs and ten directions.** CUDA, OpenMP, and OpenCL form six bidirectional standard directions; OpenMP Target adds four case-study directions. The evaluation matrix covers 142 unique source–target pairs (Section 2.2).
3. **Failure mode taxonomy.** Four failure classifications (EXTRACT_FAIL, BUILD_FAIL, RUN_FAIL, VERIFY_FAIL) enable diagnostic analysis of *where* in the pipeline translations fail. Build failures indicate incomplete API-surface adaptation; run failures indicate runtime errors or timeouts; verify failures indicate incorrect output from otherwise executable code. This granularity distinguishes PARBENCH from compile-only or text-similarity evaluations.
4. **Robustness to source-code surface-form perturbation.** Six AST-level transforms (SwapCondition, ArithmeticTransform, ChangeNames, TypedefExpansion, PointerArithmeticToArrayIndex, ChangeFunctionNames) at four intensity levels (L1–L4) test whether pass/fail status is stable under semantics-preserving code changes (Section 2.3).
5. **Direction asymmetry.** Pass-rate variation across translation directions quantifies structural difficulty differences between API pairs. CUDA-to-OpenMP consistently passes at higher rates than OpenCL-to-CUDA across all three evaluated models (Section 5.3).

Table 24: Evaluation Card summary. Each row states a key property of PARBENCH's measurement scope. Detailed discussion appears in the referenced subsection.

Dimension	Key Points	Ref.
What PARBENCH *measures*		
Functional correctness	Binary pass/fail via conjunctive build→run→verify pipeline	§L.1
Failure taxonomy	Four failure classifications (EXTRACT_FAIL, BUILD_FAIL, RUN_FAIL, VERIFY_FAIL) plus PASS	§L.1
Direction asymmetry	Pass-rate variation across 10 translation directions (3 APIs + OMP target)	§L.1
Augmentation robustness	Stability under six AST-level semantics-preserving transforms at four levels	§L.1
What PARBENCH *does not measure*		
Performance	No speedup, throughput, occupancy, memory, or energy measurement	§L.2
Numerical accuracy	80 of 87 non-KNOWN_FAIL specs use weak oracles (exit code + stdout pattern)	§L.2
Code quality	No readability, maintainability, or style assessment of translations	§L.2
Repo-level translation	Kernel-centric by design; host code and build infra remain fixed	§L.2
Self-repair capability	Single-attempt evaluation; iterative feedback not exercised	§L.2
Validity and assumptions		
Valid claims	Pass@k, failure taxonomy, direction asymmetry, augmentation robustness (scoped)	§L.3
Invalid claims	"Efficient code," "production-ready," "understands parallelism," "not memorizing"	§L.4
Key assumptions	Source correctness, oracle sufficiency (weakest), single-platform generalization	§L.5
Guidance		
User guidance	Adding models/kernels, interpreting results, comparing models, common pitfalls	§L.6
Reporting protocol	Required and recommended items for publications using PARBENCH	§L.7

6. **Cross-model discrimination.** The benchmark distinguishes models that differ in translation capability, subject to sampling-condition caveats (Section L.5). In the current evaluation, pairwise analysis on 142 shared tasks yields Cohen's $h \approx 0.8$ between Qwen 3.5 and both GPT models ($h = 0.80$ for GPT-5.4, $h = 0.77$ for GPT-5.3-codex), while GPT-5.4 and GPT-5.3-codex are statistically indistinguishable ($h = -0.031$).
7. **Per-kernel difficulty heterogeneity.** For Qwen 3.5, pass rates range from 84.2% (iso2dfd) to 0% (10 kernels), enabling fine-grained analysis of which computational patterns—stencils, graph traversal, ODE integration, pointer-heavy data structures—are harder to translate (Appendix Table 20).

L.2 What PARBENCH Does Not Measure

The following properties are explicitly outside PARBENCH's measurement scope:

1. **Performance and resource efficiency.** No speedup, throughput, occupancy, bandwidth, memory, or energy measurement is performed. Wall-clock time is captured but is unreliable for sub-millisecond baselines. No kernel-level profiling (`ncu`/`nsys` for CUDA, `omp_get_wtime` for OpenMP) is included. A functionally correct translation that runs 100× slower than the original receives PASS. TRACY Gong et al. [2025] addresses this gap.
2. **Code quality, readability, or maintainability.** Translated code is stored in result files but no AST analysis, cyclomatic complexity, or style metrics are computed. A correct but unreadable translation receives PASS.
3. **Partial correctness or correctness gradations.** Verification is binary: a numerical result that deviates from the reference by 0.02% beyond the configured tolerance receives VERIFY_FAIL, identical to a completely wrong output. There is no "partially correct" classification.
4. **Numerical accuracy for most specs.** Only 7 of 87 non-KNOWN_FAIL specs (8%) have medium or strong oracles (`numeric_comparison` or `file_hash`). The remaining 80

specs (92%) rely on `exit_code` and `stdout_pattern`, which verify that the program runs and prints expected banners but do not independently validate numerical results. A translation that computes wrong numbers but prints the correct completion message would receive PASS on those specs. The oracle strength distribution across all 206 specs is: 2 strong (`file_hash`), 5 medium (`numeric_comparison`), 46 weak (`stdout_pattern` + `exit_code`), and 153 untagged (effectively weak).

5. **Iterative self-repair or agentic translation.** Each sample is a single LLM call with no feedback loop. The framework architecture supports self-repair (Figure 1), but this capability is not exercised in the current evaluation (Section C.5).
6. **Cross-platform portability.** All evaluations run on a single platform: NVIDIA RTX 4070 (sm_89), AMD Ryzen 9 7900X, Ubuntu 24.04, HPC SDK 24.3 (Appendix Table 16). No testing on other GPU architectures (A100, H100), CPU-only environments, or other OS configurations.
7. **Repository-level translation.** By design, PARBENCH isolates kernel translation from build-system reconstruction. Host code, Makefiles, and I/O routines remain fixed. The benchmark does not measure the LLM's ability to restructure project files, generate build systems, or coordinate multi-component applications (cf. ParEval-Repo Davis et al. [2025]).
8. **Training data memorization (definitively).** Augmentation tests surface-form robustness, not algorithmic memorization (Section L.4).

L.3 Valid Claims

The following conclusions can be drawn from PARBENCH results when accompanied by the stated conditions:

1. **"Model X achieves Y% pass@k on kernel-centric parallel code translation across Z directions."** Valid when: the exact pass@k metric is specified (pass@1 vs. pass@3), augmentation levels are stated (L0 only vs. L0–L4), directions are enumerated, and confidence intervals are included.
2. **"Build-stage failure is the dominant failure mode for Model X."** Valid when: the failure taxonomy breakdown is reported with counts and percentages from the full record set.
3. **"Direction A→B is harder/easier than Direction C→D for Model X."** Valid when: backed by per-direction pass rates with Wilson confidence intervals. Claims of statistical significance should use McNemar's test on paired kernels with appropriate multiple-comparison correction.
4. **"Model X's pass rate is stable under source augmentation at levels L1–L4."** Valid when: the L0-conditional filter is disclosed, the qualifying subset size is stated, the direction restriction is noted, and the analysis is presented as descriptive rather than a formal test of memorization.
5. **"Model X discriminates from Model Y on PARBENCH."** Valid when: (a) all compared models are evaluated on the same task set, (b) McNemar's paired test is reported with effect size and concordance table, (c) sampling-configuration differences are explicitly disclosed if temperatures or reasoning modes differ, and (d) the claim is scoped to "PARBENCH tasks," not "parallel translation in general."
6. **"Kernel K is harder to translate than Kernel J."** Valid when: both kernels have sufficient sample sizes ($n \geq 18$ recommended) and confidence intervals are reported.

L.4 Invalid Claims

The following conclusions **cannot** be drawn from PARBENCH results:

1. **"Model X produces efficient/fast parallel code."** Performance is not measured. A PASS means the code compiles, runs, and produces correct output—not that it runs at acceptable speed.
2. **"Model X's translations are production-ready."** No code review, security audit, or race-condition detection is performed beyond what `exit_code` and `stdout` checks catch. Parallel code can have latent data races that produce correct output on some executions.

3. **"Model X understands parallel programming."** PARBENCH measures behavioral outcomes, not internal representations. A model could pass by pattern matching without "understanding" synchronization semantics.

4. **"PARBENCH pass rate generalizes to all HPC translation tasks."** The corpus contains 87 non-KNOWN_FAIL specs from five suites, with Rodinia contributing 53 (61%). Domains not represented (e.g., distributed-memory MPI, GPU tensor operations, FPGA HLS) are not covered.

5. **"Model X is definitively better than Model Y at parallel translation."** Only valid under matched sampling conditions. If temperatures, reasoning modes, or provider-side controls differ (as they do between Qwen 3.5 and the Azure models in this paper), the observed gap reflects an unknown mixture of model capability and sampling configuration (Section C.5).

6. **"Augmentation robustness proves the model is not memorizing."** Surface-form augmentation (variable renaming, syntax sugar) tests robustness to cosmetic code changes. Algorithmic memorization—where the model recognizes the computation and produces a known-correct translation—is not tested by these transforms.

7. **"A PASS means the translation is numerically correct."** For the 80 of 87 specs with weak oracles, verification checks that the program ran and printed expected output banners. Numerical results are not independently validated. Only 7 specs have medium or strong oracles that verify numerical output.

8. **"OpenCL→CUDA is impossible for LLMs."** Qwen 3.5 achieves 0% but both GPT-5.4 and GPT-5.3-codex achieve 19.3% on the same direction. Zero-rate results for a single model reflect that model's capability, not an inherent impossibility.

L.5 Assumptions

PARBENCH's evaluation results are meaningful under the following assumptions. We note mitigations and residual risks for each.

1. **Source implementations are functionally correct.** PARBENCH treats the original benchmark implementations as ground truth. If a source implementation has a latent bug, faithful translations of that bug would pass verification. *Mitigation:* all 87 non-KNOWN_FAIL specs pass the baseline build/run/verify pipeline on the reference platform.

2. **Verification oracles are sufficient to detect incorrect translations.** This is the weakest assumption. For the 80 specs with weak oracles, a translation that produces wrong numerical results but correct program flow (runs to completion, prints expected banners) would receive a false PASS. *Mitigation:* 7 specs have `numeric_comparison` or `file_hash` oracles; the dominant failure mode (BUILD_FAIL) is caught by all oracle types regardless of oracle strength. Upgrading weak oracles is identified as future work (Section 6).

3. **The single evaluation platform generalizes to other NVIDIA GPU configurations.** All results are from one machine (RTX 4070, sm_89). Different GPU architectures may have different compiler behavior, runtime characteristics, or memory limits.

4. **Three samples per task capture meaningful stochastic variance under each provider's sampling regime.** For Qwen 3.5 this uses temperature 0.7; for GPT-5.4 and GPT-5.3-codex sampling is provider-controlled (Section C.5). The pass@1-to-pass@3 gap (11.3 pp for Qwen 3.5, 7.0 pp for GPT-5.4, 5.6 pp for GPT-5.3-codex) suggests moderate within-task variance. More samples would provide tighter estimates at proportional cost.

5. **Kernel-centric translation isolates parallel programming capability.** By fixing host code and build infrastructure, PARBENCH prevents build-system reconstruction from dominating results (cf. ParEval-Repo's 0% at >133 SLoC Davis et al. [2025]). However, this also means that a model's ability to handle multi-component integration—a real-world requirement—is explicitly not tested.

6. **The 9 KNOWN_FAIL exclusions are legitimately infrastructure failures.** Each exclusion is documented with a specific technical cause (CUDA 12 API deprecation, missing libraries, pre-existing runtime errors). None are excluded for being "too hard" for LLMs.

7. **Benchmark suites are representative of real HPC workloads.** The five suites are well-established in HPC research (Rodinia: IISWC 2009 Che et al. [2009], HeCBench: 2023 Jin and Vetter [2023], XSBench: ANL Tramm et al. [2014]). However, they over-represent structured stencil and reduction patterns and under-represent irregular computations, multi-physics coupling, and modern GPU programming idioms (tensor cores, cooperative groups).
8. **Augmentation transforms preserve semantics.** Each transform is backed by `libclang` AST analysis and validated by 15 unit tests. 80 of 87 specs pass all L1–L4 levels. The 7 failures at L3–L4 are `omp_target`-specific (aggressive variable renaming interferes with data-mapping clauses), a known limitation.
9. **Prompt anonymization does not change task difficulty.** Stripping comments, genericizing filenames, and removing kernel names reduce memorization cues but might also remove helpful context, potentially making PARBENCH results conservative relative to non-anonymized use.

L.6 User Guidance

Adding a new model. Register the model in the evaluation pipeline's model registry, specifying provider, API model identifier, and thinking/temperature configuration. Run the canonical campaign with all suites; the evaluation pipeline handles prompt construction, LLM calls, and file extraction, and the harness handles build/run/verify.

Adding new kernels. Write a spec JSON following the schema documented in Appendix C.1. The spec must define identity (unique ID, API, suite), file partitioning (prompt payload, translation targets, support files), build commands, run configuration, and verification strategies. Validate the baseline by running the harness verify command. Append to the manifest (append-only).

Interpreting pass rates. Always report the denominator (number of valid records after KNOWN_FAIL exclusion), the augmentation level (L0 vs. L0–L4), and Wilson 95% confidence intervals. Per-direction breakdowns are essential—aggregate rates conceal strong direction asymmetry.

Comparing models. Use paired tests on task outcomes (same tasks, same direction–kernel pairs): Fisher's exact test for pairwise comparisons, McNemar's test for concordance analysis. Report concordance tables (both-pass, both-fail, discordant). Always disclose temperature, reasoning mode, and any provider-side sampling controls. If sampling conditions are unmatched, state this explicitly and scope the comparison accordingly.

Augmentation analysis. Use the L0-conditional filter: include a pair in the ablation only if ≥ 1 of its L0 samples passes. Report the filter rate (fraction of pairs qualifying). Results apply only to the qualifying subset. Single samples per augmentation level (L1–L4) trade statistical precision for budget efficiency.

Common pitfalls.

- Do not compare aggregate pass rates across models without disclosing sampling conditions.
- Do not claim "the model understands parallel programming" from pass rates alone.
- Do not interpret weak-oracle PASS as evidence of numerical correctness.
- Do not modify the KNOWN_FAIL exclusion policy without documenting the change and its effect on denominators.
- Do not change spec run arguments without reading the source code's `argc` check—a documented cause of past silent failures.

L.7 Recommended Reporting Protocol

When publishing results obtained with PARBENCH, we recommend reporting at minimum the items in Table 25. For cross-model comparisons and augmentation analyses, the additional items in Table 26 strengthen interpretability.

Example minimal results paragraph. "We evaluated [Model] on PARBENCH's 142 kernel translation tasks across 10 directions (6 standard + 4 OMP-target) under [sampling config] with n = [N] samples. After excluding 9 KNOWN_FAIL specs, [M] valid records remain. Overall pass@1 = X% [CI];

Table 25: Required reporting items for PARBENCH results.

Item	What to report
Model identity	Provider, model ID, version or date
Sampling config	Temperature/top-p if caller-controlled, otherwise "provider-controlled"; reasoning mode; n (samples)
pass@k	pass@1 and pass@3 with confidence intervals
Failure taxonomy	Counts and percentages for each failure type
Direction breakdown	Per-direction pass rates with Wilson CIs
Denominator	Total valid records after KNOWN_FAIL exclusion
Augmentation scope	Levels evaluated, filter criterion, subset size
Platform	GPU, CPU, OS, compiler versions

Table 26: Recommended additional reporting items.

Item	What to report
Cross-model comparisons	
Paired test	McNemar on paired tasks with concordance table
Effect size	Cohen's h for overall and per-direction gaps
Sampling caveat	Explicit statement if conditions are unmatched
Task variance	Fraction of hard-fail (0/n), noisy, always-pass (n/n)
Augmentation analysis	
Per-level rates	Descriptive rates at L0–L4 on balanced subset
Filter disclosure	"X of 142 pairs qualify (Y%)"
Direction scope	Which direction(s) the analysis covers
Power analysis	Detectable effect size at 80% power

pass@3 = Y% [CI]. Build-stage failure accounts for Z% of all failures. CUDA-to-OpenMP is the most tractable direction (A% [CI]); OpenCL-to-CUDA is the hardest (B% [CI]). Augmentation analysis on [K] qualifying kernels shows pass rates of [range]% at L1–L4 versus [L0 rate]% at L0."

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract states L0 pass rates (Qwen 23.9%, GPT-5.4 62.7%, GPT-5.3-codex 62.7%), failure taxonomy percentages, and augmentation robustness findings, all verified against raw result files (Section 5). Claims are scoped to the three models evaluated.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Section 6 includes a dedicated limitations subsection addressing: three-model scope, corpus coverage, single-GPU evaluation platform, weak oracle coverage, unmatched sampling conditions across providers, and the absence of performance (runtime) evaluation.

3. **Theory**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete proof?

   Answer: [N/A]

   Justification: This is an empirical benchmark paper with no theoretical results.

4. **Experiments**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper?

   Answer: [Yes]

   Justification: Section 4 and Appendix C provide model configurations (Table 15), hardware specifications (Table 16), the complete prompt template (Appendix H), and augmentation level definitions (Table 8). The evaluation pipeline is deterministic given a fixed spec and model response.

5. **Code and data**

   Question: Does the paper provide instructions for reproducing the main experimental results, including code and data?

   Answer: [Yes]

   Justification: ParBench is included as anonymized supplementary material containing all 206 kernel specifications, the build-run-verify harness, the evaluation pipeline, augmentation engine, and per-record result JSONs for all three models.

6. **Experimental details**

   Question: Does the paper specify all the training and test details necessary to understand the results?

   Answer: [Yes]

   Justification: No training is performed. Evaluation details are specified: temperature (0.7 for Qwen 3.5; provider-controlled for GPT-5.4 and GPT-5.3-codex), number of samples (3), augmentation levels (L0–L4), KNOWN_FAIL exclusion policy (9 specs), and the L0-conditional ablation design.

7. **Error bars**

   Question: Does the paper report error bars suitably and correctly?

   Answer: [Yes]

   Justification: All pass rates include Wilson 95% confidence intervals. Direction asymmetry uses McNemar's exact test with Bonferroni correction ($\alpha = 0.01$). Effect sizes reported via Cohen's h. Cross-model comparison uses McNemar's paired exact test.

8. **Compute**

   Question: For each experiment, does the paper provide sufficient information on the computer resources used?

   Answer: [Yes]

   Justification: Table 16 specifies the evaluation platform (RTX 4070, Ryzen 9 7900X, Ubuntu 24.04, HPC SDK 24.3). Appendix K reports token usage and wall-clock time for all three model campaigns.

9. **NeurIPS code of ethics**

   Question: Does the research conform to the NeurIPS Code of Ethics?

   Answer: [Yes]

   Justification: The work evaluates publicly available benchmark code using commercial LLM APIs. No human subjects, private data, or dual-use concerns.

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work?

    Answer: [Yes]

    Justification: Section 6 discusses implications for HPC code migration and identifies the limitation that LLM-generated parallel code should not be trusted without verification. The benchmark itself enables systematic measurement of translation quality, which supports responsible deployment.

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models?

    Answer: [N/A]

    Justification: ParBench releases benchmark specifications and evaluation infrastructure, not trained models or generated datasets. The benchmark code originates from established open-source HPC suites.

12. **Licenses**

    Question: Are the creators of assets used in the paper properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes]

    Justification: All five benchmark suites are cited (Rodinia Che et al. [2009], XSBench, RSBench, mixbench, HeCBench). Repository URLs and commit hashes are recorded in kernel specifications. LLM providers (Together AI, Azure OpenAI) are identified. Rodinia is distributed under the Rodinia license (BSD-like); XSBench and RSBench under MIT; HeCBench under MIT; mixbench under GPL-3.0. All are compatible with academic use and redistribution.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

    Justification: ParBench's specification schema is documented (Section 2.1, Appendix C.1). The augmentation engine, evaluation pipeline, and harness are described in Section 2. A datasheet following Gebru et al. [2021] will accompany the camera-ready release The availability plan, hosting, and license summary are provided in Appendix A (Artifact Availability).

14. **Crowdsourcing and human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants?

    Answer: [N/A]

Justification: No crowdsourcing or human subjects research.

15. **IRB approvals**

Question: Does the paper describe potential risks incurred by study participants?

Answer: [N/A]

Justification: No human subjects research.

16. **LLM usage declaration**

Question: Does the paper disclose the use of LLMs in the core methodology?

Answer: [Yes]

Justification: Three LLMs (Qwen 3.5 397B-A17B, GPT-5.4, and GPT-5.3-codex) are the primary subjects of evaluation. Model identifiers, providers, and configurations are fully disclosed in Table 15 and Section 4.